

数据驱动的世界杯观看预测、赛程与动态资源协同优化

摘要

本文研究 48 支球队、12 个小组、72 场比赛条件下的世界杯观看人数预测、场馆与时段协同排程、前两轮反馈下的第三轮资源重配及真实赛事对照。四问不是独立任务，而是“预测需求—固定赛程—动态更新—外部检验”的数据链。针对问题一，构造仅含赛前信息且交换球队次序不变的概率、实力、球迷、球星、市场和区域特征，以按日期扩展窗口验证 7 类模型；岭回归的平均 MSE 为 225.0601（单位为百万人平方），较均值基线降低 61.32%，故用于生成 140 条测试集预测和 72 场待排比赛预测。针对问题二，将六类赛事规则写成 0-1 混合整数模型；受本地 COPT 评估版 2000 变量限制，采用“可行基准场馆 + 两个安保合格的大容量场馆 + 同参考日全部时段”的可审计候选分解，并在该集合内精确线性化完整 Z_2 。COPT 以 0 间隙得到 $Z_2 = 0.4581636504$ ，较固定可行基线 0.3187898738 提高 43.72%，求解目标与独立评估器误差小于 4×10^{-15} ；在 10000 组权重扰动下改进率为 100%。针对问题三，按统一信息截面同时进行 20000 次泊松模拟，严格执行“各组前二 + 成绩最好的 8 个第三名”晋级规则，再以共同候选集分别求解静态和动态 COPT 模型。动态方案改变 10 场资源或票价决策，更新环境总目标由 6.6327957655 升至 6.6521944982，提高 0.29247%；相对 50000 次模拟的晋级概率 MAE 为 0.001416。针对问题四，从 OpenFootball 采集并解析已结束的 2026 年美加墨世界杯 72 场小组赛，与优化赛程同规模，按每队、每次转场和场馆份额构造同口径指标。实际赛程每队旅行 2058.63 km、每队跨时区 0.625 次，低于优化赛程的 4257.17 km 和 1.458 次；优化赛程平均休息 167.81 h、最小休息 159.00 h、平均容量 71353 人，分别优于实际赛程的 133.42 h、91.00 h 和 68292 人，说明区域化分组是控制跨国旅行成本的关键，而优化方案以旅行和时区负担换取更长休息与更大容量。全部输出均经模板列序、单位、约束、来源和哈希验收。

关键词： 足球世界杯；岭回归；混合整数规划；COPT；蒙特卡洛模拟；动态资源配置

一、 问题重述

1.1 问题背景

大型足球赛事同时受竞技不确定性、商业需求、场馆能力、跨时区旅行和有限公共资源影响。题目给出历史比赛、球队、72 场未来对阵、16 个场馆、80 个时段及赛后反馈，要求建立一条前后口径一致的模型链：先预测观看需求，再确定赛程，继而在赛程固定时依据前两轮信息动态重配资源，最后与一届真实世界杯比较。

1.2 四项任务及其耦合

问题一输出是问题二的转播需求输入；问题二确定的第三轮场馆和时段在问题三中不可改变；问题三只允许在统一的信息截面下更新资源；问题四必须沿用问题二的指标方向和标准化思想。图1给出整体流程。

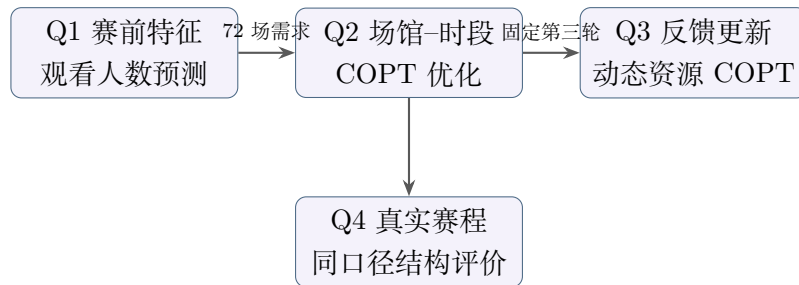


图 1 四问的数据与决策传递关系

二、 问题分析

问题一的核心风险是赛后信息泄漏和随机划分导致的过乐观验证；因此只保留赛前可得、对调主客顺序不变的特征，并按时间扩展验证。问题二是大规模组合优化，必须先建立独立评估器和硬约束检查器，再在 COPT 评估版容量内做候选分解。问题三的关键是模拟 24 场第三轮比赛时不得逐场更新，以及静态/动态方案必须用同一更新环境和归一化边界比较。问题四存在赛事规模、地理形态与公开字段不可比问题，因此仅对能从两套赛程同口径识别的结构量做规模标准化，不虚构历史商业变量。

三、 模型假设

1. 附件中的赛前赔率、Elo、球队属性与基础预测在决策时可获得；测试集真实观看人数不可见。
2. 给定题设泊松均值后，第三轮各场进球在一次模拟内条件独立；24 场均由第三轮开始前同一信息截面生成。
3. Q2 受限候选集中未保留的场馆-时段组合不参与 COPT 证明，故整数最优性仅限该集合；最终赛程仍必须通过全部原始硬约束。
4. Q4 的旅行仅计相邻比赛场馆间位移，不计球队驻地至首场馆；两届赛事的商业与风险字段无法从公开赛程同口径恢复，故不参与数值比较。

四、符号说明

表 1 符号说明

符号	含义	单位
i, v, s, t	比赛、场馆、时段与球队索引	—
x_{ivs}	比赛 i 选择场馆 v 与时段 s 的 0-1 变量	—
T, B, U, H	票务、转播、悬念与吸引力指标	—
C, D, F, R	运营成本、旅行、公平惩罚与执行风险	—
s_t, h_t	球队状态与伤病影响	—
$\lambda_{i,a}, \lambda_{i,b}$	第三轮比赛双方泊松进球均值	球
p_t, Q_i, A_i	晋级概率、晋级重要性与更新吸引力	—
δ_i	票价相对调整比例	—
Z_2, Z_3	赛程与动态资源综合目标	—

五、数据审计与预处理

附件含 15 个工作表。历史比赛共 700 行，题设固定 560 行为训练集、140 行为隐藏测试集；未来赛程为 48 队、12 组、72 场。我们完成字段类型、唯一键、球队映射、概率正值、时间时区和模板列序检查。训练目标除以 10^6 转为“百万人”，而未来比赛提交值再乘 10^6 恢复“人”。所有输出均从官方模板字段生成，避免手工表头。

对赔率使用归一化隐含概率

$$p_k = \frac{1/o_k}{\sum_{j \in \{a,d,b\}} 1/o_j}, \quad E_p = -\frac{\sum_k p_k \ln p_k}{\ln 3}. \quad (5.1)$$

其中 E_p 越大表示三种赛果越接近。球队两侧变量采用均值、绝对差、和、最大值或排序后的类别对，保证交换 a, b 不改变预测。完整 18 个特征均有题面依据：比赛阶段；概率及熵；Elo 均值与差、排名均值；球迷、球星、市场总量；足联与时区区域组合。进球、xG、射门、控球、现场观众、当场观看人数和比赛编号均被禁止进入特征矩阵。

六、问题一：转播观看人数预测

6.1 验证设计与候选模型

训练数据按日期构造 3 个扩展窗口，每折训练结束日严格早于验证开始日。比较均值基线、岭回归、弹性网络、RBF-SVR、随机森林、极端随机树和直方图梯度提升。数值特征在每一折内标准化，类别特征在每一折内独热编码，从而不使用验证期分布

信息。主指标为

$$\text{MSE} = \frac{1}{n} \sum_{j=1}^n (y_j - \hat{y}_j)^2. \quad (6.1)$$

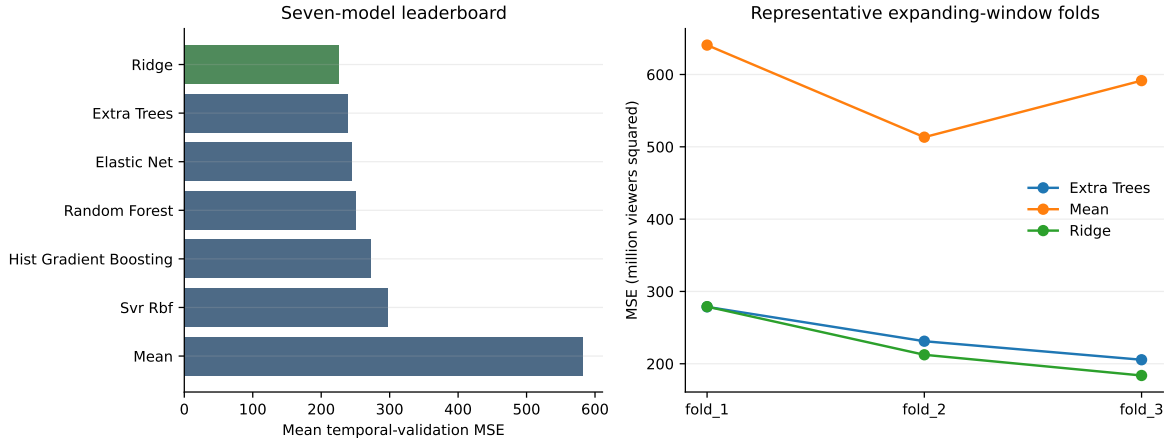


图 2 七类模型的时间验证结果；隐藏测试标签不参与选择

岭回归^[1]平均 MSE 为 225.0601，标准差 48.8524，平均 RMSE 为 14.9446，平均 MAE 为 11.8346；极端随机树 MSE 为 238.4691，均值基线为 581.8325。岭回归相对均值基线降低 61.32%，且三折 MSE 依次为 278.9919、212.4149、183.7736，故作为最终模型。该结论只说明其在当前时间验证协议和候选集中领先，不等同于隐藏测试最优。

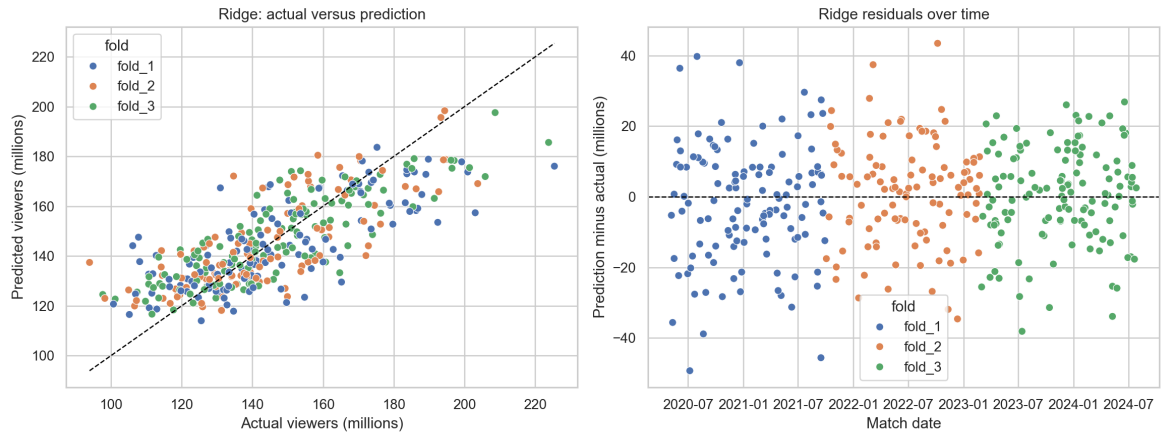


图 3 岭回归的验证期预测与时间残差

6.2 最终拟合与输出

在全部 560 条训练记录上重拟合岭回归，生成 140 条测试预测（百万人）与 72 场未来预测（人）。两文件重载后检查 ID 集合、行数、非负有限值和单位数量级；未使用隐藏测试真实值计算任何性能指标。

七、 问题二：场馆与开球时段协同优化

7.1 综合评价函数

按题面定义，所有量纲指标先做 Min-Max 归一化；极差为 0 时记为 0。目标为

$$\max Z_2 = 0.25T' + 0.25B' + 0.15U' + 0.10H' - 0.08C' - 0.07D' - 0.06F' - 0.04R'. \quad (7.1)$$

其中旅行负担以距离、旅行时间、时区差的标准化值按 0.5, 0.3, 0.2 加权；公平惩罚为黄金时段次数极差和大容量场馆次数极差各占一半；风险为气候、上座率、安保需求/能力按 0.5, 0.3, 0.2 加权。

7.2 0-1 模型与候选分解

完整组合为 $72 \times 16 \times 80$ 。本机 COPT 8.0.6 无商业许可证，单个 MIP 限 2000 变量，因此每场保留确定性可行基准的场馆、两个安保合格且容量最大的备选场馆，并保留基准参考日的全部候选时段，得到 824 个赋值候选。该选择只使用容量和安保等题设事实，不新增经验评分。

对每场恰选一次：

$$\sum_{(v,s) \in \Omega_i} x_{ivs} = 1, \quad \forall i. \quad (7.2)$$

其余约束依次为：同场馆同 UTC 时刻至多一场；同队轮次顺序与相邻比赛至少 60 h；场馆当地日和总场次上下界；任意两队黄金时段次数差不超过 2；安保能力不低于需求且第三轮每日高安保数量受限；同一时段比赛数不超过转播容量。引入场馆启用变量 y_v 计一次性成本，并用 $g^{\max}, g^{\min}, l^{\max}, l^{\min}$ 精确线性化两类资源次数极差。由此式(7.1)在受限候选集上成为线性目标。

7.3 求解与可执行性

COPT 模型^[2]含 840 个二元变量、865 个约束，约 0.46 s 在 20 个分支结点达到 0.0000% 间隙。目标 0.45816365037316586 与完全独立的赛程评估器 0.458163650373169 相差小于 4×10^{-15} 。固定可行基线为 0.3187898738，因此提高 43.72%。

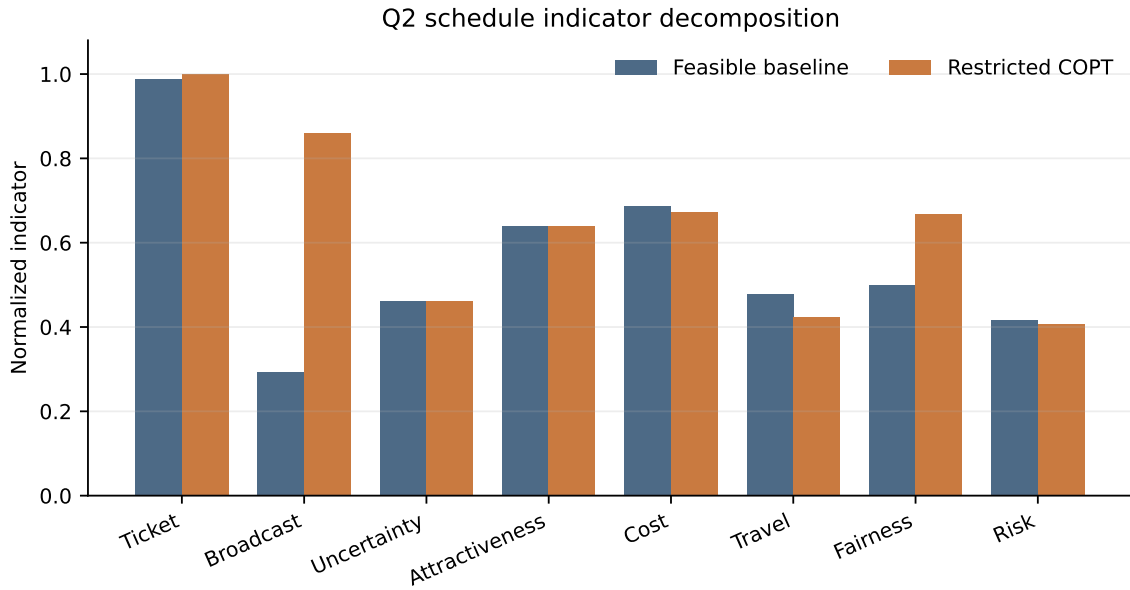


图 4 Q2 基线与受限候选 COPT 方案的八指标分解；成本型指标柱高不代表更优

最终赛程由独立函数重新验证六类硬约束，72 场均恰好一次，字段 `total_objective_value` 仅首行非空。必须强调：0 间隙证明受限候选集内整数最优，不证明未保留的全部 16×80 组合上的全局最优。

7.4 权重稳健性

将八个基准权重分别乘以 $U(0.8, 1.2)$ 后归一化，共抽样 10000 次^[3]。COPT 方案胜率为 100%，最小、5% 分位和中位优势分别为 0.105729、0.117677 和 0.139626。

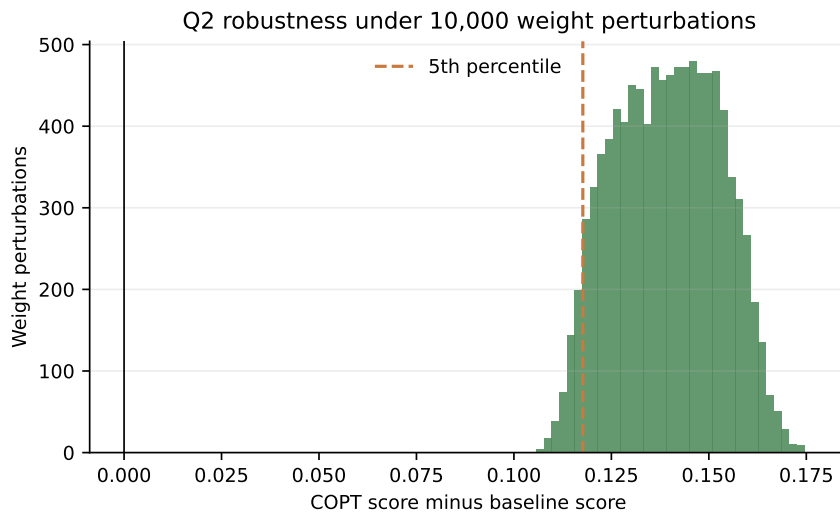


图 5 Q2 权重扰动下的目标优势分布

八、 问题三： 第三轮动态资源优化

8.1 状态、泊松模拟与晋级概率

对球队 t ，由前两轮积分、xG 和红牌构造

$$s_t = 0.45 \frac{pt_t}{3G_t} + 0.35 \left[0.5 + 0.5 \tanh \left(\frac{(xGF_t - xGA_t)/G_t}{1.25} \right) \right] + 0.20 \exp \left(-0.55 \frac{RC_t}{G_t} \right). \quad (8.1)$$

伤病 h_t 取两场平均并截断至 $[0, 3]$ 。令

$$\Delta_i = 0.32(s_a - s_b) - 0.055(h_a - h_b), \quad (8.2)$$

$$\lambda_{i,a} = \text{clip}[\lambda_{i,a}^0 e^{\Delta_i - 0.04h_a}, 0.15, 4.50], \quad \lambda_{i,b} = \text{clip}[\lambda_{i,b}^0 e^{-\Delta_i - 0.04h_b}, 0.15, 4.50]. \quad (8.3)$$

固定种子 20260814 同时模拟 24 场比赛 20000 次。每次先按积分、净胜球、总进球排每组名次，再从 12 个第三名中取 8 队；完全相同指标用独立连续随机键等比例打破。每次恰有 32 队晋级，故所有球队晋级概率之和也严格为 32。

晋级重要性、无激励风险和默契风险按题面计算：

$$Q_i = \frac{4p_a(1 - p_a) + 4p_b(1 - p_b)}{2}, \quad R_i^{\text{stake}} = \frac{1}{2}(2p_a - 1)^2 + \frac{1}{2}(2p_b - 1)^2. \quad (8.4)$$

由同组模拟统计双方在平局和各自取胜条件下的晋级概率，进而得到共同平局收益、胜负激励增量和 R_i^{col} 。图6显示大量球队已接近必然晋级或淘汰，也有一批球队处在 0.25–0.75 的高敏感区。

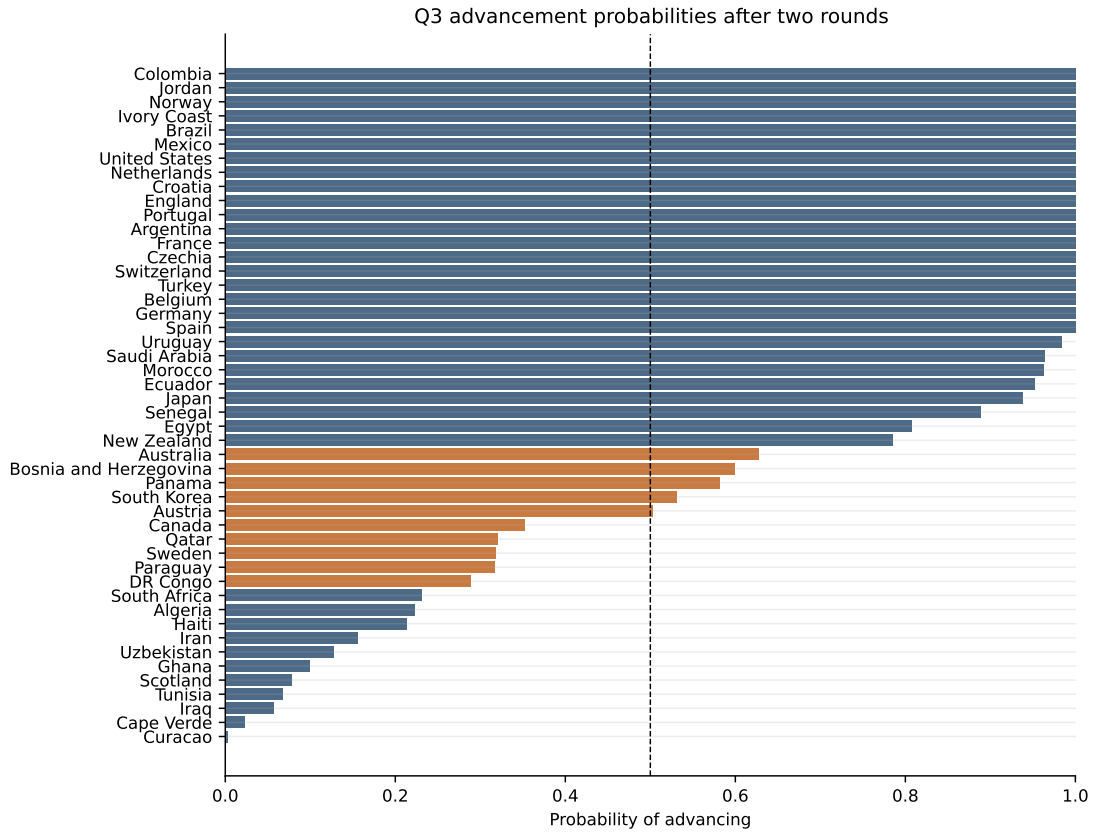


图 6 前两轮信息更新后的 48 队晋级概率

进一步把 24 场的晋级重要性与更新吸引力联合展示于图7。 Q_i 中位数为 0.2416, 有 4 场不低于 0.75; 更新吸引力范围为 $[0.3669, 0.6887]$, 二者相关系数为 0.8285。这说明晋级悬念是吸引力更新的重要来源, 但反馈、球队状态与伤病项仍保留独立影响。

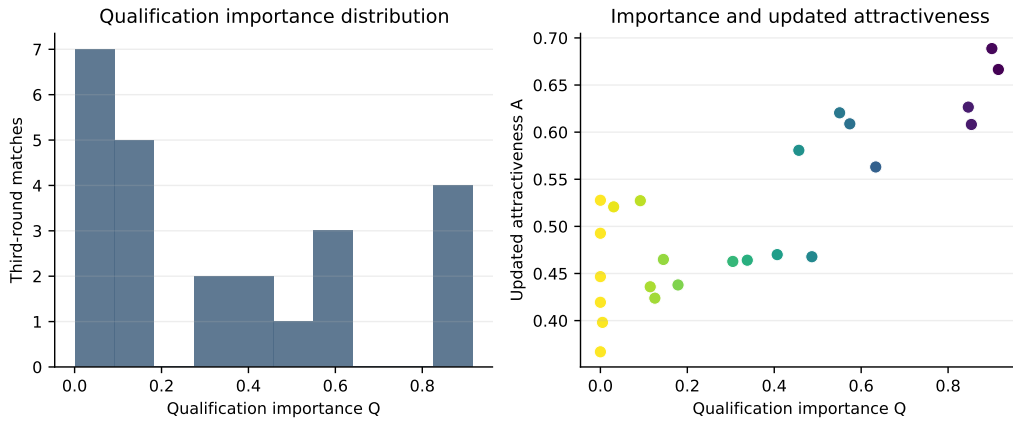


图 7 第三轮晋级重要性分布及其与更新吸引力的关系; 散点颜色表示无激励风险

8.2 需求更新与资源候选

将前两轮实际/预测现场比和转播比按球队平均并截断到 $[0.75, 1.25]$, 比赛反馈取双方几何均值。更新吸引力为

$$A_i = \text{clip}[0.50A_i^0 + 0.25Q_i + 0.12F_i + 0.08s_i + 0.05(1 - H_i), 0, 1]. \quad (8.5)$$

现场和转播需求严格采用题设乘法式。每场候选由 3 级转播、允许的安保等级、3 级交通和当日票价上下界间 5 点网格组成；若同一交通等级下涨价后的现场需求低于零调整需求的 88%，则删除该候选。五点离散化覆盖上下界、零附近和中间水平，避免引入题外变量。

8.3 静态与动态双 COPT 模型

对票务价值、转播价值、吸引力、资源成本和风险使用候选集 Min-Max 边界，目标为

$$\max Z_3 = \sum_{i \in I_3} (0.35TV'_i + 0.35BV'_i + 0.10A'_i - 0.10C'_i - 0.10R'_i). \quad (8.6)$$

静态模型在赛前环境求解；随后固定其四项决策，在更新环境下重评。动态模型在相同候选、每日容量、预算和更新边界下重求解。为检验极端情形，再在完全相同的更新候选与每日约束下最小化同一 Z_3 。三个模型各含 1623 个二元变量和 48 个约束，均达到 0 间隙且无约束违反。最小化模型给出可行下端 4.7962981525，正式动态最大化模型给出上端 6.6521944982，故同口径可行目标区间为 [4.7963, 6.6522]。静态赛前目标为 6.6797891851，固定决策的更新值为 6.6327957655；动态值为 6.6521944982，改善 0.29247%。

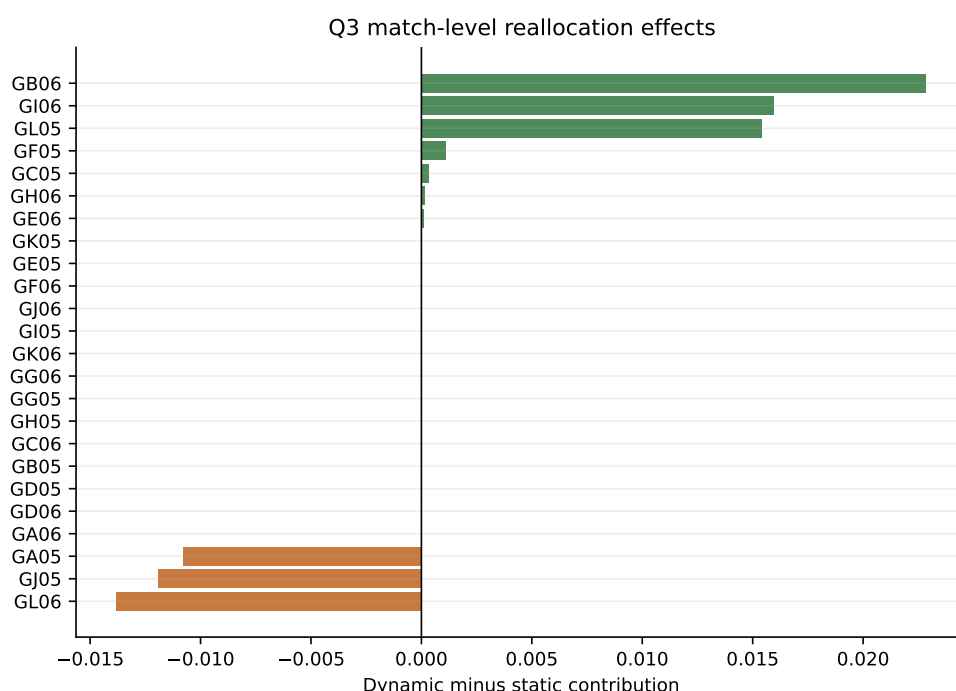


图 8 动态方案相对固定静态决策的逐场贡献变化

10 场至少改变一项资源或票价。7 场贡献增加、3 场减少并不矛盾：每日容量和预算使比赛相互耦合，动态模型会从边际收益较低的场次回收资源，因此应以全局和非逐场全正作为判据。

表2在同一更新环境下比较题目要求的总量。动态方案将资源成本指数降低 4.43%，同时令转播观看和转播价值提高 0.11%；现场和票务价值分别下降 0.78%，风险总量不

变，最终净效益提高 0.29247%。这说明模型不是同步放大全部投入，而是在每日耦合约束下将资源转向边际回报更高的比赛。

表 2 Q3 静态固定决策与动态决策的更新环境总量比较

指标	静态方案	动态方案	相对变化
预计现场观众/人	911765	904631	-0.78%
预计转播观众/十亿人次	3.2599	3.2636	+0.11%
票务价值/百万美元	111.9965	111.1235	-0.78%
转播价值/百万美元	169.5157	169.7066	+0.11%
资源成本指数	384	367	-4.43%
风险暴露指数	11.2426	11.2426	0.00%
净效益	6.6328	6.6522	+0.29247%

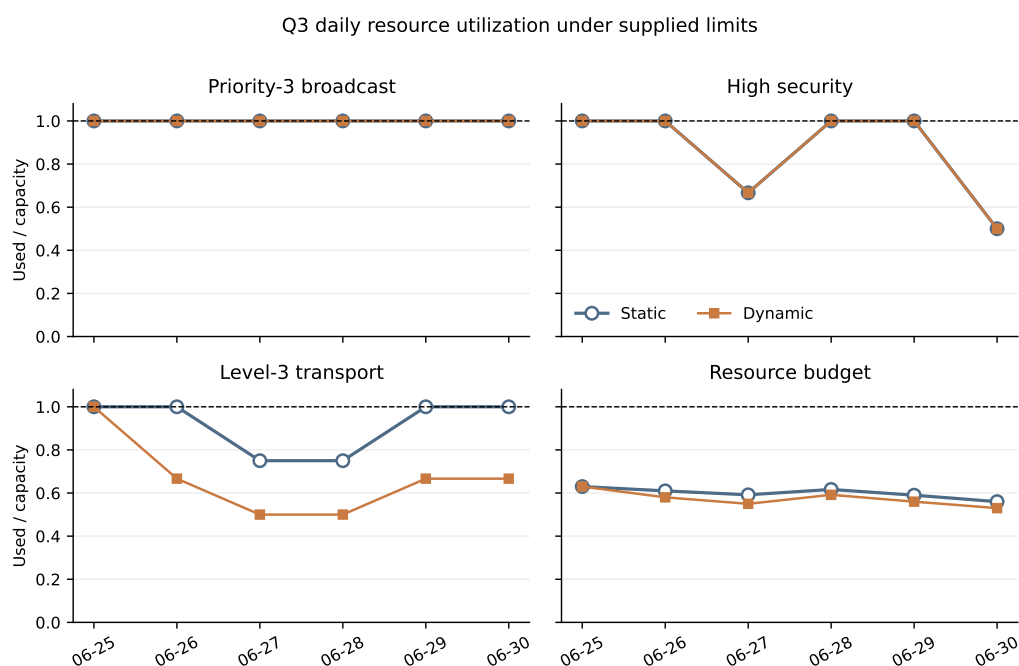


图 9 Q3 六个比赛日的静态/动态资源占用率；虚线为题面给定容量或预算上限

图9显示两方案在各日均未越过题面上限。动态方案从 6 月 26 日起减少 3 级交通配置，并使总资源成本由 384 降至 367。逐场改善率最大的是 GL05 (England–Panama)，转播优先级由 2 升至 3，更新转播观看由 1.1856 亿升至 1.2646 亿，净贡献由 0.07304 升至 0.08845，改善 21.09%。最小的是 GA05 (Mexico–Czechia)，转播优先级由 3 降至 2，成本由 13 降至 9，净贡献由 0.01123 降至 0.00046；其 -95.90% 主要源于静态分母很小，反映为其他场次释放稀缺高等级转播资源，而非全局退化。

8.4 蒙特卡洛收敛

在相同种子和算法下分别运行 5000、10000、20000 和 50000 次，以 50000 次为参考。20000 次的晋级概率 MAE 为 0.001416、最大绝对差 0.008530，默契风险 MAE 为 0.000358，且各规模晋级概率总和均为 32。

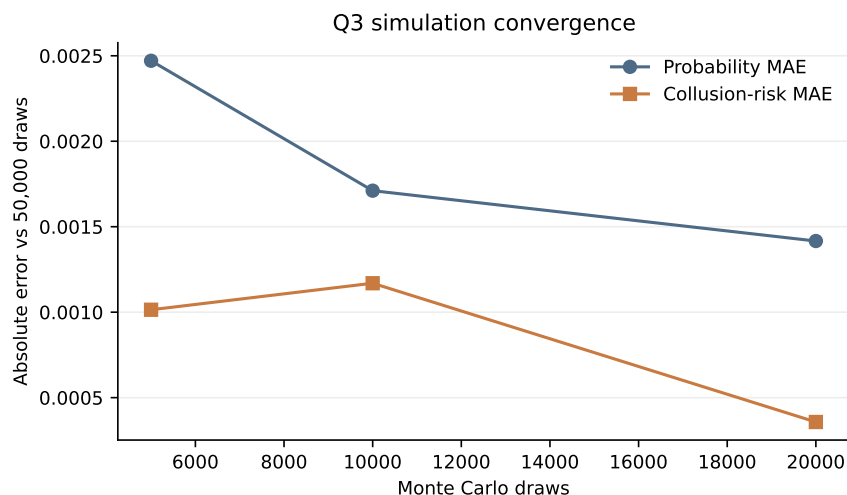


图 10 Q3 模拟次数的收敛检验

九、 问题四：真实赛程综合评价

9.1 公开数据与质量控制

选择已结束且与题设同规模的 2026 年美加墨世界杯（48 队、12 组、72 场小组赛，2026 年 7 月 19 日完赛）。程序解析 OpenFootball 赛程和场馆文件^[4]，得到 72 场、12 组、48 队、16 场馆；每组 6 场、每队 3 场，保留每行来源 URL 和采集日期；每场开球时间自带 UTC 偏移（如 UTC-6），解析后统一换算为 UTC，场馆名称、坐标、容量与国家按同源 stadiums 文件映射。原始文件哈希、解析程序和来源说明均列入支撑材料。

9.2 规模无关的可识别指标

实际与优化赛程同为 48 队、72 场、16 场馆，规模指标直接可比；结构上再比较每队旅行、每次转场距离、平均/最小休息、60 h 以下比例、每队跨时区次数、场馆负载变异系数、场馆份额 HHI 和平均容量。图中每对指标先在两方案间 Min-Max，再将成本型指标反向，因而柱越高表示结构上越优。该图是可识别结构指标的展示，不是对真实赛程完整 Z_2 的复算。

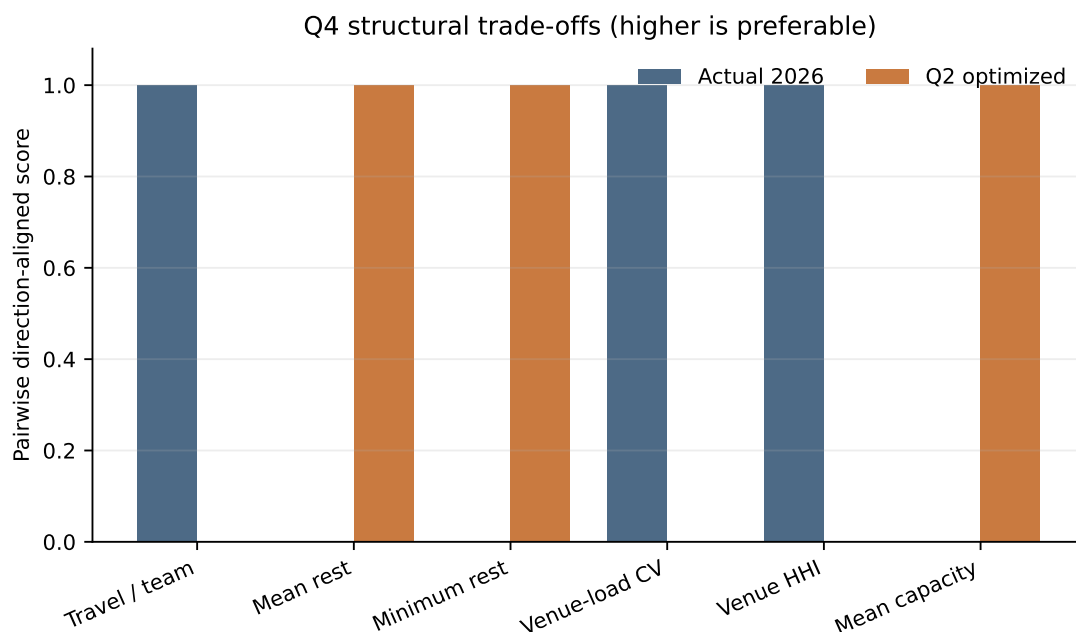


图 11 2026 实际赛程与模拟优化赛程的方向化结构权衡

表 3 Q4 主要同口径指标

指标	2026 实际	Q2 优化	方向性结论
每队转场旅行/km	2058.63	4257.17	实际优
平均休息/h	133.42	167.81	优化优
最小休息/h	91.00	159.00	优化优
每队跨时区次数	0.625	1.458	实际优
场馆负载 CV	0.1571	0.3514	实际优
场馆份额 HHI	0.0640	0.0702	实际优
平均场馆容量/人	68292	71353	优化优

FIFA 实际方案按西部、中部、东部三个地理区域组织小组，将每队旅行控制在 2058.63 km、每队跨时区 0.625 次，显著低于模拟方案的 4257.17 km 和 1.458 次；模拟方案则以区域分散换取更长的平均/最小休息（167.81/159.00 h 对 133.42/91.00 h）和略高的平均容量（71353 人对 68292 人）。场馆负载 CV 与 HHI 方向一致但数值不同：前者衡量场馆场次数离散，后者衡量份额集中度；实际方案的场馆分配更均衡（CV 0.1571、HHI 0.0640 对 0.3514、0.0702）。公开赛程不含题设定义所需的预测观众、单位转播价值、黄金时段得分、安保需求和气候风险，故不计算伪造的票务、转播、公平和风险指标。

十、 模型评价与推广

10.1 主要优点

1. 四问共享同一数据接口，Q1 单位、Q2 赛程和 Q3 信息截面均自动传递，减少人工口径漂移。
2. Q2 在许可限制下仍精确线性化受限集的完整 Z_2 ，COPT 目标与独立评估器数值对账；Q3 用共同候选和共同边界实现可识别的静态/动态比较。
3. 特征与变量均有题面或数据字段依据；对无法识别的 Q4 商业量选择缺失披露而非任意代理。
4. 采用时间验证、权重扰动、模拟收敛、独立约束检查和 SHA-256 形成多层证据链。

10.2 局限与改进

岭回归可能低估极端热门比赛，未来可在不泄漏条件下增加严格滞后的球队热度并比较 CatBoost/LightGBM；但隐藏测试标签不可用时仍不能声称外部最优。Q2 受评估许可证限制，未搜索全部 92160 个场馆-时段组合；商业许可下可用列生成或拉格朗日分解扩展候选。Q3 票价连续变量被离散为 5 点，可进一步用分段线性近似提高精度。Q4 未获得可同口径历史票务和转播数据，结论仅限结构层面；若获得官方运营数据，可复用 Q2 评估器完成更完整比较。

十一、 结论

本文给出一套从需求预测到赛程、再到赛中资源更新和外部对照的可运行方案。岭回归在时间验证中领先 7 类候选；受限候选 COPT 将 Z_2 由 0.3188 提高到 0.4582 并严格满足全部硬约束；统一信息截面的泊松模拟与双 COPT 模型使第三轮动态资源总效益提高 0.29247%；与已结束的 2026 年真实赛程对照表明，区域化分组是控制跨国旅行与时区成本的关键，而本方案的优化方向更偏向休息与容量。模型的关键价值不在于声称全局或隐藏测试最优，而在于每个数字都能由源数据、代码、日志和独立验收复现。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于审题辅助、建模方案讨论、代码编写与调试、数据核验、图表生成和论文文字整理，详细使用情况见支撑材料。所有模型公式、程序输出、求解日志、约束和论文结论均由参赛队逐项人工审查与核实。

参考文献

- [1] HOERL A E, KENNARD R W. Ridge Regression: Biased Estimation for Nonorthogonal Problems[J]. Technometrics, 1970, 12(1): 55-67.
- [2] 杉数科技. COPT 求解器 8.0.6[Z]. Cardinal Optimizer 软件及本地求解日志. 访问

日期: 2026-08-14. 2026.

- [3] SALTELLI A, RATTO M, ANDRES T, CAMPOLONGO F, CARIBONI J, GATELLI D, SAISANA M, TARANTOLA S. Global Sensitivity Analysis: The Primer[M]. John Wiley, 2008.
- [4] OpenFootball. 2026 FIFA World Cup Canada, Mexico, USA schedule and stadium data[Z]. <https://github.com/openfootball/worldcup/tree/master/2026--canada-usa-mexico>. 访问日期: 2026-08-14. 2026.

附录 A 支撑材料文件列表

路径	用途
src/c_contest_q1/*.py	Q1–Q4 特征、评价、模拟与 COPT 模型完整源程序
scripts/run_q1_demo.py	Q1 训练、验证与正式预测入口
scripts/run_q2_demo.py	Q2 确定性可行基线入口
scripts/run_q2_copt.py	Q2 完整受限候选 Z_2 COPT 入口
scripts/run_q3_copt.py	Q3 静态、动态及可行下界三 COPT 模型入口
scripts/collect_q4_openfootball.py	Q4 公开赛程解析与来源固化
scripts/run_analysis_campaign.py	权重扰动、模拟收敛和动态对比
scripts/validate_deliverables.py	六份模板、单位、约束、来源和目标对账验收
outputs/q2/copt/solver.log	Q2 COPT 版本、规模、状态、gap 和目标日志
outputs/q3/copt/solver.log	Q3 静态和动态 COPT 日志
outputs/validation/deliverable_validation.json	最终 14 项验收与输出 SHA-256
research/q4/	OpenFootball 原始赛程、场馆快照和来源说明
AI 工具使用详情.pdf	按竞赛规范披露 AI 工具、目的、过程、修改和核验

附录 B 运行环境与复现顺序

操作系统为 Windows，CPU 为 Intel Core i9-14900HX。Q1、Q4 与分析环境使用 NumPy、Pandas、scikit-learn 和 Matplotlib；Q2/Q3 使用 Python 3.12 与 COPT 8.0.6 非商业评估模式。随机种子统一为 20260814。按下列顺序运行：

Listing 1 端到端复现命令

```

1 .\venv-demo\Scripts\python.exe scripts\run_q1_demo.py
2 .\venv-demo\Scripts\python.exe scripts\run_q2_demo.py
3 .\venv\Scripts\python.exe scripts\run_q2_copt.py
4 .\venv\Scripts\python.exe scripts\run_q3_copt.py
5 .\venv-demo\Scripts\python.exe scripts\collect_q4_openfootball.py
6 .\venv-demo\Scripts\python.exe scripts\run_q4_comparison.py
7 .\venv-demo\Scripts\python.exe scripts\run_analysis_campaign.py
8 .\venv-demo\Scripts\python.exe scripts\validate_deliverables.py

```

附录 C COPT 应用证据与声明边界

Q2 求解日志记录 840 个二元变量、865 个约束、最优目标 0.458163650 和 0.0000% gap；独立评估器返回相同目标。Q3 静态、动态和可行下界三个模型各有 1623 个二元变量、48 个约束，均为 0.0000% gap；第三个模型在同一每日约束下最小化同一更新目标，给出 Z_3 可行区间 [4.7963, 6.6522]。评估版明确输出 2000 变量/约束限制，因此 Q2 采用可审计候选分解。本文只主张受限集合内整数最优，并完整提交 COPT 代码、目

志、元数据、三份选择表和独立验收结果。

附录 D 核心算法伪代码

算法 1 统一信息截面的 Q3 模拟与双模型优化

输入: 前两轮结果、Q2 第三轮固定赛程、基础泊松均值

输出: 晋级概率、风险、动态资源配置

- 1 计算 48 队状态 s_t 与伤病 h_t , 一次性更新 24 场 λ
 - 2 **for** $r = 1, \dots, 20000$ **do**
 - 3 同时抽取 24 场双方进球
 - 4 更新各组积分、净胜球、进球数
 - 5 选每组前二与最好的 8 个第三名, 完全同分用等比例随机键
 - 6 由同一模拟样本计算晋级概率、条件晋级概率和两类风险
 - 7 构造静态/动态共同资源候选, 删除违反 88% 保护的候选
 - 8 COPT 先求静态方案, 固定决策后在更新环境重评
 - 9 COPT 在相同容量、预算和边界下求动态方案
 - 10 独立校验 24 行输出、32 个晋级名额和每日资源约束
-

附录 E 完整可运行源程序

本附录逐文件给出支撑材料中的全部 Python 与 PowerShell 源程序。代码按原相对路径直接由 \lstinputlisting 载入，与支撑材料中的可执行文件为同一份源文件；运行环境、依赖、随机种子和端到端入口见附录 B 及 README.md。为便于审计，除 Q1-Q4 建模、COPT 求解、分析和制图代码外，同时列出提交构建器、独立验收器与全部自动测试。

核心建模模块

src/c_contest_q1/__init__.py

```
1 """C Contest Question 1 modeling package."""
```

src/c_contest_q1/paths.py

```
1 """Discovery of immutable contest inputs outside the generated project tree."""
2
3 from __future__ import annotations
4
5 from dataclasses import dataclass
6 from pathlib import Path
7
8 import pandas as pd
9
10
11 REQUIRED_SHEETS = frozenset(
12     {"historical_matches", "teams", "groups_matches", "base_predictions"}
13 )
14
15
16 @dataclass(frozen=True)
17 class SourcePaths:
18     """Resolved locations for the official Q1 workbook and its two CSV
19     templates."""
20
21     workbook: Path
22     test_template: Path
23     match_template: Path
24
25 def project_root() -> Path:
26     return Path(__file__).resolve().parents[2]
27
28
```

```

29 def contest_root() -> Path:
30     return project_root().parent
31
32
33 def bundled_data_root() -> Path:
34     return project_root() / "data" / "official"
35
36
37 def _find_unique_file(name: str) -> Path:
38     bundled = bundled_data_root() / name
39     if bundled.is_file():
40         return bundled.resolve()
41     candidates = [
42         path.resolve()
43         for path in contest_root().rglob(name)
44         if not path.name.startswith("~$") and project_root() not in path.
resolve().parents
45     ]
46     if len(candidates) != 1:
47         raise FileNotFoundError(f"Expected exactly one official {name}; found
{candidates}")
48     return candidates[0]
49
50
51 def _find_workbook() -> Path:
52     bundled_candidates = _workbooks_with_required_sheets(bundled_data_root().
glob("*.xlsx"))
53     if len(bundled_candidates) == 1:
54         return bundled_candidates[0]
55     if len(bundled_candidates) > 1:
56         raise FileNotFoundError(f"Expected exactly one bundled C-contest
workbook; found {bundled_candidates}")
57     candidates = _workbooks_with_required_sheets(contest_root().rglob("*.xlsx"
), exclude_project=True)
58     if len(candidates) != 1:
59         raise FileNotFoundError(f"Expected exactly one C-contest workbook;
found {candidates}")
60     return candidates[0]
61
62
63 def _workbooks_with_required_sheets(
64     workbooks: object,
65     exclude_project: bool = False,
66 ) -> list[Path]:
67     candidates: list[Path] = []

```

```

68     for workbook in workbooks:
69         if workbook.name.startswith("~$"):
70             continue
71         if exclude_project and project_root() in workbook.resolve().parents:
72             continue
73         try:
74             sheets = set(pd.ExcelFile(workbook).sheet_names)
75         except (OSError, ValueError):
76             continue
77         if REQUIRED_SHEETS.issubset(sheets):
78             candidates.append(workbook.resolve())
79     return candidates
80
81
82 def discover_template_path(name: str) -> Path:
83     """Locate one official CSV template, preferring a bundled submission copy.
84     """
85     return _find_unique_file(name)
86
87
88 def discover_source_paths() -> SourcePaths:
89     """Locate contest inputs without relying on locale-sensitive directory
90     names."""
91     return SourcePaths(
92         workbook=_find_workbook(),
93         test_template=_find_unique_file("result_1_test_prediction_template.csv
94         "),
95         match_template=_find_unique_file("result_1_match_prediction_template.
96         csv"),
97     )

```

src/c_contest_q1/data.py

```

1  """Read-only Q1 data loading and source-contract validation."""
2
3  from __future__ import annotations
4
5  from dataclasses import dataclass
6
7  import pandas as pd
8
9  from .paths import SourcePaths, discover_source_paths
10

```

```

11
12 @dataclass(frozen=True)
13 class Q1Tables:
14     paths: SourcePaths
15     historical: pd.DataFrame
16     teams: pd.DataFrame
17     group_matches: pd.DataFrame
18     base_predictions: pd.DataFrame
19     test_template: pd.DataFrame
20     match_template: pd.DataFrame
21
22
23 @dataclass(frozen=True)
24 class ValidationReport:
25     errors: tuple[str, ...]
26
27     @property
28     def ok(self) -> bool:
29         return not self.errors
30
31
32 def _read_workbook(paths: SourcePaths, sheet_name: str) -> pd.DataFrame:
33     return pd.read_excel(paths.workbook, sheet_name=sheet_name, engine="
openpyxl")
34
35
36 def load_q1_tables(paths: SourcePaths | None = None) -> Q1Tables:
37     """Load the immutable Q1 inputs and normalize only parsing types."""
38
39     paths = paths or discover_source_paths()
40     historical = _read_workbook(paths, "historical_matches")
41     historical["date"] = pd.to_datetime(historical["date"], errors="raise")
42     historical["dataset_split"] = historical["dataset_split"].astype("string")
43     .str.strip()
44     return Q1Tables(
45         paths=paths,
46         historical=historical,
47         teams=_read_workbook(paths, "teams"),
48         group_matches=_read_workbook(paths, "groups_matches"),
49         base_predictions=_read_workbook(paths, "base_predictions"),
50         test_template=pd.read_csv(paths.test_template),
51         match_template=pd.read_csv(paths.match_template),
52     )
53

```

```

54 def validate_q1_tables(tables: Q1Tables) -> ValidationReport:
55     """Validate contest invariants before any model can consume the inputs."""
56
57     errors: list[str] = []
58     history = tables.historical
59     train = history.loc[history["dataset_split"].eq("train")]
60     test = history.loc[history["dataset_split"].eq("test")]
61
62     expected_shapes = {
63         "historical": (700, 25),
64         "teams": (48, 16),
65         "group_matches": (72, 7),
66         "base_predictions": (72, 14),
67     }
68     actual_shapes = {
69         "historical": history.shape,
70         "teams": tables.teams.shape,
71         "group_matches": tables.group_matches.shape,
72         "base_predictions": tables.base_predictions.shape,
73     }
74     for name, expected in expected_shapes.items():
75         if actual_shapes[name] != expected:
76             errors.append(f"{name} shape {actual_shapes[name]} != {expected}")
77
78     for name, frame, column in (
79         ("historical", history, "match_id"),
80         ("teams", tables.teams, "team_id"),
81         ("group_matches", tables.group_matches, "match_id"),
82     ):
83         if not frame[column].is_unique:
84             errors.append(f"{name}.{column} is not unique")
85
86     if len(train) != 560 or len(test) != 140:
87         errors.append(f"split rows train={len(train)}, test={len(test)}")
88     if not train["tv_viewers"].notna().all():
89         errors.append("train tv_viewers contains missing labels")
90     if not test["tv_viewers"].isna().all():
91         errors.append("test tv_viewers must be hidden")
92     if train["date"].min() != pd.Timestamp("2018-01-12"):
93         errors.append("unexpected train start date")
94     if train["date"].max() != pd.Timestamp("2024-07-27"):
95         errors.append("unexpected train end date")
96     if test["date"].min() != pd.Timestamp("2024-08-05"):
97         errors.append("unexpected test start date")
98     if test["date"].max() != pd.Timestamp("2025-12-28"):

```

```

99         errors.append("unexpected test end date")
100     if set(tables.group_matches["match_id"]) != set(tables.base_predictions["
match_id"]):
101         errors.append("group_matches and base_predictions match IDs differ")
102     if list(tables.test_template.columns) != ["match_id_test", "
predicted_test_tv_viewers"]:
103         errors.append("unexpected test template columns")
104     if list(tables.match_template.columns) != [
105         "match_id",
106         "team_a",
107         "team_b",
108         "predicted_tv_viewers",
109     ]:
110         errors.append("unexpected match template columns")
111     return ValidationReport(tuple(errors))

```

src/c_contest_q1/features.py

```

1  """Leakage-safe, symmetric pre-match feature construction for Q1."""
2
3  from __future__ import annotations
4
5  from dataclasses import dataclass
6
7  import numpy as np
8  import pandas as pd
9
10 from .data import Q1Tables
11
12
13 FORBIDDEN_SOURCE_COLUMNS = frozenset(
14     {
15         "goals_a",
16         "goals_b",
17         "xg_a",
18         "xg_b",
19         "shots_a",
20         "shots_b",
21         "possession_a",
22         "possession_b",
23         "attendance",
24         "tv_viewers",
25         "match_id",
26         "team_a",
27         "team_b",

```

```

28     }
29 )
30
31
32 @dataclass(frozen=True)
33 class FeatureFrame:
34     features: pd.DataFrame
35     metadata: pd.DataFrame
36     target_people: pd.Series | None
37
38
39 def _sorted_pair(left: pd.Series, right: pd.Series) -> pd.Series:
40     return pd.DataFrame({"left": left.astype("string"), "right": right.astype(
41         "string")}).apply(
42         lambda row: "|".join(sorted((str(row["left"]), str(row["right"])))),
43         axis=1
44     )
45
46 def _team_attributes(tables: Q1Tables, matches: pd.DataFrame, *, future: bool)
47     -> pd.DataFrame:
48     team_columns = [
49         "team_name",
50         "confederation",
51         "market_value_musd",
52         "star_index",
53         "fan_base_index",
54         "home_timezone_region",
55     ]
56     teams = tables.teams.loc[:, team_columns]
57     left = teams.rename(columns={column: f"{column}_a" for column in
58         team_columns if column != "team_name"})
59     left = left.rename(columns={"team_name": "team_a"})
60     right = teams.rename(columns={column: f"{column}_b" for column in
61         team_columns if column != "team_name"})
62     right = right.rename(columns={"team_name": "team_b"})
63     enriched = matches.merge(left, on="team_a", how="left", validate="
64         many_to_one")
65     enriched = enriched.merge(right, on="team_b", how="left", validate="
66         many_to_one")
67     if enriched.filter(regex=r"^(confederation|market_value|star_index|
68         fan_base|home_timezone)").isna().any().any():
69         source = "future" if future else "historical"
70         raise ValueError(f"{source} match has an unmapped team attribute")
71     return enriched

```

```

65
66
67 def _probability_features(probabilities: pd.DataFrame) -> pd.DataFrame:
68     values = probabilities.to_numpy(dtype=float)
69     if not np.isfinite(values).all() or (values <= 0).any():
70         raise ValueError("pre-match probabilities must be positive and finite"
71 )
72     normalized = values / values.sum(axis=1, keepdims=True)
73     ordered = np.sort(normalized, axis=1)
74     entropy = -(normalized * np.log(normalized)).sum(axis=1) / np.log(3)
75     return pd.DataFrame(
76         {
77             "prob_low": ordered[:, 0],
78             "prob_mid": ordered[:, 1],
79             "prob_high": ordered[:, 2],
80             "prob_max": ordered[:, 2],
81             "prob_entropy": entropy,
82         },
83         index=probabilities.index,
84     )
85
86 def _assemble_features(enriched: pd.DataFrame, probabilities: pd.DataFrame) ->
87     pd.DataFrame:
88     features = _probability_features(probabilities)
89     elo_a = enriched["elo_a"].to_numpy(dtype=float)
90     elo_b = enriched["elo_b"].to_numpy(dtype=float)
91     rank_a = enriched["strength_rank_a"].to_numpy(dtype=float)
92     rank_b = enriched["strength_rank_b"].to_numpy(dtype=float)
93     fan_a = enriched["fan_base_index_a"].to_numpy(dtype=float)
94     fan_b = enriched["fan_base_index_b"].to_numpy(dtype=float)
95     star_a = enriched["star_index_a"].to_numpy(dtype=float)
96     star_b = enriched["star_index_b"].to_numpy(dtype=float)
97     value_a = enriched["market_value_musd_a"].to_numpy(dtype=float)
98     value_b = enriched["market_value_musd_b"].to_numpy(dtype=float)
99     features["competition"] = enriched["competition"].astype("string")
100     features["stage"] = enriched["stage"].astype("string")
101     features["elo_mean"] = (elo_a + elo_b) / 2.0
102     features["elo_abs_diff"] = np.abs(elo_a - elo_b)
103     features["rank_mean"] = (rank_a + rank_b) / 2.0
104     features["fan_sum"] = fan_a + fan_b
105     features["fan_max"] = np.maximum(fan_a, fan_b)
106     features["star_sum"] = star_a + star_b
107     features["star_max"] = np.maximum(star_a, star_b)
108     features["log_market_sum"] = np.log1p(value_a + value_b)

```



```

108     features["same_confederation"] = (
109         enriched["confederation_a"].eq(enriched["confederation_b"]).astype(int
110     )
111     features["confederation_pair"] = _sorted_pair(
112         enriched["confederation_a"], enriched["confederation_b"]
113     ).astype("string")
114     features["timezone_region_pair"] = _sorted_pair(
115         enriched["home_timezone_region_a"], enriched["home_timezone_region_b"]
116     ).astype("string")
117     if features.isna().any().any():
118         raise ValueError("feature construction produced missing values")
119     return features
120
121
122 def build_historical_features(tables: Q1Tables) -> FeatureFrame:
123     matches = _team_attributes(tables, tables.historical.copy(), future=False)
124     probabilities = pd.DataFrame(
125         {
126             "a": 1.0 / matches["odds_a"].astype(float),
127             "draw": 1.0 / matches["odds_draw"].astype(float),
128             "b": 1.0 / matches["odds_b"].astype(float),
129         },
130         index=matches.index,
131     )
132     metadata = matches.loc[:, ["match_id", "date", "dataset_split", "team_a",
133     "team_b"]].copy()
134     return FeatureFrame(
135         features=_assemble_features(matches, probabilities),
136         metadata=metadata,
137         target_people=matches["tv_viewers"].copy(),
138     )
139
140 def build_future_features(tables: Q1Tables) -> FeatureFrame:
141     matches = tables.group_matches.merge(
142         tables.base_predictions.loc[:, ["match_id", "p_a_win", "p_draw", "
143     p_b_win"]],
144         on="match_id",
145         how="inner",
146         validate="one_to_one",
147     )
148     matches["competition"] = "World Cup"
149     matches["stage"] = "Group"
150     matches["elo_a"] = matches["team_a"].map(tables.teams.set_index("team_name

```

```

    )["elo_rating"])
150 matches["elo_b"] = matches["team_b"].map(tables.teams.set_index("team_name")["elo_rating"])
    matches["strength_rank_a"] = matches["team_a"].map(
151         tables.teams.set_index("team_name")["strength_rank"]
152     )
153 matches["strength_rank_b"] = matches["team_b"].map(
154         tables.teams.set_index("team_name")["strength_rank"]
155     )
156 matches = _team_attributes(tables, matches, future=True)
157 probabilities = matches.loc[:, ["p_a_win", "p_draw", "p_b_win"]].rename(
158     columns={"p_a_win": "a", "p_draw": "draw", "p_b_win": "b"}
159 )
160 metadata = matches.loc[:, ["match_id", "team_a", "team_b", "group_id", "round_in_group"]].copy()
161 return FeatureFrame(
162     features=_assemble_features(matches, probabilities),
163     metadata=metadata,
164     target_people=None,
165 )
166
167
168
169 def target_in_millions(frame: FeatureFrame) -> pd.Series:
170     if frame.target_people is None:
171         raise ValueError("future feature frames have no target")
172     train_mask = frame.metadata["dataset_split"].eq("train")
173     target = frame.target_people.loc[train_mask]
174     if target.isna().any():
175         raise ValueError("training target contains missing values")
176     return target.astype(float) / 1_000_000

```

src/c_contest_q1/evaluation.py

```

1  """Date-grouped expanding-window model evaluation for Q1."""
2
3  from __future__ import annotations
4
5  from dataclasses import dataclass
6
7  import numpy as np
8  import pandas as pd
9  from sklearn.metrics import mean_absolute_error, mean_squared_error
10
11 from .models import ModelSpec
12

```

```

13
14 DEFAULT_TEMPORAL_FOLDS = (
15     ("fold_1", "2020-04-29", "2020-05-07", "2021-09-28"),
16     ("fold_2", "2021-09-28", "2021-10-03", "2023-02-23"),
17     ("fold_3", "2023-02-23", "2023-03-02", "2024-07-27"),
18 )
19
20
21 @dataclass(frozen=True)
22 class TemporalFold:
23     name: str
24     train_indices: np.ndarray
25     valid_indices: np.ndarray
26
27
28 @dataclass(frozen=True)
29 class EvaluationResult:
30     leaderboard: pd.DataFrame
31     fold_metrics: pd.DataFrame
32     oof_predictions: pd.DataFrame
33
34
35 def make_temporal_folds(metadata: pd.DataFrame) -> tuple[TemporalFold, ...]:
36     train_metadata = metadata.loc[metadata["dataset_split"].eq("train")].copy()
37     ()
38     dates = pd.to_datetime(train_metadata["date"], errors="raise")
39     folds: list[TemporalFold] = []
40     for name, train_end, valid_start, valid_end in DEFAULT_TEMPORAL_FOLDS:
41         train_mask = dates.le(pd.Timestamp(train_end))
42         valid_mask = dates.between(pd.Timestamp(valid_start), pd.Timestamp(
43             valid_end), inclusive="both")
44         train_indices = train_metadata.index[train_mask].to_numpy()
45         valid_indices = train_metadata.index[valid_mask].to_numpy()
46         if not len(train_indices) or not len(valid_indices):
47             raise ValueError(f"{name} is empty")
48         if dates.loc[train_indices].max() >= dates.loc[valid_indices].min():
49             raise ValueError(f"{name} violates temporal order")
50         folds.append(TemporalFold(name, train_indices, valid_indices))
51     return tuple(folds)
52
53 def _column_groups(features: pd.DataFrame) -> tuple[list[str], list[str]]:
54     categorical = features.select_dtypes(include=["string", "object", "
55         category"]).columns.tolist()
56     numeric = [column for column in features.columns if column not in

```

```

        categorical]
55     return numeric, categorical
56
57
58 def evaluate_model_specs(
59     features: pd.DataFrame,
60     metadata: pd.DataFrame,
61     target: pd.Series,
62     specs: tuple[ModelSpec, ...],
63 ) -> EvaluationResult:
64     train_mask = metadata["dataset_split"].eq("train")
65     train_features = features.loc[train_mask]
66     train_metadata = metadata.loc[train_mask]
67     if not target.index.equals(train_features.index):
68         raise ValueError("target and training features must share index")
69     numeric_columns, categorical_columns = _column_groups(train_features)
70     folds = make_temporal_folds(metadata)
71     metric_rows: list[dict[str, object]] = []
72     prediction_rows: list[dict[str, object]] = []
73     for spec in specs:
74         for fold in folds:
75             x_train = features.loc[fold.train_indices]
76             y_train = target.loc[fold.train_indices]
77             x_valid = features.loc[fold.valid_indices]
78             y_valid = target.loc[fold.valid_indices]
79             estimator = spec.factory(numeric_columns, categorical_columns)
80             if estimator is None:
81                 prediction = np.full(len(x_valid), y_train.mean(), dtype=float)
82             else:
83                 estimator.fit(x_train, y_train)
84                 prediction = np.asarray(estimator.predict(x_valid), dtype=
float)
85             mse = float(mean_squared_error(y_valid, prediction))
86             metric_rows.append(
87                 {
88                     "model": spec.name,
89                     "fold": fold.name,
90                     "mse": mse,
91                     "rmse": float(np.sqrt(mse)),
92                     "mae": float(mean_absolute_error(y_valid, prediction)),
93                     "n_validation": len(y_valid),
94                 }
95             )
96             for index, actual, predicted in zip(fold.valid_indices, y_valid,

```

```

        prediction, strict=True):
97             prediction_rows.append(
98                 {
99                     "model": spec.name,
100                     "fold": fold.name,
101                     "row_index": index,
102                     "match_id": train_metadata.loc[index, "match_id"],
103                     "date": train_metadata.loc[index, "date"],
104                     "actual": actual,
105                     "prediction": predicted,
106                     "residual": predicted - actual,
107                 }
108             )
109     fold_metrics = pd.DataFrame(metric_rows).sort_values(["model", "fold"]).
reset_index(drop=True)
110     leaderboard = (
111         fold_metrics.groupby("model", as_index=False)
112         .agg(
113             mse=("mse", "mean"),
114             mse_std=("mse", "std"),
115             rmse=("rmse", "mean"),
116             mae=("mae", "mean"),
117             worst_fold_mse=("mse", "max"),
118         )
119         .fillna({"mse_std": 0.0})
120         .sort_values(["mse", "worst_fold_mse"])
121         .reset_index(drop=True)
122     )
123     return EvaluationResult(
124         leaderboard=leaderboard,
125         fold_metrics=fold_metrics,
126         oof_predictions=pd.DataFrame(prediction_rows).sort_values(["model", "
row_index"]).reset_index(drop=True),
127     )

```

src/c_contest_q1/models.py

```

1  """Small-sample baseline model specifications for the first Q1 Demo."""
2
3  from __future__ import annotations
4
5  from dataclasses import dataclass
6  from typing import Callable
7
8  from sklearn.compose import ColumnTransformer

```

```

9 from sklearn.ensemble import HistGradientBoostingRegressor
10 from sklearn.ensemble import ExtraTreesRegressor, RandomForestRegressor
11 from sklearn.impute import SimpleImputer
12 from sklearn.linear_model import ElasticNet, Ridge
13 from sklearn.pipeline import Pipeline
14 from sklearn.preprocessing import OneHotEncoder, StandardScaler
15 from sklearn.svm import SVR
16
17
18 @dataclass(frozen=True)
19 class ModelSpec:
20     name: str
21     factory: Callable[[list[str], list[str]], object | None]
22
23
24 def _preprocessor(numeric_columns: list[str], categorical_columns: list[str])
25     -> ColumnTransformer:
26     return ColumnTransformer(
27         transformers=[
28             (
29                 "numeric",
30                 Pipeline(
31                     [
32                         ("impute", SimpleImputer(strategy="median")),
33                         ("scale", StandardScaler()),
34                     ]
35                 ),
36                 numeric_columns,
37             ),
38             (
39                 "categorical",
40                 Pipeline(
41                     [
42                         ("impute", SimpleImputer(strategy="most_frequent")),
43                         ("encode", OneHotEncoder(handle_unknown="ignore",
44 sparse_output=False)),
45                     ]
46                 ),
47                 categorical_columns,
48             ),
49         ],
50         remainder="drop",
51         verbose_feature_names_out=False,

```

```

52
53 def _ridge(numeric_columns: list[str], categorical_columns: list[str]) ->
    Pipeline:
54     return Pipeline(
55         [("preprocess", _preprocessor(numeric_columns, categorical_columns)),
56         ("model", Ridge(alpha=10.0))]
57     )
58
59 def _hist_gradient_boosting(numeric_columns: list[str], categorical_columns:
    list[str]) -> Pipeline:
60     return Pipeline(
61         [
62             ("preprocess", _preprocessor(numeric_columns, categorical_columns)
63             ),
64             (
65                 "model",
66                 HistGradientBoostingRegressor(
67                     learning_rate=0.05,
68                     max_leaf_nodes=12,
69                     min_samples_leaf=20,
70                     l2_regularization=2.0,
71                     early_stopping=False,
72                     random_state=20260814,
73                 ),
74             ),
75         ]
76     )
77
78 def _elastic_net(numeric_columns: list[str], categorical_columns: list[str])
    -> Pipeline:
79     return Pipeline(
80         [
81             ("preprocess", _preprocessor(numeric_columns, categorical_columns)
82             ),
83             ("model", ElasticNet(alpha=0.15, l1_ratio=0.15, max_iter=20_000,
84             random_state=20260814)),
85         ]
86     )
87
88 def _svr(numeric_columns: list[str], categorical_columns: list[str]) ->
    Pipeline:
89     return Pipeline(

```

```

89         [
90             ("preprocess", _preprocessor(numeric_columns, categorical_columns)
91         ),
92         ("model", SVR(C=3.0, epsilon=0.1, gamma="scale")),
93     ]
94 )
95
96 def _extra_trees(numeric_columns: list[str], categorical_columns: list[str])
97     -> Pipeline:
98     return Pipeline(
99         [
100             ("preprocess", _preprocessor(numeric_columns, categorical_columns)
101         ),
102         (
103             "model",
104             ExtraTreesRegressor(
105                 n_estimators=500,
106                 max_features=0.8,
107                 min_samples_leaf=4,
108                 random_state=20260814,
109                 n_jobs=-1,
110             ),
111         ],
112     )
113
114 def _random_forest(numeric_columns: list[str], categorical_columns: list[str])
115     -> Pipeline:
116     return Pipeline(
117         [
118             ("preprocess", _preprocessor(numeric_columns, categorical_columns)
119         ),
120         (
121             "model",
122             RandomForestRegressor(
123                 n_estimators=500,
124                 max_features=0.8,
125                 min_samples_leaf=5,
126                 random_state=20260814,
127                 n_jobs=-1,
128             ),
129         ],
130     )

```



```

129     )
130
131
132 def demo_model_specs() -> tuple[ModelSpec, ...]:
133     return (
134         ModelSpec("mean", lambda numeric, categorical: None),
135         ModelSpec("ridge", _ridge),
136         ModelSpec("hist_gradient_boosting", _hist_gradient_boosting),
137     )
138
139
140 def sklearn_challenger_specs() -> tuple[ModelSpec, ...]:
141     """Second-pass challengers using the unchanged Demo feature and split
142     contract."""
143
144     return (
145         ModelSpec("elastic_net", _elastic_net),
146         ModelSpec("svr_rbf", _svr),
147         ModelSpec("extra_trees", _extra_trees),
148         ModelSpec("random_forest", _random_forest),
149     )

```

src/c_contest_q1/outputs.py

```

1  """Full-train Q1 Demo prediction files and independent output validation."""
2
3  from __future__ import annotations
4
5  from dataclasses import dataclass
6  from pathlib import Path
7
8  import numpy as np
9  import pandas as pd
10
11 from .data import Q1Tables
12 from .evaluation import _column_groups
13 from .features import build_future_features, build_historical_features,
14     target_in_millions
15
16 from .models import demo_model_specs
17
18 @dataclass(frozen=True)
19 class OutputArtifacts:
20     test_csv: Path
21     match_csv: Path

```

```

21
22
23 @dataclass(frozen=True)
24 class OutputValidationReport:
25     errors: tuple[str, ...]
26
27     @property
28     def ok(self) -> bool:
29         return not self.errors
30
31
32 def _fit_demo_ridge(tables: Q1Tables):
33     historical = build_historical_features(tables)
34     future = build_future_features(tables)
35     train_mask = historical.metadata["dataset_split"].eq("train")
36     numeric, categorical = _column_groups(historical.features)
37     ridge_spec = next(spec for spec in demo_model_specs() if spec.name == "
    ridge")
38     model = ridge_spec.factory(numeric, categorical)
39     assert model is not None
40     model.fit(historical.features.loc[train_mask], target_in_millions(
    historical))
41     return model, historical, future
42
43
44 def build_demo_outputs(tables: Q1Tables, output_dir: Path) -> OutputArtifacts:
45     """Train the selected Demo Ridge model and write official Q1-format CSVs."
    ""
46
47     output_dir.mkdir(parents=True, exist_ok=True)
48     model, historical, future = _fit_demo_ridge(tables)
49     test_mask = historical.metadata["dataset_split"].eq("test")
50     test_predictions = np.clip(model.predict(historical.features.loc[test_mask
    ]), 0.0, None)
51     future_predictions_people = np.clip(model.predict(future.features) * 1
    _000_000, 0.0, None)
52
53     test_output = pd.DataFrame(
54         {
55             "match_id_test": historical.metadata.loc[test_mask, "match_id"].
    to_numpy(),
56             "predicted_test_tv_viewers": test_predictions,
57         }
58     )
59     match_output = future.metadata.loc[:, ["match_id", "team_a", "team_b"]].

```

```

        copy()
60     match_output["predicted_tv_viewers"] = future_predictions_people
61     test_csv = output_dir / "result_1_test_prediction.csv"
62     match_csv = output_dir / "result_1_match_prediction.csv"
63     test_output.to_csv(test_csv, index=False)
64     match_output.to_csv(match_csv, index=False)
65     return OutputArtifacts(test_csv=test_csv, match_csv=match_csv)
66
67
68 def validate_demo_outputs(
69     tables: Q1Tables, test_csv: Path, match_csv: Path
70 ) -> OutputValidationReport:
71     """Reload generated files and verify exact identifiers, schemas, and units
72     ."""
73
74     errors: list[str] = []
75     test_output = pd.read_csv(test_csv)
76     match_output = pd.read_csv(match_csv)
77     expected_test_ids = tables.historical.loc[
78         tables.historical["dataset_split"].eq("test"), "match_id"
79     ]
80     expected_match_keys = tables.group_matches.loc[:, ["match_id", "team_a", "
81 team_b"]]
82
83     if list(test_output.columns) != list(tables.test_template.columns):
84         errors.append("test output columns differ from template")
85     if list(match_output.columns) != list(tables.match_template.columns):
86         errors.append("match output columns differ from template")
87     if len(test_output) != 140 or set(test_output["match_id_test"]) != set(
88         expected_test_ids):
89         errors.append("test output does not contain exactly the official 140
89 IDs")
90     if len(match_output) != 72:
91         errors.append("match output does not contain 72 rows")
92     elif not match_output.loc[:, ["match_id", "team_a", "team_b"]].equals(
93         expected_match_keys):
94         errors.append("match output keys differ from groups_matches")
95     for name, values in (
96         ("test predictions", test_output.get("predicted_test_tv_viewers", pd.
97 Series(dtype=float))),
98         ("match predictions", match_output.get("predicted_tv_viewers", pd.
99 Series(dtype=float))),
100     ):
101         if len(values) == 0 or not np.isfinite(values).all() or (values < 0).
102 any():
103             errors.append(f"{name} must be finite and nonnegative")

```

```

96     if len(test_output) and not test_output["predicted_test_tv_viewers"].
    between(1.0, 500.0).all():
97         errors.append("test predictions are not expressed in millions")
98     if len(match_output) and not match_output["predicted_tv_viewers"].between
    (1_000_000, 500_000_000).all():
99         errors.append("match predictions are not expressed in people")
100    return OutputValidationReport(tuple(errors))

```

src/c_contest_q1/q2_demo.py

```

1  """Feasibility-first Q2 group schedule Demo and independent hard-constraint
    checks.
2
3  The schedule is a deterministic, auditable incumbent. It is also the data
4  interface used by the future COPT model; it never claims to be a COPT optimum.
5  """
6
7  from __future__ import annotations
8
9  from dataclasses import dataclass
10 from pathlib import Path
11 from zoneinfo import ZoneInfo
12
13 import numpy as np
14 import pandas as pd
15
16 from .data import Q1Tables
17
18
19 GROUP_START_DATES = {
20     "A": "2026-06-11", "B": "2026-06-11", "C": "2026-06-12", "D": "2026-06-12"
    ,
21     "E": "2026-06-13", "F": "2026-06-13", "G": "2026-06-14", "H": "2026-06-14"
    ,
22     "I": "2026-06-15", "J": "2026-06-15", "K": "2026-06-16", "L": "2026-06-16"
    ,
23 }
24
25
26 @dataclass(frozen=True)
27 class Q2Validation:
28     errors: tuple[str, ...]
29
30     @property
31     def ok(self) -> bool:

```

```

32         return not self.errors
33
34
35 def _minmax(value: float, lower: float, upper: float) -> float:
36     return 0.0 if np.isclose(lower, upper) else float(np.clip((value - lower)
37         / (upper - lower), 0.0, 1.0))
38
39 def _load_q2_tables(tables: Q1Tables) -> dict[str, pd.DataFrame]:
40     names = ["venues", "time_slots", "ticket_broadcast", "
41         security_requirements", "dynamic_resource_limits", "objective_weights"]
42     return {name: pd.read_excel(tables.paths.workbook, sheet_name=name) for
43         name in names}
44
45 def _q1_predictions(tables: Q1Tables, q1_match_csv: Path) -> pd.DataFrame:
46     prediction = pd.read_csv(q1_match_csv)
47     expected = tables.group_matches[["match_id", "team_a", "team_b"]]
48     if not prediction[["match_id", "team_a", "team_b"]].equals(expected):
49         raise ValueError("Q1 match prediction keys do not match groups_matches
50 ")
51     return prediction
52
53 def build_q2_demo_schedule(tables: Q1Tables, q1_match_csv: Path) -> pd.
54     DataFrame:
55     data = _load_q2_tables(tables)
56     venues, slots = data["venues"], data["time_slots"].copy()
57     slots["date"] = pd.to_datetime(slots["date"])
58     slots["utc_datetime"] = pd.to_datetime(slots["reference_utc_time"], utc=
59     True)
60     matches = tables.group_matches.merge(_q1_predictions(tables, q1_match_csv)
61     [["match_id", "predicted_tv_viewers"]], on="match_id")
62     matches = matches.merge(pd.read_excel(tables.paths.workbook, sheet_name="
63     base_predictions"), on=["match_id", "group_id", "round_in_group", "team_a",
64     "team_b"])
65     matches = matches.merge(data["security_requirements"][["match_id", "
66     required_security_level"]], on="match_id")
67     rows: list[dict[str, object]] = []
68     venue_cycle = venues.sort_values("venue_id").reset_index(drop=True)
69     venue_total = {str(venue_id): 0 for venue_id in venue_cycle["venue_id"]}
70     venue_day_use: set[tuple[str, pd.Timestamp]] = set()
71     for ordinal, match in matches.sort_values(["round_in_group", "group_id", "
72     match_id"]).reset_index(drop=True).iterrows():
73         base = pd.Timestamp(GROUP_START_DATES[match.group_id])

```

```

66     date = base + pd.Timedelta(days=7 * (int(match.round_in_group) - 1))
67     day_slots = slots.loc[slots["date"].eq(date)].sort_values("
reference_utc_time").reset_index(drop=True)
68     group_matches_on_day = matches.loc[(matches["group_id"].map(
GROUP_START_DATES).map(pd.Timestamp).eq(base)) & (matches["round_in_group"
].eq(match.round_in_group))].sort_values("match_id")
69     slot_index = group_matches_on_day.index.get_loc(matches.index[matches[
"match_id"].eq(match.match_id)][0])
70     non_prime = day_slots.loc[~day_slots["slot_id"].isin(set(slots.
nlargest(len(slots) // 4, "global_prime_score")["slot_id"]))]
71     # Two non-golden UTC slots carry two matches each; their broadcast
capacity is at least two.
72     slot = non_prime.iloc[slot_index // 2]
73     eligible = venue_cycle.loc[venue_cycle["security_level"].ge(int(match.
required_security_level))].copy()
74     eligible["_assigned"] = eligible["venue_id"].map(venue_total)
75     eligible = eligible.loc[~eligible["venue_id"].map(lambda venue_id: (
str(venue_id), date) in venue_day_use)]
76     if eligible.empty:
77         raise ValueError(f"No eligible unused venue for {match.match_id}
on {date.date()}")
78     venue = eligible.sort_values(["_assigned", "venue_id"]).iloc[0]
79     venue_total[str(venue.venue_id)] += 1
80     venue_day_use.add((str(venue.venue_id), date))
81     local = slot.utc_datetime.tz_convert(ZoneInfo(venue.timezone))
82     stage_key = f"Group_Match_R{int(match.round_in_group)}"
83     economics = data["ticket_broadcast"].loc[data["ticket_broadcast"]["
match_stage"].eq(stage_key)].iloc[0]
84     expected_attendance = min(float(match.expected_attendance_base), float
(venue.capacity))
85     ticket = expected_attendance * float(economics.base_ticket_price_usd)
86     broadcast = float(match.predicted_tv_viewers) * float(economics.
broadcast_unit_value_usd) * float(slot.global_prime_score) * float(
economics.sponsor_weight)
87     risk = 0.5 * float(venue.climate_risk) + 0.3 * (expected_attendance /
float(venue.capacity)) + 0.2 * (float(match.required_security_level) /
float(venue.security_level))
88     rows.append({
89         "match_id": match.match_id, "group_id": match.group_id, "
round_in_group": int(match.round_in_group),
90         "team_a": match.team_a, "team_b": match.team_b, "venue_id": venue.
venue_id, "slot_id": slot.slot_id,
91         "city": venue.city, "country": venue.country, "reference_date":
slot.date.date().isoformat(),
92         "reference_kickoff_time": slot.reference_kickoff_time, "

```

```

local_datetime": local.isoformat(),
93     "utc_datetime": slot.utc_datetime.isoformat(), "
required_security_level": int(match.required_security_level),
94     "expected_attendance": expected_attendance, "expected_tv_viewers":
float(match.predicted_tv_viewers),
95     "ticket_revenue_usd": ticket, "broadcast_value_usd": broadcast,
96     "travel_cost_index": np.nan, "fairness_penalty": np.nan, "
risk_index": risk,
97     })
98     schedule = pd.DataFrame(rows).sort_values("match_id").reset_index(drop=
True)
99     # Equal resource-use summary is a schedule-level metric repeated for each
row.
100     prime_ids = set(slots.nlargest(max(1, len(slots) // 4), "
global_prime_score")["slot_id"])
101     large_ids = set(venues.nlargest(max(1, len(venues) // 4), "capacity")["
venue_id"])
102     team_counts: dict[str, list[int]] = {}
103     for team in sorted(set(schedule.team_a).union(schedule.team_b)):
104         uses = schedule.loc[schedule.team_a.eq(team) | schedule.team_b.eq(team
)]
105         team_counts[team] = [int(uses.slot_id.isin(prime_ids).sum()), int(uses
.venue_id.isin(large_ids).sum())]
106         golden_range = max(value[0] for value in team_counts.values()) - min(value
[0] for value in team_counts.values())
107         large_range = max(value[1] for value in team_counts.values()) - min(value
[1] for value in team_counts.values())
108         fairness = 0.5 * golden_range / 3 + 0.5 * large_range / 3
109         schedule["fairness_penalty"] = fairness
110         schedule["travel_cost_index"] = 0.0 # Full candidate-set travel
normalization is implemented in the COPT/score upgrade, not fabricated here
.
111         raw = schedule.ticket_revenue_usd.sum() + schedule.broadcast_value_usd.sum
() - schedule.risk_index.sum() * 1_000_000
112         schedule["total_objective_value"] = np.nan
113         schedule.loc[0, "total_objective_value"] = raw
114         return schedule
115
116
117 def validate_q2_schedule(tables: Q1Tables, schedule: pd.DataFrame) ->
Q2Validation:
118     data = _load_q2_tables(tables)
119     errors: list[str] = []
120     required = ["match_id", "venue_id", "slot_id", "utc_datetime", "
required_security_level"]

```

```

121     if any(column not in schedule for column in required):
122         return Q2Validation(("schedule missing required columns",))
123     if len(schedule) != 72 or set(schedule.match_id) != set(
124         group_matches.match_id) or not schedule.match_id.is_unique:
125         errors.append("exactly one assignment for each of 72 matches is
126         required")
127     if schedule.duplicated(["venue_id", "utc_datetime"]).any():
128         errors.append("venue/UTC collision")
129     utc = pd.to_datetime(schedule.utc_datetime, utc=True)
130     for team in sorted(set(schedule.team_a).union(schedule.team_b)):
131         games = schedule.loc[(schedule.team_a.eq(team) | schedule.team_b.eq(
132             team))].assign(_utc=utc.loc[(schedule.team_a.eq(team) | schedule.team_b.eq(
133                 team))]).sort_values("round_in_group")
134         if len(games) != 3 or not (games.round_in_group.to_list() == [1, 2,
135             3]):
136             errors.append(f"{team} round participation invalid")
137             elif (games._utc.diff().dropna() < pd.Timedelta(hours=60)).any():
138                 errors.append(f"{team} has less than 60 hours rest")
139     venues = data["venues"].set_index("venue_id")
140     for _, row in schedule.iterrows():
141         if int(venues.loc[row.venue_id, "security_level"]) < int(row.
142             required_security_level):
143             errors.append(f"security-ineligible venue for {row.match_id}")
144     totals = schedule.venue_id.value_counts()
145     for venue_id, venue in venues.iterrows():
146         total = int(totals.get(venue_id, 0))
147         if not int(venue.min_total_matches) <= total <= int(venue.
148             max_total_matches):
149             errors.append(f"venue total bound violated: {venue_id}")
150     local_dates = []
151     for _, row in schedule.iterrows():
152         local_dates.append(pd.Timestamp(row.local_datetime).date())
153     local_use = schedule.assign(_local_date=local_dates).groupby(["venue_id",
154         "_local_date"]).size()
155     for (venue_id, _), count in local_use.items():
156         if count > int(venues.loc[venue_id, "max_matches_per_day"]):
157             errors.append(f"venue local-day bound violated: {venue_id}")
158     slots = data["time_slots"].set_index("slot_id")
159     simultaneous = schedule.slot_id.value_counts()
160     for slot_id, count in simultaneous.items():
161         if count > int(slots.loc[slot_id, "broadcast_capacity"]):
162             errors.append(f"broadcast capacity violated: {slot_id}")
163     prime = set(slots.nlargest(len(slots) // 4, "global_prime_score").index)
164     per_team_prime = []
165     for team in sorted(set(schedule.team_a).union(schedule.team_b)):

```



```

158     per_team_prime.append(int(schedule.loc[schedule.team_a.eq(team) |
schedule.team_b.eq(team), "slot_id"].isin(prime).sum()))
159     if max(per_team_prime) - min(per_team_prime) > 2:
160         errors.append("golden-slot fairness range exceeds 2")
161     limits = data["dynamic_resource_limits"].copy()
162     limits.reference_date = pd.to_datetime(limits.reference_date).dt.date.
astype(str)
163     r3 = schedule.loc[schedule.round_in_group.eq(3)].copy()
164     for date, block in r3.groupby("reference_date"):
165         cap = int(limits.loc[limits.reference_date.eq(date), "
high_security_capacity"].iloc[0])
166         if int(block.required_security_level.ge(3).sum()) > cap:
167             errors.append(f"third-round high-security capacity violated on {
date}")
168     return Q2Validation(tuple(dict.fromkeys(errors)))
169
170
171 def evaluate_q2_schedule(tables: Q1Tables, schedule: pd.DataFrame) -> tuple[pd
.DataFrame, dict[str, float]]:
172     """Score a feasible schedule on the official P2 indicators.
173
174     Candidate ranges are fixed before reading the selected venue/slot. In
175     particular, the travel basis follows the stated prior-round eligible-venue
176     averaging rule, preventing an optimistic range from being chosen after the
177     fact. Setup cost is necessarily schedule-level because venue activation
178     is
179     a binary decision.
180
181     """
182     data = _load_q2_tables(tables)
183     venues = data["venues"].copy()
184     slots = data["time_slots"].copy()
185     base = tables.base_predictions.copy()
186     q1_viewers = schedule[["match_id", "expected_tv_viewers"]].rename(columns
={"expected_tv_viewers": "predicted_tv_viewers"})
187     matches = tables.group_matches.merge(base, on=["match_id", "group_id", "
round_in_group", "team_a", "team_b"]).merge(q1_viewers, on="match_id")
188     matches = matches.merge(data["security_requirements"][["match_id", "
required_security_level"]], on="match_id")
189     team_id = tables.teams.set_index("team_name")["team_id"].to_dict()
190     distance = pd.read_excel(tables.paths.workbook, sheet_name="
distance_matrix")
191     eligible = {row.match_id: venues.loc[venues.security_level.ge(row.
required_security_level), "venue_id"].astype(str).tolist() for row in
matches.itertuples(index=False)}

```

```

191
192     # Construct exactly the candidate paths prescribed in the statement.
193     path_rows: list[dict[str, object]] = []
194     match_by_team_round: dict[tuple[str, int], object] = {}
195     for row in matches.itertuples(index=False):
196         match_by_team_round[(row.team_a, int(row.round_in_group))] = row
197         match_by_team_round[(row.team_b, int(row.round_in_group))] = row
198     for row in matches.itertuples(index=False):
199         round_number = int(row.round_in_group)
200         for team in (row.team_a, row.team_b):
201             for venue_id in eligible[row.match_id]:
202                 if round_number == 1:
203                     origin_id = team_id[team]
204                     candidate = distance.loc[(distance.relation_type.eq("
team_to_venue")) & distance.origin_id.eq(origin_id) & distance.
destination_id.astype(str).eq(venue_id)]
205                 else:
206                     previous = match_by_team_round[(team, round_number - 1)]
207                     candidate = distance.loc[(distance.relation_type.eq("
venue_to_venue")) & distance.origin_id.astype(str).isin(eligible[previous.
match_id]) & distance.destination_id.astype(str).eq(venue_id)]
208                 if candidate.empty:
209                     raise ValueError(f"missing travel candidate for {row.
match_id}, {team}, {venue_id}")
210                 path_rows.append({"match_id": row.match_id, "team": team, "
venue_id": venue_id, "distance_km": candidate.distance_km.mean(), "
travel_time_hour": candidate.travel_time_hour.mean(), "timezone_diff":
candidate.timezone_diff.mean()})
211     paths = pd.DataFrame(path_rows)
212     for column in ["distance_km", "travel_time_hour", "timezone_diff"]:
213         paths[f"{column}_norm"] = (paths[column] - paths[column].min()) / (
paths[column].max() - paths[column].min())
214     paths["travel_burden"] = 0.5 * paths.distance_km_norm + 0.3 * paths.
travel_time_hour_norm + 0.2 * paths.timezone_diff_norm
215     candidate_travel = paths.groupby(["match_id", "venue_id"], as_index=False)
.travel_burden.mean()
216
217     economics = data["ticket_broadcast"].set_index("match_stage")
218     ticket_candidates: dict[str, list[float]] = {}
219     broadcast_candidates: dict[str, list[float]] = {}
220     candidate_c: list[float] = []
221     for row in matches.itertuples(index=False):
222         price = economics.loc[f"Group_Match_R{int(row.round_in_group)}"]
223         ticket_candidates[row.match_id] = []
224         broadcast_candidates[row.match_id] = []

```

```

225         for venue in venues.itertuples(index=False):
226             if venue.venue_id in eligible[row.match_id]:
227                 ticket_candidates[row.match_id].append(min(float(row.
expected_attendance_base), float(venue.capacity)) * float(price.
base_ticket_price_usd))
228                 candidate_c.append(float(venue.operation_cost_musd_per_match)
+ 0.1 * int(row.required_security_level) * float(venue.security_cost_index)
)
229         for slot in slots.itertuples(index=False):
230             broadcast_candidates[row.match_id].append(float(row.
predicted_tv_viewers) * float(price.broadcast_unit_value_usd) * float(slot.
global_prime_score) * float(price.sponsor_weight))
231             selected = schedule.copy()
232             selected["venue_id"] = selected.venue_id.astype(str)
233             selected = selected.merge(candidate_travel, on=["match_id", "venue_id"],
how="left")
234             if selected.travel_burden.isna().any():
235                 raise ValueError("selected schedule has a venue without a travel
candidate")
236             selected = selected.merge(matches[["match_id", "uncertainty_index", "
attractiveness_index", "expected_attendance_base"]], on="match_id", how="
left")
237             selected = selected.merge(venues[["venue_id", "capacity", "setup_cost_musd
", "operation_cost_musd_per_match", "security_cost_index", "climate_risk",
"security_level"]], on="venue_id", how="left", suffixes=("", "_venue"))
238             selected["operation_cost_musd"] = selected.operation_cost_musd_per_match +
0.1 * selected.required_security_level * selected.security_cost_index
239             selected["risk_raw"] = 0.5 * selected.climate_risk + 0.3 * (selected.
expected_attendance / selected.capacity) + 0.2 * selected.
required_security_level / selected.security_level
240             selected["travel_cost_index"] = selected.travel_burden
241             prime = set(slots.nlargest(len(slots) // 4, "global_prime_score").slot_id.
astype(str))
242             large = set(venues.nlargest(len(venues) // 4, "capacity").venue_id.astype(
str))
243             resource_counts = []
244             for team in sorted(set(selected.team_a).union(selected.team_b)):
245                 games = selected.loc[selected.team_a.eq(team) | selected.team_b.eq(
team)]
246                 resource_counts.append((int(games.slot_id.astype(str).isin(prime).sum
()), int(games.venue_id.isin(large).sum()))))
247             fairness = 0.5 * (max(item[0] for item in resource_counts) - min(item[0]
for item in resource_counts)) / 3 + 0.5 * (max(item[1] for item in
resource_counts) - min(item[1] for item in resource_counts)) / 3
248             raw = {

```

```

249         "T": float(selected.ticket_revenue_usd.sum()),
250         "B": float(selected.broadcast_value_usd.sum()),
251         "U": float(selected.uncertainty_index.sum()),
252         "H": float((selected.attractiveness_index / 100).sum()),
253         "C": float(selected.operation_cost_musd.sum() + selected.
drop_duplicates("venue_id").setup_cost_musd.sum()),
254         "D": float(selected.travel_burden.sum()),
255         "F": float(fairness),
256         "R": float(selected.risk_raw.mean()),
257     }
258     ranges = {
259         "T": (sum(min(values) for values in ticket_candidates.values()), sum(
max(values) for values in ticket_candidates.values())),
260         "B": (sum(min(values) for values in broadcast_candidates.values()),
sum(max(values) for values in broadcast_candidates.values())),
261         "C": (sum(min(candidate_c) for _ in range(len(matches))), sum(max(
candidate_c) for _ in range(len(matches))) + float(venues.setup_cost_musd.
sum())),
262         "D": (sum(candidate_travel.groupby("match_id").travel_burden.min()),
sum(candidate_travel.groupby("match_id").travel_burden.max())),
263     }
264     normalized = {"T": _minmax(raw["T"], *ranges["T"]), "B": _minmax(raw["B"],
*ranges["B"]), "U": raw["U"] / len(selected), "H": raw["H"] / len(selected
), "C": _minmax(raw["C"], *ranges["C"]), "D": _minmax(raw["D"], *ranges["D"
]), "F": raw["F"], "R": raw["R"]}
265     objective = 0.25 * normalized["T"] + 0.25 * normalized["B"] + 0.15 *
normalized["U"] + 0.10 * normalized["H"] - 0.08 * normalized["C"] - 0.07 *
normalized["D"] - 0.06 * normalized["F"] - 0.04 * normalized["R"]
266     selected["fairness_penalty"] = fairness
267     selected["total_objective_value"] = np.nan
268     selected.loc[selected.index[0], "total_objective_value"] = objective
269     report = {**{"raw_{key}": value for key, value in raw.items()}, **{"f"
norm_{key}": value for key, value in normalized.items()}, "objective":
objective}
270     return selected, report

```

src/c_contest_q1/q2_copt.py

```

1  """Restricted-candidate COPT MIP for the Q2 venue/time joint schedule.
2
3  The evaluation COPT license limits MIP models to 2,000 variables. We
    therefore
4  retain each incumbent venue plus the two largest security-eligible
    alternatives
5  and every candidate time slot on that match's incumbent reference date. The

```

```

6 restriction is explicit, reproducible, and preserves the official hard
7 constraints. The full P2 evaluator still scores the selected schedule.
8 """
9
10 from __future__ import annotations
11
12 from dataclasses import dataclass
13 from pathlib import Path
14 from zoneinfo import ZoneInfo
15
16 import numpy as np
17 import pandas as pd
18
19 from .data import Q1Tables
20 from .q2_demo import _load_q2_tables, evaluate_q2_schedule
21
22
23 @dataclass(frozen=True)
24 class CoptQ2Artifacts:
25     schedule: pd.DataFrame
26     candidate_count: int
27     status: int
28     mip_objective: float
29     score_report: dict[str, float]
30     solve_evidence: dict[str, float | int]
31
32
33 def _candidate_travel(tables: Q1Tables, incumbent: pd.DataFrame, matches: pd.
34 DataFrame, venues: pd.DataFrame) -> pd.DataFrame:
35     """Construct official travel burdens, normalizing before candidate
36     restriction."""
37
38     distance = pd.read_excel(tables.paths.workbook, sheet_name="
39 distance_matrix")
40     team_id = tables.teams.set_index("team_name")["team_id"].to_dict()
41     eligible = {row.match_id: venues.loc[venues.security_level.ge(row.
42 required_security_level), "venue_id"].astype(str).tolist() for row in
43 matches.itertuples(index=False)}
44     by_team_round: dict[tuple[str, int], object] = {}
45     for row in matches.itertuples(index=False):
46         by_team_round[(row.team_a, int(row.round_in_group))] = row
47         by_team_round[(row.team_b, int(row.round_in_group))] = row
48     allowed = incumbent.groupby("match_id").candidate_venue_id.unique().
49 to_dict()
50     rows: list[dict[str, object]] = []

```

```

45     for row in matches.itertuples(index=False):
46         for team in (row.team_a, row.team_b):
47             for venue_id in eligible[row.match_id]:
48                 if int(row.round_in_group) == 1:
49                     candidate = distance.loc[(distance.relation_type.eq("
team_to_venue")) & distance.origin_id.eq(team_id[team]) & distance.
destination_id.astype(str).eq(venue_id)]
50                 else:
51                     previous = by_team_round[(team, int(row.round_in_group) -
1)]
52                     candidate = distance.loc[(distance.relation_type.eq("
venue_to_venue")) & distance.origin_id.astype(str).isin(eligible[previous.
match_id]) & distance.destination_id.astype(str).eq(venue_id)]
53                     if candidate.empty:
54                         raise ValueError(f"missing travel record for {row.match_id
}/{team}/{venue_id}")
55                     rows.append({"match_id": row.match_id, "candidate_venue_id":
venue_id, "distance_km": candidate.distance_km.mean(), "travel_time_hour":
candidate.travel_time_hour.mean(), "timezone_diff": candidate.timezone_diff
.mean()})
56     travel = pd.DataFrame(rows)
57     for column in ("distance_km", "travel_time_hour", "timezone_diff"):
58         spread = travel[column].max() - travel[column].min()
59         travel[f"n_{column}"] = 0.0 if np.isclose(spread, 0) else (travel[
column] - travel[column].min()) / spread
60     travel["travel_burden"] = 0.5 * travel.n_distance_km + 0.3 * travel.
n_travel_time_hour + 0.2 * travel.n_timezone_diff
61     travel = travel.groupby(["match_id", "candidate_venue_id"], as_index=False
).travel_burden.mean()
62     keep = pd.DataFrame(
63         [(match_id, venue_id) for match_id, venue_ids in allowed.items() for
venue_id in venue_ids],
64         columns=["match_id", "candidate_venue_id"],
65     )
66     return keep.merge(travel, on=["match_id", "candidate_venue_id"], validate=
"one_to_one")
67
68
69 def build_q2_copt_candidates(tables: Q1Tables, incumbent: pd.DataFrame) -> pd.
DataFrame:
70     data = _load_q2_tables(tables)
71     venues, slots = data["venues"].copy(), data["time_slots"].copy()
72     slots["date"] = pd.to_datetime(slots.date).dt.date.astype(str)
73     slots["utc_datetime"] = pd.to_datetime(slots.reference_utc_time, utc=True)
74     q1_viewers = incumbent[["match_id", "expected_tv_viewers"]].rename(columns

```

```

    ={"expected_tv_viewers": "predicted_tv_viewers"})
75 matches = tables.group_matches.merge(tables.base_predictions, on=["
match_id", "group_id", "round_in_group", "team_a", "team_b"]).merge(
q1_viewers, on="match_id").merge(data["security_requirements"][["match_id",
"required_security_level"]], on="match_id")
76 incumbent = incumbent[["match_id", "venue_id", "reference_date"]].copy()
77 incumbent["venue_id"] = incumbent.venue_id.astype(str)
78 rows: list[dict[str, object]] = []
79 for match in matches.itertuples(index=False):
80     incumbent_venue = incumbent.loc[incumbent.match_id.eq(match.match_id),
"venue_id"].iloc[0]
81     alternatives = venues.loc[venues.security_level.ge(match.
required_security_level)].sort_values(["capacity", "venue_id"], ascending=[
False, True]).venue_id.astype(str).tolist()
82     candidate_venues = list(dict.fromkeys([incumbent_venue, *alternatives
[:2]]))
83     for venue_id in candidate_venues:
84         rows.append({"match_id": match.match_id, "candidate_venue_id":
venue_id})
85 restricted = pd.DataFrame(rows).drop_duplicates()
86 travel = _candidate_travel(tables, restricted, matches, venues)
87 base = incumbent[["match_id", "reference_date"]].merge(matches, on="
match_id").merge(restricted, on="match_id").merge(travel, on=["match_id", "
candidate_venue_id"])
88 base = base.merge(venues.add_prefix("venue_").rename(columns={"
venue_venue_id": "candidate_venue_id"}), on="candidate_venue_id")
89 economics = data["ticket_broadcast"].set_index("match_stage")
90 for index, row in base.iterrows():
91     economics_row = economics.loc[f"Group_Match_R{int(row.round_in_group)}
"]
92     base.loc[index, "ticket_revenue_usd"] = min(float(row.
expected_attendance_base), float(row.venue_capacity)) * float(economics_row
.base_ticket_price_usd)
93     base.loc[index, "operation_cost_musd"] = float(row.
venue_operation_cost_musd_per_match) + 0.1 * int(row.
required_security_level) * float(row.venue_security_cost_index)
94     base.loc[index, "risk_index"] = 0.5 * float(row.venue_climate_risk) +
0.3 * min(float(row.expected_attendance_base), float(row.venue_capacity)) /
float(row.venue_capacity) + 0.2 * int(row.required_security_level) / int(
row.venue_security_level)
95 candidates = base.merge(slots[["slot_id", "date", "reference_kickoff_time"
, "utc_datetime", "global_prime_score", "broadcast_capacity"]], left_on="
reference_date", right_on="date")
96 candidates["slot_id"] = candidates.slot_id.astype(str)
97 candidates["broadcast_value_usd"] = candidates.predicted_tv_viewers *

```

```

candidates.apply(lambda row: float(economics.loc[f"Group_Match_R{int(row.
round_in_group)}"].broadcast_unit_value_usd) * float(row.global_prime_score
) * float(economics.loc[f"Group_Match_R{int(row.round_in_group)}"].
sponsor_weight), axis=1)
98 # Use the full official candidate universe for every Min-Max bound, even
99 # though the evaluation-license MIP retains only three venues per match.
100 eligible = {
101     row.match_id: venues.loc[
102         venues.security_level.ge(row.required_security_level), "venue_id"
103     ].astype(str).tolist()
104     for row in matches.itertuples(index=False)
105 }
106 ticket_bounds: dict[str, tuple[float, float]] = {}
107 broadcast_bounds: dict[str, tuple[float, float]] = {}
108 operation_candidates: list[float] = []
109 for row in matches.itertuples(index=False):
110     price = economics.loc[f"Group_Match_R{int(row.round_in_group)}"]
111     ticket_values = [
112         min(float(row.expected_attendance_base), float(venue.capacity))
113         * float(price.base_ticket_price_usd)
114         for venue in venues.loc[venues.venue_id.astype(str).isin(eligible[
row.match_id])].itertuples(index=False)
115     ]
116     broadcast_values = [
117         float(row.predicted_tv_viewers)
118         * float(price.broadcast_unit_value_usd)
119         * float(slot.global_prime_score)
120         * float(price.sponsor_weight)
121         for slot in slots.itertuples(index=False)
122     ]
123     operation_candidates.extend(
124         float(venue.operation_cost_musd_per_match)
125         + 0.1 * int(row.required_security_level) * float(venue.
security_cost_index)
126         for venue in venues.loc[venues.venue_id.astype(str).isin(eligible[
row.match_id])].itertuples(index=False)
127     )
128     ticket_bounds[row.match_id] = (min(ticket_values), max(ticket_values))
129     broadcast_bounds[row.match_id] = (min(broadcast_values), max(
broadcast_values))
130
131 travel_all = _candidate_travel(
132     tables,
133     pd.DataFrame(
134         [(match_id, venue_id) for match_id, venue_ids in eligible.items()

```



```

    for venue_id in venue_ids],
135         columns=["match_id", "candidate_venue_id"],
136     ),
137     matches,
138     venues,
139 )
140 ranges = {
141     "T": (
142         sum(value[0] for value in ticket_bounds.values()),
143         sum(value[1] for value in ticket_bounds.values()),
144     ),
145     "B": (
146         sum(value[0] for value in broadcast_bounds.values()),
147         sum(value[1] for value in broadcast_bounds.values()),
148     ),
149     "C": (
150         len(matches) * min(operation_candidates),
151         len(matches) * max(operation_candidates) + float(venues.
setup_cost_musd.sum()),
152     ),
153     "D": (
154         float(travel_all.groupby("match_id").travel_burden.min().sum()),
155         float(travel_all.groupby("match_id").travel_burden.max().sum()),
156     ),
157 }
158 candidates.attrs["z2_ranges"] = ranges
159 candidates.attrs["z2_fixed"] = {
160     "U": float(matches.uncertainty_index.sum() / len(matches)),
161     "H": float((matches.attractiveness_index / 100).sum() / len(matches)),
162 }
163 return candidates
164
165
166 def _scaled_coefficient(weight: float, bounds: tuple[float, float]) -> tuple[
float, float]:
167     lower, upper = bounds
168     if np.isclose(lower, upper):
169         return 0.0, 0.0
170     return weight / (upper - lower), -weight * lower / (upper - lower)
171
172
173 def _copt_log_name(path: Path) -> str:
174     """Use an ASCII relative name because COPT 8.0.6 cannot open Chinese
Windows paths."""
175

```

```

176     resolved = path.resolve()
177     try:
178         return resolved.relative_to(Path.cwd().resolve()).as_posix()
179     except ValueError:
180         return str(resolved)
181
182
183 def solve_copt_q2(
184     tables: Q1Tables,
185     incumbent_csv: Path,
186     log_path: Path | None = None,
187 ) -> CoptQ2Artifacts:
188     try:
189         import coptpy as cp
190     except ImportError as exc: # pragma: no cover - environment dependent
191         raise RuntimeError("coptpy is required for the COPT Q2 run") from exc
192     incumbent = pd.read_csv(incumbent_csv)
193     candidates = build_q2_copt_candidates(tables, incumbent)
194     ranges = candidates.attrs["z2_ranges"]
195     fixed = candidates.attrs["z2_fixed"]
196     data = _load_q2_tables(tables)
197     venues, slots = data["venues"].copy(), data["time_slots"].copy()
198     slots["slot_id"] = slots.slot_id.astype(str)
199     prime = set(slots.nlargest(len(slots) // 4, "global_prime_score").slot_id)
200     limits = data["dynamic_resource_limits"].copy()
201     limits["reference_date"] = limits.reference_date.astype(str)
202     env = cp.Envr()
203     model = env.createModel("q2_restricted_venue_slot_mip")
204     if log_path is not None:
205         log_path.parent.mkdir(parents=True, exist_ok=True)
206         log_path.unlink(missing_ok=True)
207         model.setLogFile(_copt_log_name(log_path))
208     x = {index: model.addVar(vtype=cp.COPT.BINARY, name=f"x_{index}") for
index in candidates.index}
209     y = {str(venue): model.addVar(vtype=cp.COPT.BINARY, name=f"use_{venue}")
for venue in venues.venue_id}
210     for match_id, block in candidates.groupby("match_id"):
211         model.addConstr(cp.quicksum(x[index] for index in block.index) == 1,
name=f"assignment_{match_id}")
212     for venue in venues.itertuples(index=False):
213         block = candidates.loc[candidates.candidate_venue_id.eq(str(venue.
venue_id))]
214         # The input table imposes each venue's stage-wide lower bound directly
;
215         # it is not an optional activation threshold.

```

```

216     model.addConstr(cp.quicksum(x[index] for index in block.index) >= int(
venue.min_total_matches), name=f"min_use_{venue.venue_id}")
217     model.addConstr(cp.quicksum(x[index] for index in block.index) <= int(
venue.max_total_matches) * y[str(venue.venue_id)], name=f"max_use_{venue.
venue_id}")
218     for (venue, date), block in candidates.groupby(["candidate_venue_id", "
reference_date"]):
219         max_daily = int(venues.loc[venues.venue_id.astype(str).eq(venue), "
max_matches_per_day"].iloc[0])
220         model.addConstr(cp.quicksum(x[index] for index in block.index) <=
max_daily, name=f"daily_{venue}_{date}")
221         for slot_id, block in candidates.groupby("slot_id"):
222             capacity = int(slots.loc[slots.slot_id.eq(slot_id), "
broadcast_capacity"].iloc[0])
223             model.addConstr(cp.quicksum(x[index] for index in block.index) <=
capacity, name=f"slot_{slot_id}")
224         for (venue, utc), block in candidates.groupby(["candidate_venue_id", "
utc_datetime"]):
225             model.addConstr(cp.quicksum(x[index] for index in block.index) <= 1,
name=f"collision_{venue}_{str(utc)[:10]}")
226             g_max, g_min = model.addVar(lb=0.0, ub=3.0, name="golden_max"), model.
addVar(lb=0.0, ub=3.0, name="golden_min")
227             l_max, l_min = model.addVar(lb=0.0, ub=3.0, name="large_venue_max"), model
.addVar(lb=0.0, ub=3.0, name="large_venue_min")
228             large = set(venues.nlargest(len(venues) // 4, "capacity").venue_id.astype(
str))
229             for team in sorted(set(incumbent.team_a).union(incumbent.team_b)):
230                 team_block = candidates.loc[(candidates.team_a.eq(team) | candidates.
team_b.eq(team)) & candidates.slot_id.isin(prime)]
231                 total = cp.quicksum(x[index] for index in team_block.index)
232                 model.addConstr(total <= g_max, name=f"golden_max_{team}")
233                 model.addConstr(total >= g_min, name=f"golden_min_{team}")
234                 large_block = candidates.loc[(candidates.team_a.eq(team) | candidates.
team_b.eq(team)) & candidates.candidate_venue_id.isin(large)]
235                 large_total = cp.quicksum(x[index] for index in large_block.index)
236                 model.addConstr(large_total <= l_max, name=f"large_max_{team}")
237                 model.addConstr(large_total >= l_min, name=f"large_min_{team}")
238             model.addConstr(g_max - g_min <= 2, name="golden_fairness")
239             r3 = candidates.loc[candidates.round_in_group.eq(3)]
240             for date, block in r3.groupby("reference_date"):
241                 capacity = int(limits.loc[limits.reference_date.eq(date), "
high_security_capacity"].iloc[0])
242                 high = block.loc[block.required_security_level.ge(3)]
243                 model.addConstr(cp.quicksum(x[index] for index in high.index) <=
capacity, name=f"third_round_security_{date}")

```

```

244     t_coef, t_constant = _scaled_coefficient(0.25, ranges["T"])
245     b_coef, b_constant = _scaled_coefficient(0.25, ranges["B"])
246     c_coef, c_constant = _scaled_coefficient(-0.08, ranges["C"])
247     d_coef, d_constant = _scaled_coefficient(-0.07, ranges["D"])
248     fixed_constant = t_constant + b_constant + c_constant + d_constant + 0.15
249     * fixed["U"] + 0.10 * fixed["H"]
250     additive = cp.quicksum(
251         (
252             t_coef * float(candidates.loc[index, "ticket_revenue_usd"])
253             + b_coef * float(candidates.loc[index, "broadcast_value_usd"])
254             + c_coef * float(candidates.loc[index, "operation_cost_musd"])
255             + d_coef * float(candidates.loc[index, "travel_burden"])
256             - 0.04 / len(incumbent) * float(candidates.loc[index, "risk_index"]
257         )) * x[index]
258     for index in candidates.index
259 )
260     setup = cp.quicksum(
261         c_coef * float(venues.loc[venues.venue_id.astype(str).eq(venue), "
262     setup_cost_musd"].iloc[0]) * y[venue]
263     for venue in y
264 )
265     fairness = -0.01 * (g_max - g_min) - 0.01 * (l_max - l_min)
266     model.setObjective(fixed_constant + additive + setup + fairness, cp.COPT.
267     MAXIMIZE)
268     model.solve()
269     if not getattr(model, "hasmipsol", False) and not getattr(model, "
270     hasmipSol", False):
271         raise RuntimeError(f"COPT returned no MIP solution, status={model.
272     status}")
273     chosen = candidates.loc[[index for index, variable in x.items() if
274     variable.x > 0.5]].copy()
275     if len(chosen) != 72:
276         raise RuntimeError("COPT did not choose exactly 72 match assignments")
277     output = chosen.rename(columns={"candidate_venue_id": "venue_id", "
278     venue_city": "city", "venue_country": "country", "venue_timezone": "
279     timezone"})
280     output["local_datetime"] = [pd.Timestamp(value).tz_convert(ZoneInfo(
281     timezone)).isoformat() for value, timezone in zip(output.utc_datetime,
282     output.timezone, strict=True)]
283     output["utc_datetime"] = output.utc_datetime.map(lambda value: pd.
284     Timestamp(value).isoformat())
285     output["expected_attendance"] = np.minimum(output.expected_attendance_base
286     , output.venue_capacity)
287     output["expected_tv_viewers"] = output.predicted_tv_viewers

```

```

276     output["fairness_penalty"] = np.nan
277     output["travel_cost_index"] = output.travel_burden
278     output["total_objective_value"] = np.nan
279     columns = ["match_id", "group_id", "round_in_group", "team_a", "team_b", "
venue_id", "slot_id", "city", "country", "reference_date", "
reference_kickoff_time", "local_datetime", "utc_datetime", "
required_security_level", "expected_attendance", "expected_tv_viewers", "
ticket_revenue_usd", "broadcast_value_usd", "travel_cost_index", "
fairness_penalty", "risk_index", "total_objective_value"]
280     scored, report = evaluate_q2_schedule(tables, output[columns])
281     if not np.isclose(float(model.objval), report["objective"], atol=1e-8):
282         raise RuntimeError(
283             f"COPT Z2 {float(model.objval)} disagrees with independent
evaluator {report['objective']}")
284     )
285     evidence = {
286         "rows": int(model.getAttr(cp.COPT.Attr.Rows)),
287         "columns": int(model.getAttr(cp.COPT.Attr.Cols)),
288         "binary_variables": int(model.getAttr(cp.COPT.Attr.Bins)),
289         "best_bound": float(model.getAttr(cp.COPT.Attr.BestObj)),
290         "relative_gap": float(model.getAttr(cp.COPT.Attr.BestGap)),
291         "solve_time_seconds": float(model.getAttr(cp.COPT.Attr.SolvingTime)),
292     }
293     return CoptQ2Artifacts(
294         schedule=scored[columns].sort_values("match_id").reset_index(drop=True
),
295         candidate_count=len(candidates),
296         status=int(model.status),
297         mip_objective=float(model.objval),
298         score_report=report,
299         solve_evidence=evidence,
300     )

```

src/c_contest_q1/q3_demo.py

```

1  """Official Q3 state update and synchronous qualification simulation."""
2
3  from __future__ import annotations
4
5  from dataclasses import dataclass
6  from pathlib import Path
7
8  import numpy as np
9  import pandas as pd
10

```

```

11 from .data import Q1Tables
12
13
14 OUTPUT_COLUMNS = [
15     "match_id", "group_id", "team_a", "team_b",
16     "updated_p_team_a_advance", "updated_p_team_b_advance",
17     "updated_expected_attendance", "updated_expected_tv_viewers",
18     "stakeless_risk", "collusion_risk", "updated_attractiveness",
19     "recommended_broadcast_priority", "recommended_security_level",
20     "recommended_transport_level", "recommended_ticket_adjustment",
21     "updated_ticket_revenue_usd", "updated_broadcast_value_usd",
22     "resource_cost_index", "risk_exposure_index", "static_net_value",
23     "dynamic_net_value", "improvement_rate",
24 ]
25
26
27 @dataclass(frozen=True)
28 class Q3StateArtifacts:
29     matches: pd.DataFrame
30     simulation_summary: pd.DataFrame
31
32
33 @dataclass(frozen=True)
34 class Q3Validation:
35     errors: tuple[str, ...]
36
37     @property
38     def ok(self) -> bool:
39         return not self.errors
40
41
42 def _table(tables: Q1Tables, name: str) -> pd.DataFrame:
43     return pd.read_excel(tables.paths.workbook, sheet_name=name)
44
45
46 def _team_state(live: pd.DataFrame) -> pd.DataFrame:
47     records: list[dict[str, float | str]] = []
48     for row in live.itertuples(index=False):
49         for side, other in (("a", "b"), ("b", "a")):
50             gf, ga = getattr(row, f"goals_{side}"), getattr(row, f"goals_{
other}")
51             records.append({
52                 "team": getattr(row, f"team_{side}"),
53                 "points": 3 if gf > ga else 1 if gf == ga else 0,
54                 "gf": gf, "ga": ga,

```

```

55         "xgf": getattr(row, f"xg_{side}"), "xga": getattr(row, f"xg_{
other}")),
56         "red": getattr(row, f"red_cards_{side}"), "games": 1,
57     })
58     state = pd.DataFrame(records).groupby("team", as_index=False).sum(
numeric_only=True)
59     state["s"] = (
60         0.45 * state.points / (3 * state.games)
61         + 0.35 * (0.5 + 0.5 * np.tanh(((state.xgf - state.xga) / state.games)
/ 1.25))
62         + 0.20 * np.exp(-0.55 * state.red / state.games)
63     ).clip(0.0, 1.0)
64     return state
65
66
67 def _simulate_qualification(
68     r3: pd.DataFrame,
69     state: pd.DataFrame,
70     n_simulations: int = 20_000,
71     seed: int = 20260814,
72 ) -> pd.DataFrame:
73     """Simulate all 24 R3 matches together and retain conditional
qualification risks."""
74
75     rng = np.random.default_rng(seed)
76     r3 = r3.copy().reset_index(drop=True)
77     state_idx = state.set_index("team")
78     r3["state_a"] = r3.team_a.map(state_idx.s)
79     r3["state_b"] = r3.team_b.map(state_idx.s)
80     delta = 0.32 * (r3.state_a - r3.state_b) - 0.055 * (r3.injury_a - r3.
injury_b)
81     r3["lambda_a"] = np.clip(r3.expected_goals_a * np.exp(delta - 0.04 * r3.
injury_a), 0.15, 4.50)
82     r3["lambda_b"] = np.clip(r3.expected_goals_b * np.exp(-delta - 0.04 * r3.
injury_b), 0.15, 4.50)
83     if not np.isfinite(r3[["lambda_a", "lambda_b"]].to_numpy()).all():
84         raise ValueError("third-round Poisson means must be finite")
85
86     groups = sorted(r3.group_id.unique())
87     team_sets = [
88         sorted(set(r3.loc[r3.group_id.eq(group), "team_a"]).union(r3.loc[r3.
group_id.eq(group), "team_b"]))
89         for group in groups
90     ]
91     group_lookup = {group: index for index, group in enumerate(groups)}

```

```

92     team_lookup = {
93         group: {team: index for index, team in enumerate(teams)}
94         for group, teams in zip(groups, team_sets, strict=True)
95     }
96     points = np.array([[state_idx.loc[team, "points"] for team in teams] for
97                        teams in team_sets], dtype=float)
98     gf = np.array([[state_idx.loc[team, "gf"] for team in teams] for teams in
99                   team_sets], dtype=float)
100    ga = np.array([[state_idx.loc[team, "ga"] for team in teams] for teams in
101                  team_sets], dtype=float)
102    match_group = r3.group_id.map(group_lookup).to_numpy()
103    match_a = np.array([team_lookup[group][team] for group, team in zip(r3.
104                              group_id, r3.team_a, strict=True)])
105    match_b = np.array([team_lookup[group][team] for group, team in zip(r3.
106                              group_id, r3.team_b, strict=True)])
107
108    sim_points = np.broadcast_to(points, (n_simulations, *points.shape)).copy()
109
110    sim_gf = np.broadcast_to(gf, (n_simulations, *gf.shape)).copy()
111    sim_ga = np.broadcast_to(ga, (n_simulations, *ga.shape)).copy()
112    scores_a = rng.poisson(r3.lambda_a.to_numpy(), size=(n_simulations, len(r3
113    )))
114    scores_b = rng.poisson(r3.lambda_b.to_numpy(), size=(n_simulations, len(r3
115    )))
116    draws = np.arange(n_simulations)
117    for index in range(len(r3)):
118        group, a, b = match_group[index], match_a[index], match_b[index]
119        goals_a, goals_b = scores_a[:, index], scores_b[:, index]
120        sim_gf[:, group, a] += goals_a
121        sim_ga[:, group, a] += goals_b
122        sim_gf[:, group, b] += goals_b
123        sim_ga[:, group, b] += goals_a
124        sim_points[:, group, a] += np.where(goals_a > goals_b, 3, np.where(
125        goals_a == goals_b, 1, 0))
126        sim_points[:, group, b] += np.where(goals_b > goals_a, 3, np.where(
127        goals_a == goals_b, 1, 0))
128
129    sim_gd = sim_gf - sim_ga
130    qualified = np.zeros((n_simulations, len(groups), 4), dtype=bool)
131    third_team = np.empty((n_simulations, len(groups)), dtype=int)
132    for group_index in range(len(groups)):
133        order = np.lexsort((rng.random((n_simulations, 4)), sim_gf[:,
134        group_index], sim_gd[:, group_index], sim_points[:, group_index]), axis=1)
135        qualified[draws[:, None], group_index, order[:, :2]] = True

```



```

125     third_team[:, group_index] = order[:, 2]
126     third_points = np.take_along_axis(sim_points, third_team[:, :, None], axis
=2).squeeze(2)
127     third_gd = np.take_along_axis(sim_gd, third_team[:, :, None], axis=2).
squeeze(2)
128     third_gf = np.take_along_axis(sim_gf, third_team[:, :, None], axis=2).
squeeze(2)
129     third_order = np.lexsort((rng.random((n_simulations, len(groups))),
third_gf, third_gd, third_points), axis=1)[:, ::-1]
130     for rank in range(8):
131         selected_group = third_order[:, rank]
132         qualified[draws, selected_group, third_team[draws, selected_group]] =
True
133
134     probabilities: dict[str, float] = {}
135     for group_index, teams in enumerate(team_sets):
136         for team_index, team in enumerate(teams):
137             probabilities[team] = float(qualified[:, group_index, team_index].
mean())
138     r3["updated_p_team_a_advance"] = r3.team_a.map(probabilities)
139     r3["updated_p_team_b_advance"] = r3.team_b.map(probabilities)
140     r3["stakeless_risk"] = (
141         0.5 * (2 * r3.updated_p_team_a_advance - 1) ** 2
142         + 0.5 * (2 * r3.updated_p_team_b_advance - 1) ** 2
143     )
144
145     collusion: list[float] = []
146     conditional_rows: list[dict[str, float | str]] = []
147     for index, match in enumerate(r3.itertuples(index=False)):
148         group, a, b = match_group[index], match_a[index], match_b[index]
149         q_a, q_b = qualified[:, group, a], qualified[:, group, b]
150         draw_mask = scores_a[:, index] == scores_b[:, index]
151         a_win_mask = scores_a[:, index] > scores_b[:, index]
152         b_win_mask = scores_b[:, index] > scores_a[:, index]
153         p_a_draw = float(q_a[draw_mask].mean())
154         p_b_draw = float(q_b[draw_mask].mean())
155         p_a_win = float(q_a[a_win_mask].mean())
156         p_b_win = float(q_b[b_win_mask].mean())
157         common_draw = min(p_a_draw, p_b_draw)
158         incentive_gain = 0.5 * max(p_a_win - p_a_draw, 0.0) + 0.5 * max(
p_b_win - p_b_draw, 0.0)
159         risk = float(np.clip(common_draw * (1 - np.clip(incentive_gain, 0, 1))
* (1 - 0.35 * r3.loc[index, "stakeless_risk"]), 0, 1))
160         collusion.append(risk)
161         conditional_rows.append({

```

```

162         "match_id": match.match_id,
163         "p_a_advance_given_draw": p_a_draw,
164         "p_b_advance_given_draw": p_b_draw,
165         "p_a_advance_given_a_win": p_a_win,
166         "p_b_advance_given_b_win": p_b_win,
167         "common_draw_benefit": common_draw,
168         "win_incentive_gain": incentive_gain,
169     })
170     r3["collusion_risk"] = collusion
171     conditional = pd.DataFrame(conditional_rows)
172     r3 = r3.merge(conditional, on="match_id", how="left")
173     return r3
174
175
176 def build_q3_state(
177     tables: Q1Tables,
178     q2_csv: Path,
179     q1_csv: Path,
180     n_simulations: int = 20_000,
181     seed: int = 20260814,
182 ) -> Q3StateArtifacts:
183     """Build every pre-resource static and feedback-updated dynamic Q3
184     quantity."""
185
186     live = _table(tables, "live_group_results")
187     base = _table(tables, "base_predictions")
188     venues = _table(tables, "venues")
189     ticket = _table(tables, "ticket_broadcast")
190     security = _table(tables, "security_requirements")
191     schedule = pd.read_csv(q2_csv)
192     q1 = pd.read_csv(q1_csv)
193     r3 = schedule.loc[schedule.round_in_group.eq(3)].copy()
194     r3 = r3.drop(columns=[column for column in ("expected_tv_viewers",) if
195     column in r3]).merge(q1[["match_id", "predicted_tv_viewers"]], on="match_id
196 ")
197     r3 = r3.merge(base[["match_id", "expected_goals_a", "expected_goals_b", "
198     uncertainty_index", "attractiveness_index"]], on="match_id")
199     r3 = r3.merge(venues[["venue_id", "capacity", "security_level"]].rename(
200     columns={"capacity": "venue_capacity", "security_level": "
201     venue_security_level"}), on="venue_id")
202     r3 = r3.merge(security[["match_id", "security_demand_score"]], on="
203     match_id")
204     r3["match_stage"] = "Group_Match_R3"
205     r3 = r3.merge(ticket, on="match_stage")

```

```

200     injury_records: list[dict[str, float | str]] = []
201     for row in live.itertuples(index=False):
202         injury_records.extend((
203             {"team": row.team_a, "injury": float(row.injury_impact_level)},
204             {"team": row.team_b, "injury": float(row.injury_impact_level)},
205         ))
206     injury = pd.DataFrame(injury_records).groupby("team").injury.mean().clip(
207         (0.0, 3.0)
208     )
209     r3["injury_a"] = r3.team_a.map(injury).fillna(0.0)
210     r3["injury_b"] = r3.team_b.map(injury).fillna(0.0)
211     r3 = _simulate_qualification(r3, _team_state(live), n_simulations=
212         n_simulations, seed=seed)
213
214     prior = live.merge(base[["match_id", "expected_attendance_base"]], on="
215         match_id").merge(q1[["match_id", "predicted_tv_viewers"]], on="match_id")
216     feedback_rows: list[dict[str, float | str]] = []
217     for row in prior.itertuples(index=False):
218         attendance_ratio = float(np.clip(row.attendance / row.
219             expected_attendance_base, 0.60, 1.50))
220         tv_ratio = float(np.clip(row.tv_viewers / row.predicted_tv_viewers,
221             0.60, 1.50))
222         for team in (row.team_a, row.team_b):
223             feedback_rows.append({"team": team, "attendance_ratio":
224                 attendance_ratio, "tv_ratio": tv_ratio})
225     feedback = pd.DataFrame(feedback_rows).groupby("team").mean(numeric_only=
226         True).clip(0.75, 1.25)
227     r3["feedback_attendance"] = np.sqrt(r3.team_a.map(feedback.
228         attendance_ratio) * r3.team_b.map(feedback.attendance_ratio)).fillna(1.0)
229     r3["feedback_tv"] = np.sqrt(r3.team_a.map(feedback.tv_ratio) * r3.team_b.
230         map(feedback.tv_ratio)).fillna(1.0)
231     r3["feedback_index"] = np.clip((0.5 * r3.feedback_attendance + 0.5 * r3.
232         feedback_tv - 0.75) / 0.50, 0.0, 1.0)
233     r3["importance"] = (
234         4 * r3.updated_p_team_a_advance * (1 - r3.updated_p_team_a_advance)
235         + 4 * r3.updated_p_team_b_advance * (1 - r3.updated_p_team_b_advance)
236     ) / 2
237     r3["state_mean"] = (r3.state_a + r3.state_b) / 2
238     r3["injury_mean"] = (r3.injury_a + r3.injury_b) / 6
239     r3["updated_attractiveness"] = np.clip(
240         0.50 * r3.attractiveness_index / 100
241         + 0.25 * r3.importance
242         + 0.12 * r3.feedback_index
243         + 0.08 * r3.state_mean
244         + 0.05 * (1 - r3.injury_mean),
245         0.0, 1.0,

```

```

235 )
236 r3["dynamic_attendance_demand"] = (
237     r3.expected_attendance * r3.feedback_attendance
238     * (0.88 + 0.27 * r3.importance)
239     * (0.94 + 0.12 * r3.state_mean)
240     * (1 - 0.10 * r3.injury_mean)
241 )
242 r3["dynamic_tv_demand"] = (
243     r3.predicted_tv_viewers * r3.feedback_tv
244     * (0.87 + 0.30 * r3.importance)
245     * (0.90 + 0.20 * r3.updated_attractiveness)
246     * (1 - 0.06 * r3.injury_mean)
247 )
248 occupancy = np.clip(r3.dynamic_attendance_demand / r3.venue_capacity, 0.0,
249                     1.0)
250 r3["dynamic_security_demand"] = np.clip(
251     0.45 * r3.security_demand_score + 0.25 * occupancy
252     + 0.15 * r3.updated_attractiveness
253     + 0.15 * (r3.stakeless_risk + r3.collusion_risk) / 2,
254     0.0, 1.0,
255 )
256 r3["dynamic_attractiveness"] = r3.updated_attractiveness
257 r3["dynamic_stakeless_risk"] = r3.stakeless_risk
258 r3["dynamic_collusion_risk"] = r3.collusion_risk
259 r3["static_attractiveness"] = r3.attractiveness_index / 100
260 r3["static_attendance_demand"] = r3.expected_attendance
261 r3["static_tv_demand"] = r3.predicted_tv_viewers
262 r3["static_stakeless_risk"] = 1 - r3.uncertainty_index
263 r3["static_collusion_risk"] = 0.0
264 static_occupancy = np.clip(r3.static_attendance_demand / r3.venue_capacity
265 , 0.0, 1.0)
266 r3["static_security_demand"] = np.clip(
267     0.45 * r3.security_demand_score + 0.25 * static_occupancy
268     + 0.15 * r3.static_attractiveness + 0.15 * r3.static_stakeless_risk /
269     2,
270     0.0, 1.0,
271 )
272 summary_columns = [
273     "match_id", "lambda_a", "lambda_b", "updated_p_team_a_advance",
274     "updated_p_team_b_advance", "p_a_advance_given_draw",
275     "p_b_advance_given_draw", "p_a_advance_given_a_win",
276     "p_b_advance_given_b_win", "common_draw_benefit",
277     "win_incentive_gain", "importance", "updated_attractiveness",
278     "stakeless_risk", "collusion_risk",
279 ]

```

```

277     return Q3StateArtifacts(matches=r3.sort_values("match_id").reset_index(
simulation_summary=r3[summary_columns].sort_values("match_id").
reset_index(drop=True))
278
279
280 def validate_q3_result(tables: Q1Tables, q2_csv: Path, result: pd.DataFrame)
-> Q3Validation:
281     errors: list[str] = []
282     if list(result.columns) != OUTPUT_COLUMNS:
283         return Q3Validation(("result_3 columns do not follow the official
template order",))
284     if len(result) != 24 or not result.match_id.is_unique:
285         errors.append("result_3 must contain each of the 24 third-round
matches exactly once")
286     numeric = result.drop(columns=["match_id", "group_id", "team_a", "team_b"
])
287     allowed_nan = np.asarray(numeric.columns == "improvement_rate")
288     finite_or_allowed = np.isfinite(numeric.to_numpy(dtype=float)) | np.
broadcast_to(allowed_nan, numeric.shape)
289     if not finite_or_allowed.all():
290         errors.append("result_3 contains non-finite required values")
291     probabilities = result[["updated_p_team_a_advance", "
updated_p_team_b_advance"]].to_numpy(dtype=float)
292     if not np.logical_and(probabilities >= 0, probabilities <= 1).all():
293         errors.append("advance probabilities must lie in [0, 1]")
294     if not np.isclose(probabilities.sum(), 32.0, atol=1e-8):
295         errors.append("total expected qualification places must equal 32")
296     for column in ("stakeless_risk", "collusion_risk", "updated_attractiveness
", "risk_exposure_index"):
297         if not result[column].between(0, 1).all():
298             errors.append(f"{column} must lie in [0, 1]")
299
300     schedule = pd.read_csv(q2_csv).loc[lamba frame: frame.round_in_group.eq
(3), ["match_id", "venue_id", "reference_date", "required_security_level"]]
301     venues = _table(tables, "venues").set_index("venue_id")
302     limits = _table(tables, "dynamic_resource_limits").copy()
303     limits["reference_date"] = limits.reference_date.astype(str)
304     checked = result.merge(schedule, on="match_id", how="left")
305     if checked[["venue_id", "reference_date"]].isna().any().any():
306         return Q3Validation(tuple(dict.fromkeys(*errors, "result_3 does not
match the fixed third-round schedule"))))
307     for row in checked.itertuples(index=False):
308         if not int(row.required_security_level) <= int(row.
recommended_security_level) <= int(venues.loc[row.venue_id, "security_level
"]):

```

```

309         errors.append(f"security level outside permitted range for {row.
match_id}")
310     for date, day in checked.groupby("reference_date"):
311         limit = limits.loc[limits.reference_date.eq(str(date))].iloc[0]
312         if day.recommended_broadcast_priority.eq(3).sum() > int(limit.
high_broadcast_capacity):
313             errors.append(f"high broadcast capacity exceeded on {date}")
314             if day.recommended_security_level.ge(3).sum() > int(limit.
high_security_capacity):
315                 errors.append(f"high security capacity exceeded on {date}")
316                 if day.recommended_transport_level.eq(3).sum() > int(limit.
enhanced_transport_capacity):
317                     errors.append(f"enhanced transport capacity exceeded on {date}")
318                     if day.resource_cost_index.sum() > float(limit.
daily_resource_budget_index) + 1e-9:
319                         errors.append(f"daily resource budget exceeded on {date}")
320                         if not day.recommended_ticket_adjustment.between(-float(limit.
max_ticket_discount_rate), float(limit.max_ticket_increase_rate)).all():
321                             errors.append(f"ticket adjustment outside daily bounds on {date}")
322     return Q3Validation(tuple(dict.fromkeys(errors)))

```

src/c_contest_q1/q3_copt.py

```

1  """Formal COPT static/dynamic resource optimization for Question 3."""
2
3  from __future__ import annotations
4
5  from dataclasses import dataclass
6  from pathlib import Path
7
8  import numpy as np
9  import pandas as pd
10
11  from .data import Q1Tables
12  from .q3_demo import OUTPUT_COLUMNS, build_q3_state
13
14
15  @dataclass(frozen=True)
16  class CoptQ3Artifacts:
17      result: pd.DataFrame
18      simulation_summary: pd.DataFrame
19      static_selection: pd.DataFrame
20      dynamic_selection: pd.DataFrame
21      lower_bound_selection: pd.DataFrame
22      candidate_count: int

```

```

23     static_status: int
24     dynamic_status: int
25     lower_bound_status: int
26     static_optimization_objective: float
27     static_updated_evaluation: float
28     dynamic_optimization_objective: float
29     feasible_objective_lower_bound: float
30     solve_evidence: dict[str, dict[str, float | int]]
31
32
33 def _table(tables: Q1Tables, name: str) -> pd.DataFrame:
34     return pd.read_excel(tables.paths.workbook, sheet_name=name)
35
36
37 def _normalization_bounds(candidates: pd.DataFrame, prefix: str) -> dict[str,
    tuple[float, float]]:
38     return {
39         metric: (float(candidates[f"{prefix}_{metric}"].min()), float(
    candidates[f"{prefix}_{metric}"].max()))
40         for metric in ("ticket_value", "broadcast_value", "attractiveness", "
    resource_cost", "risk")
41     }
42
43
44 def _normalize(values: pd.Series, bounds: tuple[float, float]) -> pd.Series:
45     lower, upper = bounds
46     if np.isclose(lower, upper):
47         return pd.Series(0.0, index=values.index)
48     return (values - lower) / (upper - lower)
49
50
51 def _add_scores(candidates: pd.DataFrame, prefix: str, bounds: dict[str, tuple
    [float, float]], score_name: str) -> None:
52     normalized: dict[str, pd.Series] = {}
53     for metric in bounds:
54         normalized[metric] = _normalize(candidates[f"{prefix}_{metric}"],
    bounds[metric])
55     candidates[score_name] = (
56         0.35 * normalized["ticket_value"]
57         + 0.35 * normalized["broadcast_value"]
58         + 0.10 * normalized["attractiveness"]
59         - 0.10 * normalized["resource_cost"]
60         - 0.10 * normalized["risk"]
61     )
62

```

```

63
64 def _copt_log_name(path: Path) -> str:
65     """Use an ASCII relative name because COPT 8.0.6 cannot open Chinese
        Windows paths."""
66
67     resolved = path.resolve()
68     try:
69         return resolved.relative_to(Path.cwd().resolve()).as_posix()
70     except ValueError:
71         return str(resolved)
72
73
74 def build_q3_candidates(tables: Q1Tables, matches: pd.DataFrame) -> tuple[pd.
    DataFrame, dict[str, tuple[float, float]]]:
75     """Enumerate a common configuration set and evaluate it in both
        information environments."""
76
77     costs = _table(tables, "dynamic_resource_costs").set_index(["resource_type",
        "resource_level"])
78     limits = _table(tables, "dynamic_resource_limits").copy()
79     limits["reference_date"] = limits.reference_date.astype(str)
80     rows: list[dict[str, float | int | str]] = []
81     for match in matches.itertuples(index=False):
82         limit = limits.loc[limits.reference_date.eq(str(match.reference_date))
        ].iloc[0]
83         price_grid = np.linspace(-float(limit.max_ticket_discount_rate), float
        (limit.max_ticket_increase_rate), 5)
84         for broadcast in (1, 2, 3):
85             for security in range(int(match.required_security_level), int(
        match.venue_security_level) + 1):
86                 for transport in (1, 2, 3):
87                     for delta in price_grid:
88                         multipliers = {
89                             "broadcast": float(costs.loc[("broadcast",
        broadcast), "demand_multiplier"]),
90                             "transport": float(costs.loc[("transport",
        transport), "demand_multiplier"]),
91                             "risk": float(costs.loc[("security", security), "
        risk_multiplier"]),
92                         }
93                         resource_cost = sum(
94                             float(costs.loc[(kind, level), "unit_cost_index"])
95                             for kind, level in (("broadcast", broadcast), ("
        security", security), ("transport", transport))
96                         )

```



```

97         row: dict[str, float | int | str] = {
98             "match_id": match.match_id,
99             "reference_date": str(match.reference_date),
100             "broadcast": broadcast,
101             "security": security,
102             "transport": transport,
103             "ticket_adjustment": float(delta),
104         }
105         valid = True
106         for prefix in ("static", "dynamic"):
107             attendance_demand = float(getattr(match, f"{prefix}
attendance_demand"))
108             no_price_attendance = min(float(match.
venue_capacity), attendance_demand * multipliers["transport"])
109             attendance = min(
110                 float(match.venue_capacity),
111                 attendance_demand * multipliers["transport"] *
(1 + float(delta)) ** float(match.price_elasticity),
112             )
113             if attendance + 1e-9 < 0.88 * no_price_attendance:
114                 valid = False
115                 break
116             viewers = float(getattr(match, f"{prefix}
_tv_demand")) * multipliers["broadcast"]
117             stake = float(getattr(match, f"{prefix}
_stakeless_risk"))
118             collusion = float(getattr(match, f"{prefix}
_collusion_risk"))
119             security_demand = float(getattr(match, f"{prefix}
_security_demand"))
120             row.update({
121                 f"{prefix}_attendance": attendance,
122                 f"{prefix}_tv_viewers": viewers,
123                 f"{prefix}_ticket_value": float(match.
base_ticket_price_usd) * (1 + float(delta)) * attendance,
124                 f"{prefix}_broadcast_value": float(match.
broadcast_unit_value_usd) * viewers,
125                 f"{prefix}_attractiveness": float(getattr(
match, f"{prefix}_attractiveness")),
126                 f"{prefix}_resource_cost": resource_cost,
127                 f"{prefix}_risk": 0.40 * stake + 0.40 *
collusion + 0.20 * security_demand * multipliers["risk"],
128             })
129             if valid:
130                 rows.append(row)

```

```

131     candidates = pd.DataFrame(rows)
132     if candidates.empty or candidates.groupby("match_id").size().size != 24:
133         raise ValueError("common Q3 candidate set is incomplete")
134     dynamic_bounds = _normalization_bounds(candidates, "dynamic")
135     static_bounds = _normalization_bounds(candidates, "static")
136     _add_scores(candidates, "static", static_bounds, "
static_optimization_score")
137     _add_scores(candidates, "dynamic", dynamic_bounds, "
updated_evaluation_score")
138     return candidates, dynamic_bounds
139
140
141 def _solve_selection(
142     candidates: pd.DataFrame,
143     limits: pd.DataFrame,
144     score_column: str,
145     model_name: str,
146     log_path: Path | None = None,
147     maximize: bool = True,
148 ) -> tuple[pd.DataFrame, int, float, dict[str, float | int]]:
149     try:
150         import coptpy as cp
151     except ImportError as exc: # pragma: no cover - environment dependent
152         raise RuntimeError("coptpy is required for Q3 optimization") from exc
153
154     environment = cp.Envr()
155     model = environment.createModel(model_name)
156     if log_path is not None:
157         log_path.parent.mkdir(parents=True, exist_ok=True)
158         log_path.unlink(missing_ok=True)
159         model.setLogFile(_copt_log_name(log_path))
160     decision = {index: model.addVar(vtype=cp.COPT.BINARY, name=f"x_{index}")
for index in candidates.index}
161     for match_id, block in candidates.groupby("match_id"):
162         model.addConstr(cp.quicksum(decision[index] for index in block.index)
== 1, name=f"assign_{match_id}")
163     for date, block in candidates.groupby("reference_date"):
164         limit = limits.loc[limits.reference_date.eq(str(date))].iloc[0]
165         model.addConstr(cp.quicksum(decision[index] for index in block.index
if candidates.loc[index, "broadcast"] == 3) <= int(limit.
high_broadcast_capacity), name=f"broadcast_{date}")
166         model.addConstr(cp.quicksum(decision[index] for index in block.index
if candidates.loc[index, "security"] >= 3) <= int(limit.
high_security_capacity), name=f"security_{date}")
167         model.addConstr(cp.quicksum(decision[index] for index in block.index

```

```

    if candidates.loc[index, "transport"] == 3) <= int(limit.
enhanced_transport_capacity), name=f"transport_{date}")
168     model.addConstr(cp.quicksum(float(candidates.loc[index, "
dynamic_resource_cost"]) * decision[index] for index in block.index) <=
float(limit.daily_resource_budget_index), name=f"budget_{date}")
169     objective = cp.quicksum(
170         float(candidates.loc[index, score_column]) * decision[index]
171         for index in candidates.index
172     )
173     model.setObjective(objective, cp.COPT.MAXIMIZE if maximize else cp.COPT.
MINIMIZE)
174     model.solve()
175     if not getattr(model, "hasmipsol", False) and not getattr(model, "
hasmipSol", False):
176         raise RuntimeError(f"COPT returned no solution for {model_name};
status={model.status}")
177     chosen = candidates.loc[[index for index, variable in decision.items() if
variable.x > 0.5]].copy()
178     if len(chosen) != 24:
179         raise RuntimeError(f"{model_name} selected {len(chosen)} rather than
24 configurations")
180     evidence = {
181         "rows": int(model.getAttr(cp.COPT.Attr.Rows)),
182         "columns": int(model.getAttr(cp.COPT.Attr.Cols)),
183         "binary_variables": int(model.getAttr(cp.COPT.Attr.Bins)),
184         "best_bound": float(model.getAttr(cp.COPT.Attr.BestObj)),
185         "relative_gap": float(model.getAttr(cp.COPT.Attr.BestGap)),
186         "solve_time_seconds": float(model.getAttr(cp.COPT.Attr.SolvingTime)),
187     }
188     return chosen.sort_values("match_id").reset_index(drop=True), int(model.
status), float(model.objval), evidence
189
190
191 def solve_copt_q3(
192     tables: Q1Tables,
193     q2_csv: Path,
194     q1_csv: Path,
195     n_simulations: int = 20_000,
196     seed: int = 20260814,
197     log_dir: Path | None = None,
198 ) -> CoptQ3Artifacts:
199     """Optimize static and dynamic plans, then evaluate both under the updated
environment."""
200
201     state = build_q3_state(tables, q2_csv, q1_csv, n_simulations=n_simulations

```

```

, seed=seed)
202 candidates, _ = build_q3_candidates(tables, state.matches)
203 limits = _table(tables, "dynamic_resource_limits").copy()
204 limits["reference_date"] = limits.reference_date.astype(str)
205 static_log = log_dir / "static_solver.log" if log_dir is not None else
None
206 dynamic_log = log_dir / "dynamic_solver.log" if log_dir is not None else
None
207 lower_bound_log = log_dir / "lower_bound_solver.log" if log_dir is not
None else None
208 static, static_status, static_objective, static_evidence =
_solve_selection(
209     candidates, limits, "static_optimization_score", "
q3_static_resource_mip", static_log
210 )
211 dynamic, dynamic_status, dynamic_objective, dynamic_evidence =
_solve_selection(
212     candidates, limits, "updated_evaluation_score", "
q3_dynamic_resource_mip", dynamic_log
213 )
214 lower_bound, lower_bound_status, lower_bound_objective,
lower_bound_evidence = _solve_selection(
215     candidates,
216     limits,
217     "updated_evaluation_score",
218     "q3_feasible_objective_lower_bound_mip",
219     lower_bound_log,
220     maximize=False,
221 )
222 if log_dir is not None:
223     combined = (
224         "=== Q3 STATIC COPT MODEL ===\n"
225         + static_log.read_text(encoding="utf-8", errors="replace")
226         + "\n=== Q3 DYNAMIC COPT MODEL ===\n"
227         + dynamic_log.read_text(encoding="utf-8", errors="replace")
228         + "\n=== Q3 FEASIBLE OBJECTIVE LOWER-BOUND COPT MODEL ===\n"
229         + lower_bound_log.read_text(encoding="utf-8", errors="replace")
230     )
231     (log_dir / "solver.log").write_text(combined, encoding="utf-8")
232
233 static_eval = static[["match_id", "updated_evaluation_score"]].rename(
columns={"updated_evaluation_score": "static_net_value"})
234 dynamic_output = dynamic.rename(columns={
235     "broadcast": "recommended_broadcast_priority",
236     "security": "recommended_security_level",

```

```

237     "transport": "recommended_transport_level",
238     "ticket_adjustment": "recommended_ticket_adjustment",
239     "dynamic_attendance": "updated_expected_attendance",
240     "dynamic_tv_viewers": "updated_expected_tv_viewers",
241     "dynamic_ticket_value": "updated_ticket_revenue_usd",
242     "dynamic_broadcast_value": "updated_broadcast_value_usd",
243     "dynamic_resource_cost": "resource_cost_index",
244     "dynamic_risk": "risk_exposure_index",
245     "updated_evaluation_score": "dynamic_net_value",
246 })
247 state_columns = [
248     "match_id", "group_id", "team_a", "team_b",
249     "updated_p_team_a_advance", "updated_p_team_b_advance",
250     "stakeless_risk", "collusion_risk", "updated_attractiveness",
251 ]
252 result = state.matches[state_columns].merge(dynamic_output[[
253     "match_id", "recommended_broadcast_priority", "
recommended_security_level",
254     "recommended_transport_level", "recommended_ticket_adjustment",
255     "updated_expected_attendance", "updated_expected_tv_viewers",
256     "updated_ticket_revenue_usd", "updated_broadcast_value_usd",
257     "resource_cost_index", "risk_exposure_index", "dynamic_net_value",
258 ]], on="match_id").merge(static_eval, on="match_id")
259 result["improvement_rate"] = (result.dynamic_net_value - result.
static_net_value) / result.static_net_value.abs().replace(0, np.nan)
260 result = result[OUTPUT_COLUMNS].sort_values("match_id").reset_index(drop=
True)
261 return CoptQ3Artifacts(
262     result=result,
263     simulation_summary=state.simulation_summary,
264     static_selection=static,
265     dynamic_selection=dynamic,
266     lower_bound_selection=lower_bound,
267     candidate_count=len(candidates),
268     static_status=static_status,
269     dynamic_status=dynamic_status,
270     lower_bound_status=lower_bound_status,
271     static_optimization_objective=static_objective,
272     static_updated_evaluation=float(static.updated_evaluation_score.sum())
,
273     dynamic_optimization_objective=dynamic_objective,
274     feasible_objective_lower_bound=lower_bound_objective,
275     solve_evidence={
276         "static": static_evidence,
277         "dynamic": dynamic_evidence,

```

```

278         "feasible_lower_bound": lower_bound_evidence,
279     },
280 )

```

src/c_contest_q1/q4_demo.py

```

1  """Q4 sourced actual-schedule and scale-neutral structural comparison."""
2
3  from __future__ import annotations
4
5  import math
6  from pathlib import Path
7  from zoneinfo import ZoneInfo
8
9  import numpy as np
10 import pandas as pd
11
12 from .data import Q1Tables
13
14
15 ACTUAL_REQUIRED = [
16     "match_id", "competition", "stage", "group_id", "round_in_group", "team_a",
17     "team_b",
18     "venue", "city", "country", "date", "local_kickoff_time", "utc_datetime",
19     "goals_a",
20     "goals_b", "source_url", "retrieval_date",
21 ]
22
23 ACTUAL_STADIUMS_COLUMNS = ["city", "venue", "country", "utc_offset", "capacity",
24                             "latitude", "longitude"]
25
26 def validate_actual_schedule(actual: pd.DataFrame) -> tuple[str, ...]:
27     errors: list[str] = []
28     if list(actual.columns) != ACTUAL_REQUIRED:
29         return ("actual schedule columns do not follow the official template",)
30     if actual.empty:
31         errors.append("actual schedule has no rows")
32     if not actual.match_id.is_unique:
33         errors.append("actual schedule match_id must be unique")
34     if actual[["source_url", "retrieval_date"]].isna().any().any() or actual[["source_url", "retrieval_date"]].astype(str).apply(lambda column: column.str.strip().eq("")).any().any():

```

```

34     errors.append("every actual schedule row requires source_url and
retrieval_date")
35     if pd.to_datetime(actual.utc_datetime, utc=True, errors="coerce").isna().
any():
36         errors.append("actual schedule has unparsable UTC datetimes")
37     return tuple(errors)
38
39
40 def _haversine_km(lat1: float, lon1: float, lat2: float, lon2: float) -> float
:
41     radius = 6371.0088
42     phi1, phi2 = math.radians(lat1), math.radians(lat2)
43     dphi, dlamba = math.radians(lat2 - lat1), math.radians(lon2 - lon1)
44     a = math.sin(dphi / 2) ** 2 + math.cos(phi1) * math.cos(phi2) * math.sin(
dlamba / 2) ** 2
45     return 2 * radius * math.asin(math.sqrt(a))
46
47
48 def _team_transitions(frame: pd.DataFrame, venue_column: str, coordinates:
dict[str, tuple[float, float]], timezone_offsets: dict[str, float]) -> pd.
DataFrame:
49     utc = pd.to_datetime(frame.utc_datetime, utc=True)
50     rows: list[dict[str, float | str]] = []
51     for team in sorted(set(frame.team_a).union(frame.team_b)):
52         games = frame.loc[frame.team_a.eq(team) | frame.team_b.eq(team)].copy
()
53         games["utc_value"] = utc.loc[games.index]
54         games = games.sort_values("utc_value")
55         previous = None
56         for game in games.itertuples(index=False):
57             if previous is not None:
58                 previous_venue, previous_time = getattr(previous, venue_column
), previous.utc_value
59                 venue = getattr(game, venue_column)
60                 lat1, lon1 = coordinates[previous_venue]
61                 lat2, lon2 = coordinates[venue]
62                 rows.append({
63                     "team": team,
64                     "from_venue": previous_venue,
65                     "to_venue": venue,
66                     "distance_km": _haversine_km(lat1, lon1, lat2, lon2),
67                     "rest_hours": (game.utc_value - previous_time).
total_seconds() / 3600,
68                     "timezone_crossing": float(not np.isclose(timezone_offsets
[previous_venue], timezone_offsets[venue])),

```

```

69         })
70         previous = game
71     return pd.DataFrame(rows)
72
73
74 def _venue_metrics(frame: pd.DataFrame, venue_column: str) -> dict[str, float
75 ]:
76     shares = frame[venue_column].value_counts(normalize=True)
77     counts = frame[venue_column].value_counts()
78     return {
79         "unique_venues": float(len(counts)),
80         "matches_per_venue_mean": float(counts.mean()),
81         "matches_per_venue_cv": float(counts.std(ddof=0) / counts.mean()),
82         "venue_match_hhi": float((shares ** 2).sum()),
83     }
84
85 def _metrics_actual(actual: pd.DataFrame, stadiums: pd.DataFrame) -> dict[str,
86 float]:
87     coordinates = {row.venue: (row.latitude, row.longitude) for row in
88 stadiums.itertuples(index=False)}
89     offsets = {row.venue: row.utc_offset for row in stadiums.itertuples(index=
90 False)}
91     capacities = {row.venue: row.capacity for row in stadiums.itertuples(index
92 =False)}
93     transitions = _team_transitions(actual, "venue", coordinates, offsets)
94     metrics = _venue_metrics(actual, "venue")
95     metrics.update({
96         "matches": float(len(actual)),
97         "teams": float(pd.concat([actual.team_a, actual.team_b]).nunique()),
98         "travel_km_per_team": float(transitions.distance_km.sum() / pd.concat
99 ([actual.team_a, actual.team_b]).nunique()),
100         "mean_transition_km": float(transitions.distance_km.mean()),
101         "mean_rest_hours": float(transitions.rest_hours.mean()),
102         "minimum_rest_hours": float(transitions.rest_hours.min()),
103         "rest_under_60h_rate": float(transitions.rest_hours.lt(60).mean()),
104         "timezone_crossings_per_team": float(transitions.timezone_crossing.sum
105 () / pd.concat([actual.team_a, actual.team_b]).nunique()),
106         "mean_stadium_capacity": float(actual.venue.map(capacities).mean()),
107     })
108     return metrics
109
110 def _metrics_optimized(tables: Q1Tables, optimized: pd.DataFrame) -> dict[str,
111 float]:

```



```

106     venues = pd.read_excel(tables.paths.workbook, sheet_name="venues").
        set_index("venue_id")
107     coordinates = {venue: (float(row.latitude), float(row.longitude)) for
        venue, row in venues.iterrows()}
108     offsets: dict[str, float] = {}
109     reference_time = pd.Timestamp("2026-06-20T12:00:00Z")
110     for venue, row in venues.iterrows():
111         offsets[venue] = reference_time.tz_convert(ZoneInfo(row.timezone)).
        utcoffset().total_seconds() / 3600
112     transitions = _team_transitions(optimized, "venue_id", coordinates,
        offsets)
113     metrics = _venue_metrics(optimized, "venue_id")
114     team_count = pd.concat([optimized.team_a, optimized.team_b]).nunique()
115     metrics.update({
116         "matches": float(len(optimized)),
117         "teams": float(team_count),
118         "travel_km_per_team": float(transitions.distance_km.sum() / team_count
        ),
119         "mean_transition_km": float(transitions.distance_km.mean()),
120         "mean_rest_hours": float(transitions.rest_hours.mean()),
121         "minimum_rest_hours": float(transitions.rest_hours.min()),
122         "rest_under_60h_rate": float(transitions.rest_hours.lt(60).mean()),
123         "timezone_crossings_per_team": float(transitions.timezone_crossing.sum
        () / team_count),
124         "mean_stadium_capacity": float(optimized.venue_id.map(venues.capacity)
        .mean()),
125     })
126     return metrics
127
128
129 def compare_structural_schedule(tables: Q1Tables, actual_csv: Path,
        optimized_csv: Path) -> pd.DataFrame:
130     actual = pd.read_csv(actual_csv)
131     errors = validate_actual_schedule(actual)
132     if errors:
133         raise ValueError("; ".join(errors))
134     stadiums_path = Path(actual_csv).parent / "actual_stadiums.csv"
135     if not stadiums_path.exists():
136         raise FileNotFoundError(f"actual stadiums table missing: {
        stadiums_path}")
137     stadiums = pd.read_csv(stadiums_path)
138     if list(stadiums.columns) != ACTUAL_STADIUMS_COLUMNS:
139         raise ValueError("actual stadiums columns do not follow the expected
        template")
140     optimized = pd.read_csv(optimized_csv)

```

```

141     actual_metrics, optimized_metrics = _metrics_actual(actual, stadiums),
    _metrics_optimized(tables, optimized)
142     definitions = [
143         ("matches", "scale", "target"),
144         ("teams", "scale", "target"),
145         ("travel_km_per_team", "travel", "lower"),
146         ("mean_transition_km", "travel", "lower"),
147         ("mean_rest_hours", "rest", "higher"),
148         ("minimum_rest_hours", "rest", "higher"),
149         ("rest_under_60h_rate", "rest", "lower"),
150         ("timezone_crossings_per_team", "travel", "lower"),
151         ("unique_venues", "venue_utilization", "target"),
152         ("matches_per_venue_mean", "venue_utilization", "target"),
153         ("matches_per_venue_cv", "venue_utilization", "lower"),
154         ("venue_match_hhi", "venue_utilization", "lower"),
155         ("mean_stadium_capacity", "capacity", "higher"),
156     ]
157     output: list[dict[str, object]] = []
158     for name, category, direction in definitions:
159         observed, proposed = actual_metrics[name], optimized_metrics[name]
160         difference = proposed - observed
161         if direction == "higher":
162             relative = difference / abs(observed) if observed else np.nan
163             result = "improved" if difference > 0 else "not_improved"
164         elif direction == "lower":
165             relative = -difference / abs(observed) if observed else np.nan
166             result = "improved" if difference < 0 else "not_improved"
167         else:
168             relative = np.nan
169             result = "context_only"
170         output.append({
171             "indicator_name": name,
172             "indicator_category": category,
173             "actual_schedule_value": observed,
174             "optimized_schedule_value": proposed,
175             "absolute_difference": difference,
176             "relative_improvement": relative,
177             "preferred_direction": direction,
178             "evaluation_result": result,
179         })
180     return pd.DataFrame(output)

```

运行、分析、制图与提交脚本

scripts/run_all.ps1

```
1 [CmdletBinding()]
2 param(
3     [switch]$SkipAnalysis,
4     [switch]$SkipPdf
5 )
6
7 $ErrorActionPreference = "Stop"
8 $projectRoot = Split-Path -Parent $PSScriptRoot
9 $demoPython = Join-Path $projectRoot ".venv-demo\Scripts\python.exe"
10 $coptPython = Join-Path $projectRoot ".venv\Scripts\python.exe"
11
12 foreach ($executable in @($demoPython, $coptPython)) {
13     if (-not (Test-Path -LiteralPath $executable)) {
14         throw "Required Python environment not found: $executable"
15     }
16 }
17
18 function Invoke-Step {
19     param(
20         [Parameter(Mandatory = $true)][string]$Name,
21         [Parameter(Mandatory = $true)][string]$Executable,
22         [Parameter(Mandatory = $true)][string[]]$Arguments
23     )
24     Write-Host "`n==> $Name" -ForegroundColor Cyan
25     & $Executable @Arguments
26     if ($LASTEXITCODE -ne 0) {
27         throw "$Name failed with exit code $LASTEXITCODE"
28     }
29 }
30
31 Push-Location $projectRoot
32 try {
33     Invoke-Step "Q1 prediction" $demoPython @("scripts/run_q1_demo.py")
34     Invoke-Step "Q1 seven-model comparison" $demoPython @("scripts/
run_q1_sklearn_challengers.py")
35     Invoke-Step "Q1 report" $demoPython @("scripts/build_q1_demo_report.py")
36     Invoke-Step "Q2 feasible baseline" $demoPython @("scripts/run_q2_demo.py")
37     Invoke-Step "Q2 COPT optimization" $coptPython @("scripts/run_q2_copt.py")
38     Invoke-Step "Q3 COPT optimization" $coptPython @("scripts/run_q3_copt.py")
39     Invoke-Step "Q4 source parsing" $demoPython @("scripts/
collect_q4_openfootball.py")
40     Invoke-Step "Q4 structural comparison" $demoPython @("scripts/
```

```

run_q4_comparison.py")
41
42     if (-not $SkipAnalysis) {
43         Invoke-Step "Robustness and convergence analysis" $demoPython @"
scripts/run_analysis_campaign.py")
44     }
45
46     Invoke-Step "Paper figures" $demoPython @"scripts/build_paper_figures.py
")
47     Invoke-Step "Python tests" $demoPython @("-m", "pytest", "tests", "-q")
48     Invoke-Step "Deliverable validation" $demoPython @"scripts/
validate_deliverables.py")
49
50     if (-not $SkipPdf) {
51         $env:PYTHONIOENCODING = "utf-8"
52         $env:PERL_UNICODE = "SDA"
53         Invoke-Step "Main paper PDF" "latexmk" @("-cd", "-g", "-xelatex", "-
interaction=nonstopmode", "-halt-on-error", "-outdir=build", "paper/main.
tex")
54         Invoke-Step "AI usage PDF" "latexmk" @("-cd", "-g", "-xelatex", "-
interaction=nonstopmode", "-halt-on-error", "-outdir=build", "paper/
ai_usage.tex")
55     }
56 }
57 finally {
58     Pop-Location
59 }
60
61 Write-Host "`nAll requested reproduction steps completed." -ForegroundColor
Green

```

scripts/run_q1_demo.py

```

1  """Run the first complete Q1 Demo evaluation and persist reproducible evidence
   ."""
2
3  from __future__ import annotations
4
5  import json
6  import sys
7  from pathlib import Path
8
9  PROJECT_ROOT = Path(__file__).resolve().parents[1]
10 SRC_ROOT = PROJECT_ROOT / "src"
11 if str(SRC_ROOT) not in sys.path:

```

```

12     sys.path.insert(0, str(SRC_ROOT))
13
14 from c_contest_q1.data import load_q1_tables, validate_q1_tables
15 from c_contest_q1.evaluation import evaluate_model_specs
16 from c_contest_q1.features import build_historical_features,
    target_in_millions
17 from c_contest_q1.models import demo_model_specs
18 from c_contest_q1.outputs import build_demo_outputs, validate_demo_outputs
19
20
21 def main() -> None:
22     output_dir = Path("outputs/q1/demo")
23     output_dir.mkdir(parents=True, exist_ok=True)
24     tables = load_q1_tables()
25     report = validate_q1_tables(tables)
26     if not report.ok:
27         raise RuntimeError("Input validation failed: " + "; ".join(report.
errors))
28     frame = build_historical_features(tables)
29     result = evaluate_model_specs(
30         frame.features,
31         frame.metadata,
32         target_in_millions(frame),
33         demo_model_specs(),
34     )
35     result.leaderboard.to_csv(output_dir / "leaderboard.csv", index=False)
36     result.fold_metrics.to_csv(output_dir / "fold_metrics.csv", index=False)
37     result.oof_predictions.to_csv(output_dir / "oof_predictions.csv", index=
False)
38     artifacts = build_demo_outputs(tables, output_dir)
39     output_report = validate_demo_outputs(tables, artifacts.test_csv,
artifacts.match_csv)
40     if not output_report.ok:
41         raise RuntimeError("Output validation failed: " + "; ".join(
output_report.errors))
42     (output_dir / "feature_manifest.json").write_text(
43         json.dumps(
44             {
45                 "features": list(frame.features.columns),
46                 "target_unit": "millions_of_people",
47                 "n_training_rows": int(target_in_millions(frame).shape[0]),
48                 "n_hidden_test_rows": int(frame.metadata["dataset_split"].eq("
test").sum()),
49             },
50             ensure_ascii=False,

```

```

51         indent=2,
52     ),
53     encoding="utf-8",
54 )
55 print(result.leaderboard.to_string(index=False))
56
57
58 if __name__ == "__main__":
59     main()

```

scripts/run_q1_sklearn_challengers.py

```

1  """Compare sklearn Q1 challengers under the immutable Demo evaluation contract
   ."""
2
3  from __future__ import annotations
4
5  import sys
6  from pathlib import Path
7
8  PROJECT_ROOT = Path(__file__).resolve().parents[1]
9  SRC_ROOT = PROJECT_ROOT / "src"
10 if str(SRC_ROOT) not in sys.path:
11     sys.path.insert(0, str(SRC_ROOT))
12
13 import pandas as pd
14
15 from c_contest_q1.data import load_q1_tables
16 from c_contest_q1.evaluation import evaluate_model_specs
17 from c_contest_q1.features import build_historical_features,
   target_in_millions
18 from c_contest_q1.models import demo_model_specs, sklearn_challenger_specs
19
20
21 def main() -> None:
22     output_dir = PROJECT_ROOT / "outputs" / "q1" / "sklearn_challengers"
23     output_dir.mkdir(parents=True, exist_ok=True)
24     frame = build_historical_features(load_q1_tables())
25     result = evaluate_model_specs(
26         frame.features,
27         frame.metadata,
28         target_in_millions(frame),
29         demo_model_specs() + sklearn_challenger_specs(),
30     )
31     result.leaderboard.to_csv(output_dir / "leaderboard.csv", index=False)

```

```

32     result.fold_metrics.to_csv(output_dir / "fold_metrics.csv", index=False)
33     result.oof_predictions.to_csv(output_dir / "oof_predictions.csv", index=
False)
34     print(result.leaderboard.to_string(index=False))
35
36
37 if __name__ == "__main__":
38     main()

```

scripts/build_q1_demo_report.py

```

1  """Build a compact evidence report and plots for the first Q1 Demo."""
2
3  from __future__ import annotations
4
5  import hashlib
6  import json
7  import sys
8  from pathlib import Path
9
10 PROJECT_ROOT = Path(__file__).resolve().parents[1]
11 SRC_ROOT = PROJECT_ROOT / "src"
12 if str(SRC_ROOT) not in sys.path:
13     sys.path.insert(0, str(SRC_ROOT))
14
15 import matplotlib.pyplot as plt
16 import pandas as pd
17 import seaborn as sns
18
19 from c_contest_q1.data import load_q1_tables
20 from c_contest_q1.outputs import validate_demo_outputs
21
22
23 def _sha256(path: Path) -> str:
24     digest = hashlib.sha256()
25     with path.open("rb") as source:
26         for chunk in iter(lambda: source.read(1024 * 1024), b''):
27             digest.update(chunk)
28     return digest.hexdigest()
29
30
31 def _markdown_table(frame: pd.DataFrame) -> str:
32     """Render a compact Markdown table without pandas' optional tabulate
dependency."""
33

```

```

34     display = frame.copy()
35     for column in display.select_dtypes(include="number"):
36         display[column] = display[column].map(lambda value: f"{value:.4f}")
37     headers = [str(column) for column in display.columns]
38     rows = [[str(value) for value in row] for row in display.itertuples(index=
False, name=None)]
39     return "\n".join(
40         [
41             "| " + " | ".join(headers) + " |",
42             "| " + " | ".join(["---"] * len(headers)) + " |",
43             *["| " + " | ".join(row) + " |" for row in rows],
44         ]
45     )
46
47
48 def main() -> None:
49     output_dir = PROJECT_ROOT / "outputs" / "q1" / "demo"
50     figures_dir = output_dir / "figures"
51     figures_dir.mkdir(parents=True, exist_ok=True)
52     leaderboard = pd.read_csv(output_dir / "leaderboard.csv")
53     fold_metrics = pd.read_csv(output_dir / "fold_metrics.csv")
54     oof = pd.read_csv(output_dir / "oof_predictions.csv", parse_dates=["date"
])
55     ridge = oof.loc[oof["model"].eq("ridge")].copy()
56
57     sns.set_theme(style="whitegrid", context="notebook")
58     fig, axes = plt.subplots(1, 2, figsize=(13, 5))
59     sns.barplot(data=leaderboard, x="model", y="mse", hue="model", legend=
False, ax=axes[0])
60     axes[0].set_title("Temporal validation MSE")
61     axes[0].set_xlabel("")
62     axes[0].set_ylabel("MSE (millions squared)")
63     axes[0].tick_params(axis="x", rotation=18)
64     sns.lineplot(data=fold_metrics, x="fold", y="mse", hue="model", marker="o"
, ax=axes[1])
65     axes[1].set_title("MSE by expanding-window fold")
66     axes[1].set_xlabel("Validation period")
67     axes[1].set_ylabel("MSE (millions squared)")
68     fig.tight_layout()
69     fig.savefig(figures_dir / "model_comparison.png", dpi=180)
70     plt.close(fig)
71
72     fig, axes = plt.subplots(1, 2, figsize=(13, 5))
73     sns.scatterplot(data=ridge, x="actual", y="prediction", hue="fold", ax=
axes[0])

```



```

74     limits = [min(ridge[["actual", "prediction"]].min()), max(ridge[["actual",
75         "prediction"]].max())]
76     axes[0].plot(limits, limits, color="black", linestyle="--", linewidth=1)
77     axes[0].set_title("Ridge: actual versus prediction")
78     axes[0].set_xlabel("Actual viewers (millions)")
79     axes[0].set_ylabel("Predicted viewers (millions)")
80     sns.scatterplot(data=ridge, x="date", y="residual", hue="fold", ax=axes
81         [1])
82     axes[1].axhline(0, color="black", linestyle="--", linewidth=1)
83     axes[1].set_title("Ridge residuals over time")
84     axes[1].set_xlabel("Match date")
85     axes[1].set_ylabel("Prediction minus actual (millions)")
86     fig.tight_layout()
87     fig.savefig(figures_dir / "ridge_diagnostics.png", dpi=180)
88     plt.close(fig)
89
90     tables = load_q1_tables()
91     test_csv = output_dir / "result_1_test_prediction.csv"
92     match_csv = output_dir / "result_1_match_prediction.csv"
93     validation = validate_demo_outputs(tables, test_csv, match_csv)
94     environment = {
95         "python": sys.version,
96         "pandas": pd.__version__,
97         "matplotlib": plt.matplotlib.__version__,
98         "seaborn": sns.__version__,
99     }
100     (output_dir / "environment.json").write_text(
101         json.dumps(environment, ensure_ascii=False, indent=2), encoding="utf-8"
102     )
103     artifact_hashes = {path.name: _sha256(path) for path in (test_csv,
104         match_csv)}
105     (output_dir / "artifact_hashes.json").write_text(
106         json.dumps(artifact_hashes, ensure_ascii=False, indent=2), encoding="
107         utf-8"
108     )
109     test_prediction = pd.read_csv(test_csv)
110     match_prediction = pd.read_csv(match_csv)
111     report = f"""# Q1 Demo Model Effect Report
112
113     ## Current Demo Decision
114
115     The current frozen Demo model is Ridge regression with fold-fitted one-hot
116         categorical encoding and standardized numeric features. It is selected
117         solely because it has the lowest mean original-scale temporal-validation

```

```

    MSE among the Demo candidates and is better in every validation period.
112
113 ## Common Evaluation Contract
114
115 - Training labels: 560 rows, converted from people to millions of people.
116 - Hidden official test: 140 rows, never used for model selection.
117 - Validation: three expanding time windows grouped by complete match date.
118 - Metrics: MSE is primary; RMSE and MAE are auxiliary, all in millions of
    people.
119 - Feature boundary: only shared pre-match information. Current-match results,
    xG, shots, possession, attendance, IDs, and future-only composite
    commercial indices are excluded.
120
121 ## Leaderboard
122
123 {_markdown_table(leaderboard)}
124
125 ## Fold Evidence
126
127 {_markdown_table(fold_metrics)}
128
129 Ridge improves mean MSE by {(1 - leaderboard.loc[leaderboard['model'].eq('
    ridge'), 'mse'].iloc[0] / leaderboard.loc[leaderboard['model'].eq('mean'),
    'mse'].iloc[0]) * 100:.1f}% versus the mean baseline. Its newest-fold MSE
    is 183.77, below both other Demo candidates.
130
131 ## Outputs
132
133 - `result_1_test_prediction.csv`: {len(test_prediction)} rows, values in
    millions of viewers.
134 - `result_1_match_prediction.csv`: {len(match_prediction)} rows, values in
    people.
135 - Independent reload validation: {'PASS' if validation.ok else 'FAIL: ' + ';
    '.join(validation.errors)}.
136 - SHA-256 values: see `artifact_hashes.json`.
137
138 ## Figures
139
140 - `figures/model_comparison.png`: candidate leaderboard and fold-level MSE.
141 - `figures/ridge_diagnostics.png`: actual-predicted relation and temporal
    residuals.
142
143 ## Interpretation And Limitations
144
145 This is a valid end-to-end Demo, not a final optimality claim. Ridge

```

```

    outperforms the initial constrained histogram gradient boosting model,
    indicating that the current small, common feature set has a largely stable
    linear signal or that the tree model is under-tuned. The model has not yet
    compared CatBoost, LightGBM, XGBoost, Extra Trees, SVR, target
    transformation variants, or a tightly controlled two-model blend. The
    future 72-match setting is a World Cup group-stage domain shift while the
    historical data contains only a limited number of World Cup matches;
    therefore later iterations must report subgroup performance and cannot
    infer hidden-test superiority from this Demo alone.
146
147 ## Next Upgrade Question
148
149 The next high-stakes reasoning step is whether to extend the feature contract
    with strictly lagged historical-form variables and how to compare boosted-
    tree models under the World Cup domain shift. Per the project rule,
    switching to `sol/ultra` for that route decision requires explicit user
    approval first.
150 """
151     (output_dir / "model_effect_report.md").write_text(report, encoding="utf-8")
152     print(output_dir / "model_effect_report.md")
153
154
155 if __name__ == "__main__":
156     main()

```

scripts/run_q2_demo.py

```

1 from __future__ import annotations
2
3 import sys
4 from pathlib import Path
5
6 ROOT = Path(__file__).resolve().parents[1]
7 sys.path.insert(0, str(ROOT / "src"))
8
9 from c_contest_q1.data import load_q1_tables
10 from c_contest_q1.paths import discover_template_path
11 from c_contest_q1.q2_demo import build_q2_demo_schedule, evaluate_q2_schedule,
    validate_q2_schedule
12
13
14 def main() -> None:
15     tables = load_q1_tables()
16     q1 = ROOT / "outputs" / "q1" / "demo" / "result_1_match_prediction.csv"

```

```

17 out_dir = ROOT / "outputs" / "q2" / "demo"
18 out_dir.mkdir(parents=True, exist_ok=True)
19 schedule = build_q2_demo_schedule(tables, q1)
20 validation = validate_q2_schedule(tables, schedule)
21 if not validation.ok:
22     raise RuntimeError("Q2 feasibility failed: " + "; ".join(validation.
23 errors))
24 schedule, score = evaluate_q2_schedule(tables, schedule)
25 template = discover_template_path("result_2_template.csv")
26 columns = __import__('pandas').read_csv(template).columns.tolist()
27 schedule.loc[:, columns].to_csv(out_dir / "result_2_group_schedule.csv",
28 index=False)
29 (out_dir / "feasibility_report.txt").write_text(
30     "PASS: all six hard-constraint groups recomputed from CSV.\n"
31     "P2 score uses fixed eligible-candidate min-max ranges for ticket,
32 broadcast, operation cost, and travel.\n",
33     encoding="utf-8",
34 )
35 __import__('json').dump(score, (out_dir / "objective_score.json").open("w"
36 , encoding="utf-8"), ensure_ascii=False, indent=2)
37 print(schedule.loc[:, columns].head().to_string(index=False))
38
39 if __name__ == "__main__":
40     main()

```

scripts/run_q2_copt.py

```

1 from __future__ import annotations
2
3 import json
4 import sys
5 from pathlib import Path
6
7 ROOT = Path(__file__).resolve().parents[1]
8 sys.path.insert(0, str(ROOT / "src"))
9
10 from c_contest_q1.data import load_q1_tables
11 from c_contest_q1.q2_copt import solve_copt_q2
12 from c_contest_q1.q2_demo import validate_q2_schedule
13
14
15 def main() -> None:
16     out = ROOT / "outputs" / "q2" / "copt"
17     out.mkdir(parents=True, exist_ok=True)

```

```

18     artifacts = solve_copt_q2(
19         load_q1_tables(),
20         ROOT / "outputs" / "q2" / "demo" / "result_2_group_schedule.csv",
21         log_path=out / "solver.log",
22     )
23     report = validate_q2_schedule(load_q1_tables(), artifacts.schedule)
24     if not report.ok:
25         raise RuntimeError("COPT Q2 feasibility failure: " + "; ".join(report.
26             errors))
27     artifacts.schedule.to_csv(out / "result_2_group_schedule.csv", index=False
28 )
29     (out / "solve_metadata.json").write_text(json.dumps({"solver": "COPT 8.0.6
30 ", "license_mode": "non-commercial evaluation, 2000-variable/constraint
31 limit", "candidate_count": artifacts.candidate_count, "status": artifacts.
32 status, "mip_objective": artifacts.mip_objective, "solve_evidence":
33 artifacts.solve_evidence, "full_p2_score": artifacts.score_report},
34     ensure_ascii=False, indent=2), encoding="utf-8")
35     print(artifacts.schedule.head().to_string(index=False))
36     print({"candidate_count": artifacts.candidate_count, "status": artifacts.
37 status, "mip_objective": artifacts.mip_objective, "full_p2_objective":
38 artifacts.score_report["objective"]})
39
40
41
42 if __name__ == "__main__":
43     main()

```

scripts/run_q3_demo.py

```

1  from __future__ import annotations
2  import sys
3  from pathlib import Path
4  ROOT = Path(__file__).resolve().parents[1]
5  sys.path.insert(0, str(ROOT / "src"))
6  from c_contest_q1.data import load_q1_tables
7  from c_contest_q1.q3_demo import build_q3_state
8
9  def main() -> None:
10     out = ROOT / "outputs" / "q3" / "demo"; out.mkdir(parents=True, exist_ok=
11         True)
12     artifacts = build_q3_state(load_q1_tables(), ROOT / "outputs" / "q2" / "
13         copt" / "result_2_group_schedule.csv", ROOT / "outputs" / "q1" / "demo" / "
14         result_1_match_prediction.csv")
15     artifacts.matches.to_csv(out / "q3_state_inputs.csv", index=False)
16     artifacts.simulation_summary.to_csv(out / "simulation_summary.csv", index=
17         False)

```

```

14     print(artifacts.simulation_summary.head().to_string(index=False))
15
16 if __name__ == "__main__": main()

```

scripts/run_q3_copt.py

```

1  from __future__ import annotations
2
3  import json
4  import sys
5  from pathlib import Path
6
7  ROOT = Path(__file__).resolve().parents[1]
8  sys.path.insert(0, str(ROOT / "src"))
9
10 from c_contest_q1.data import load_q1_tables
11 from c_contest_q1.q3_copt import solve_copt_q3
12 from c_contest_q1.q3_demo import validate_q3_result
13
14
15 def main() -> None:
16     out = ROOT / "outputs" / "q3" / "copt"
17     out.mkdir(parents=True, exist_ok=True)
18     artifacts = solve_copt_q3(
19         load_q1_tables(),
20         ROOT / "outputs" / "q2" / "copt" / "result_2_group_schedule.csv",
21         ROOT / "outputs" / "q1" / "demo" / "result_1_match_prediction.csv",
22         log_dir=out,
23     )
24     validation = validate_q3_result(load_q1_tables(), ROOT / "outputs" / "q2"
25 / "copt" / "result_2_group_schedule.csv", artifacts.result)
26     if not validation.ok:
27         raise RuntimeError("Q3 validation failed: " + "; ".join(validation.
28 errors))
29     artifacts.result.to_csv(out / "result_3_dynamic_strategy.csv", index=False
30 )
31     artifacts.simulation_summary.to_csv(out / "simulation_summary.csv", index=
32 False)
33     artifacts.static_selection.to_csv(out / "static_selection.csv", index=
34 False)
35     artifacts.dynamic_selection.to_csv(out / "dynamic_selection.csv", index=
36 False)
37     artifacts.lower_bound_selection.to_csv(out / "
38 feasible_lower_bound_selection.csv", index=False)
39     metadata = {

```

```

33     "candidate_count": artifacts.candidate_count,
34     "static_status": artifacts.static_status,
35     "dynamic_status": artifacts.dynamic_status,
36     "lower_bound_status": artifacts.lower_bound_status,
37     "static_optimization_objective": artifacts.
static_optimization_objective,
38     "static_updated_evaluation": artifacts.static_updated_evaluation,
39     "dynamic_optimization_objective": artifacts.
dynamic_optimization_objective,
40     "feasible_objective_lower_bound": artifacts.
feasible_objective_lower_bound,
41     "feasible_objective_upper_bound": artifacts.
dynamic_optimization_objective,
42     "solver": "COPT 8.0.6",
43     "license_mode": "non-commercial evaluation, 2000-variable/constraint
limit",
44     "solve_evidence": artifacts.solve_evidence,
45     "total_improvement_rate": (
46         artifacts.dynamic_optimization_objective - artifacts.
static_updated_evaluation
47     ) / abs(artifacts.static_updated_evaluation),
48 }
49 (out / "solve_metadata.json").write_text(json.dumps(metadata, indent=2),
encoding="utf-8")
50 print(artifacts.result.head().to_string(index=False))
51 print(metadata)
52
53
54 if __name__ == "__main__":
55     main()

```

scripts/collect_q4_openfootball.py

```

1 from __future__ import annotations
2
3 import hashlib
4 import json
5 import re
6 import sys
7 from datetime import datetime, timezone, timedelta
8 from pathlib import Path
9
10 import pandas as pd
11
12 ROOT = Path(__file__).resolve().parents[1]

```

```

13 sys.path.insert(0, str(ROOT / "src"))
14
15 from c_contest_q1.q4_demo import ACTUAL_REQUIRED, validate_actual_schedule
16
17
18 SOURCE_URL = "https://github.com/openfootball/worldcup/blob/master/2026--
    canada-usa-mexico/cup.txt"
19 STADIUMS_URL = "https://github.com/openfootball/worldcup/blob/master/2026--
    canada-usa-mexico/stadiums.csv"
20 RETRIEVAL_DATE = "2026-08-14"
21 YEAR = 2026
22 COUNTRY_NAMES = {"ca": "Canada", "us": "United States", "mx": "Mexico"}
23 DATE_PATTERN = re.compile(r"^(?:Sun|Mon|Tue|Wed|Thu|Fri|Sat)\s+June\s+(\d
    {1,2})\s*$")
24 MATCH_PATTERN = re.compile(
25     r"^\s*(?:\(\d{1,2}:\d{2}\)\s+UTC([+-]\d+)\s+)?(.*?)\s+(\d+)-(\d+)"
26     r"(?:\s+(\d+-\d+)\s+)?\s+(.*?)\s+@\s+(.*?)\s*$"
27 )
28 COORD_PATTERN = re.compile(
29     r"' '(\d+(?:\.\d+)?)°(?:\((?:\d+)\s+)?(?:\d+(?:\.\d+)?)'")\s*([NSEW])' '
30 )
31
32
33 def _dms_to_decimal(text: str) -> float:
34     match = COORD_PATTERN.fullmatch(text.strip())
35     if not match:
36         raise ValueError(f"unparsed coordinate: {text!r}")
37     degrees, minutes, seconds, hemisphere = match.groups()
38     value = float(degrees) + float(minutes or 0) / 60 + float(seconds or 0) /
39         3600
40     return value if hemisphere in ("N", "E") else -value
41
42 def parse_stadiums(path: Path) -> pd.DataFrame:
43     raw = pd.read_csv(path, comment="#", skipinitialspace=True)
44     stadiums = raw.iloc[:, :5].copy()
45     stadiums.columns = ["city", "timezone", "cc", "venue", "capacity"]
46     stadiums["city"] = stadiums.city.str.strip()
47     stadiums["venue"] = stadiums.venue.str.strip()
48     coordinates = raw["coords"].str.split(n=1, expand=True)
49     coordinates.columns = ["latitude_text", "longitude_text"]
50     stadiums["latitude"] = coordinates.latitude_text.map(_dms_to_decimal)
51     stadiums["longitude"] = coordinates.longitude_text.map(_dms_to_decimal)
52     stadiums["utc_offset"] = stadiums.timezone.str.extract(r"UTC([+-]\d+)",
    expand=False).astype(int)

```



```

53     stadiums["country"] = stadiums.cc.map(COUNTRY_NAMES)
54     columns = ["city", "venue", "country", "utc_offset", "capacity", "latitude", "longitude"]
55     result = stadiums[columns].dropna()
56     if len(result) != 16:
57         raise ValueError(f"expected 16 stadiums, got {len(result)}")
58     return result
59
60
61 def parse_schedule(path: Path, stadiums: pd.DataFrame) -> pd.DataFrame:
62     text = path.read_text(encoding="utf-8-sig")
63     groups: dict[str, list[str]] = {}
64     for line in text.splitlines():
65         roster_match = re.fullmatch(r"Group\s+([A-L])\s*\|\s*(.+)", line.strip())
66         if roster_match:
67             teams = re.split(r"\s{2,}", roster_match.group(2).strip())
68             if len(teams) != 4:
69                 raise ValueError(f"group {roster_match.group(1)} does not list four teams: {teams!r}")
70             groups[roster_match.group(1)] = teams
71     if len(groups) != 12:
72         raise ValueError(f"expected 12 groups, got {len(groups)}")
73     venue_by_city = dict(zip(stadiums.city, stadiums.venue))
74     country_by_city = dict(zip(stadiums.city, stadiums.country))
75     group: str | None = None
76     current_date: datetime | None = None
77     current_time: str | None = None
78     current_offset: int | None = None
79     group_match_count = {key: 0 for key in groups}
80     rows: list[dict[str, object]] = []
81     for raw_line in text.splitlines():
82         line = raw_line.strip()
83         group_match = re.fullmatch(r"\u25aa\s*Group\s+([A-L])", line)
84         if group_match:
85             group = group_match.group(1)
86             current_time = None
87             current_offset = None
88             continue
89         date_match = DATE_PATTERN.fullmatch(line)
90         if date_match and group:
91             current_date = datetime(YEAR, 6, int(date_match.group(1)))
92             current_time = None
93             current_offset = None
94             continue

```

```

95         if not group or current_date is None or "@" not in line:
96             continue
97         match = MATCH_PATTERN.match(raw_line)
98         if not match:
99             raise ValueError(f"unparsed match line: {raw_line!r}")
100         explicit_time, explicit_offset, team_a, goals_a, goals_b, team_b, city
= match.groups()
101         if explicit_time is not None:
102             current_time, current_offset = explicit_time, int(explicit_offset)
103         if current_time is None or current_offset is None:
104             raise ValueError(f"match without explicit or inherited time: {
raw_line!r}")
105         team_a, team_b = team_a.strip(), team_b.strip()
106         if team_a not in groups[group] or team_b not in groups[group]:
107             raise ValueError(f"unexpected teams in group {group}: {team_a}, {
team_b}")
108         if city.strip() not in venue_by_city:
109             raise ValueError(f"city not in stadiums table: {city!r}")
110         group_match_count[group] += 1
111         round_number = (group_match_count[group] - 1) // 2 + 1
112         hour, minute = map(int, current_time.split(":"))
113         local = current_date.replace(
114             hour=hour, minute=minute, tzinfo=timezone(timedelta(hours=
current_offset))
115         )
116         rows.append({
117             "match_id": f"WC26-{group}{group_match_count[group]:02d}",
118             "competition": "2026 FIFA World Cup Canada, Mexico, United States"
,
119             "stage": "Group Stage",
120             "group_id": group,
121             "round_in_group": round_number,
122             "team_a": team_a,
123             "team_b": team_b,
124             "venue": venue_by_city[city.strip()],
125             "city": city.strip(),
126             "country": country_by_city[city.strip()],
127             "date": local.date().isoformat(),
128             "local_kickoff_time": current_time,
129             "utc_datetime": local.astimezone(timezone.utc).isoformat(),
130             "goals_a": int(goals_a),
131             "goals_b": int(goals_b),
132             "source_url": SOURCE_URL,
133             "retrieval_date": RETRIEVAL_DATE,
134         })

```

```

135     result = pd.DataFrame(rows, columns=ACTUAL_REQUIRED)
136     if len(result) != 72 or group_match_count != {key: 6 for key in groups}:
137         raise ValueError(f"expected 72 matches and six per group, got {len(
result)} / {group_match_count}")
138     appearances = pd.concat([result.team_a, result.team_b]).value_counts()
139     if len(appearances) != 48 or not appearances.eq(3).all():
140         raise ValueError("each of the 48 actual teams must appear exactly
three times")
141     errors = validate_actual_schedule(result)
142     if errors:
143         raise ValueError("; ".join(errors))
144     return result
145
146
147 def main() -> None:
148     source = ROOT / "research" / "q4" / "openfootball_2026_cup.txt"
149     stadiums_path = ROOT / "research" / "q4" / "openfootball_2026_stadiums.csv
"
150     out = ROOT / "outputs" / "q4"
151     out.mkdir(parents=True, exist_ok=True)
152     stadiums = parse_stadiums(stadiums_path)
153     schedule = parse_schedule(source, stadiums)
154     schedule.to_csv(out / "actual_schedule.csv", index=False)
155     stadiums.to_csv(out / "actual_stadiums.csv", index=False)
156     hashes = {
157         path.name: hashlib.sha256(path.read_bytes()).hexdigest()
158         for path in (source, stadiums_path)
159     }
160     (out / "source_hashes.json").write_text(json.dumps(hashes, indent=2),
encoding="utf-8")
161     print(schedule.head().to_string(index=False))
162     print({"rows": len(schedule), "groups": schedule.group_id.nunique(), "
teams": pd.concat([schedule.team_a, schedule.team_b]).nunique(), "venues":
schedule.venue.nunique()})
163
164
165 if __name__ == "__main__":
166     main()

```

scripts/run_q4_comparison.py

```

1 from __future__ import annotations
2
3 import sys
4 from pathlib import Path

```

```

5
6 ROOT = Path(__file__).resolve().parents[1]
7 sys.path.insert(0, str(ROOT / "src"))
8
9 from c_contest_q1.data import load_q1_tables
10 from c_contest_q1.q4_demo import compare_structural_schedule
11
12
13 def main() -> None:
14     actual = ROOT / "outputs" / "q4" / "actual_schedule.csv"
15     optimized = ROOT / "outputs" / "q2" / "copt" / "result_2_group_schedule.
        csv"
16     if not actual.exists():
17         raise FileNotFoundError(
18             "Q4 actual_schedule.csv is intentionally absent. Acquire a
        publicly sourced completed World Cup schedule "
19             "using the official template and source_url/retrieval_date fields,
        then rerun this script."
20         )
21     output = compare_structural_schedule(load_q1_tables(), actual, optimized)
22     out_dir = ROOT / "outputs" / "q4" / "demo"
23     out_dir.mkdir(parents=True, exist_ok=True)
24     output.to_csv(out_dir / "result_4_schedule_comparison.csv", index=False)
25     print(output.to_string(index=False))
26
27
28 if __name__ == "__main__":
29     main()

```

scripts/run_analysis_campaign.py

```

1 from __future__ import annotations
2
3 import json
4 import sys
5 from pathlib import Path
6
7 import numpy as np
8 import pandas as pd
9
10 ROOT = Path(__file__).resolve().parents[1]
11 sys.path.insert(0, str(ROOT / "src"))
12
13 from c_contest_q1.data import load_q1_tables
14 from c_contest_q1.q3_demo import build_q3_state

```

```

15
16
17 P2_KEYS = ["T", "B", "U", "H", "C", "D", "F", "R"]
18 P2_DIRECTIONS = np.array([1, 1, 1, 1, -1, -1, -1, -1], dtype=float)
19 P2_WEIGHTS = np.array([0.25, 0.25, 0.15, 0.10, 0.08, 0.07, 0.06, 0.04], dtype=
    float)
20
21
22 def q2_weight_robustness(out: Path, n_samples: int = 10_000, seed: int =
    20260814) -> dict[str, float]:
23     baseline = json.loads((ROOT / "outputs" / "q2" / "demo" / "objective_score
    .json").read_text(encoding="utf-8"))
24     copt = json.loads((ROOT / "outputs" / "q2" / "copt" / "solve_metadata.json
    ").read_text(encoding="utf-8"))["full_p2_score"]
25     baseline_values = np.array([baseline[f"norm_{key}"] for key in P2_KEYS],
    dtype=float)
26     copt_values = np.array([copt[f"norm_{key}"] for key in P2_KEYS], dtype=
    float)
27     rng = np.random.default_rng(seed)
28     factors = rng.uniform(0.8, 1.2, size=(n_samples, len(P2_KEYS)))
29     weights = factors * P2_WEIGHTS
30     weights /= weights.sum(axis=1, keepdims=True)
31     baseline_scores = (weights * P2_DIRECTIONS * baseline_values).sum(axis=1)
32     copt_scores = (weights * P2_DIRECTIONS * copt_values).sum(axis=1)
33     differences = copt_scores - baseline_scores
34     samples = pd.DataFrame(weights, columns=[f"weight_{key}" for key in
    P2_KEYS])
35     samples["baseline_score"] = baseline_scores
36     samples["copt_score"] = copt_scores
37     samples["copt_minus_baseline"] = differences
38     samples.to_csv(out / "q2_weight_robustness_samples.csv", index=False)
39     return {
40         "samples": n_samples,
41         "copt_win_rate": float((differences > 0).mean()),
42         "difference_min": float(differences.min()),
43         "difference_p05": float(np.quantile(differences, 0.05)),
44         "difference_median": float(np.median(differences)),
45         "difference_p95": float(np.quantile(differences, 0.95)),
46         "difference_max": float(differences.max()),
47     }
48
49
50 def q3_simulation_convergence(out: Path) -> dict[str, float]:
51     tables = load_q1_tables()
52     q2 = ROOT / "outputs" / "q2" / "copt" / "result_2_group_schedule.csv"

```

```

53 q1 = ROOT / "outputs" / "q1" / "demo" / "result_1_match_prediction.csv"
54 counts = [5_000, 10_000, 20_000, 50_000]
55 probability_runs: dict[int, pd.DataFrame] = {}
56 summaries: list[dict[str, float | int]] = []
57 for count in counts:
58     state = build_q3_state(tables, q2, q1, n_simulations=count, seed
=20260814)
59     frame = state.matches[["match_id", "team_a", "team_b", "
updated_p_team_a_advance", "updated_p_team_b_advance", "stakeless_risk", "
collusion_risk"]].copy()
60     probability_runs[count] = frame
61     frame.to_csv(out / f"q3_simulation_{count}.csv", index=False)
62 reference = probability_runs[50_000]
63 for count in counts[:-1]:
64     current = probability_runs[count]
65     differences = np.concatenate([
66         current.updated_p_team_a_advance.to_numpy() - reference.
updated_p_team_a_advance.to_numpy(),
67         current.updated_p_team_b_advance.to_numpy() - reference.
updated_p_team_b_advance.to_numpy(),
68     ])
69     risk_difference = current.collusion_risk.to_numpy() - reference.
collusion_risk.to_numpy()
70     summaries.append({
71         "simulations": count,
72         "probability_mae_vs_50000": float(np.abs(differences).mean()),
73         "probability_max_abs_vs_50000": float(np.abs(differences).max()),
74         "collusion_risk_mae_vs_50000": float(np.abs(risk_difference).mean
()),
75         "qualification_total": float(current[["updated_p_team_a_advance",
"updated_p_team_b_advance"]].sum().sum()),
76     })
77 convergence = pd.DataFrame(summaries)
78 convergence.to_csv(out / "q3_simulation_convergence.csv", index=False)
79 row_20k = convergence.loc[convergence.simulations.eq(20_000)].iloc[0]
80 return {
81     "reference_simulations": 50_000,
82     "probability_mae_20000_vs_50000": float(row_20k.
probability_mae_vs_50000),
83     "probability_max_abs_20000_vs_50000": float(row_20k.
probability_max_abs_vs_50000),
84     "collusion_risk_mae_20000_vs_50000": float(row_20k.
collusion_risk_mae_vs_50000),
85 }
86

```

```

87
88 def q3_resource_comparison(out: Path) -> dict[str, float]:
89     static = pd.read_csv(ROOT / "outputs" / "q3" / "copt" / "static_selection.
90         csv")
91     dynamic = pd.read_csv(ROOT / "outputs" / "q3" / "copt" / "
92         dynamic_selection.csv")
93     result = pd.read_csv(ROOT / "outputs" / "q3" / "copt" / "
94         result_3_dynamic_strategy.csv")
95     keys = ["match_id", "broadcast", "security", "transport", "
96         ticket_adjustment"]
97     decisions = static[keys].merge(dynamic[keys], on="match_id", suffixes=("_static", "_dynamic"))
98     decisions["decision_changed"] = (
99         decisions.broadcast_static.ne(decisions.broadcast_dynamic)
100         | decisions.security_static.ne(decisions.security_dynamic)
101         | decisions.transport_static.ne(decisions.transport_dynamic)
102         | ~np.isclose(decisions.ticket_adjustment_static, decisions.
103             ticket_adjustment_dynamic)
104     )
105     decisions.to_csv(out / "q3_static_dynamic_decisions.csv", index=False)
106     limits = _load_daily_limits()
107     utilization_rows: list[dict[str, float | str]] = []
108     for plan_name, selection in (("Static", static), ("Dynamic", dynamic)):
109         daily = selection.groupby("reference_date").agg(
110             broadcast_used=("broadcast", lambda values: int(values.eq(3).sum()
111             )),
112             security_used=("security", lambda values: int(values.ge(3).sum()))
113         ,
114             transport_used=("transport", lambda values: int(values.eq(3).sum()
115             )),
116             budget_used=("dynamic_resource_cost", "sum"),
117         ).reset_index()
118         daily = daily.merge(limits, on="reference_date", validate="one_to_one"
119         )
120         for row in daily.itertuples(index=False):
121             utilization_rows.append({
122                 "reference_date": row.reference_date,
123                 "plan": plan_name,
124                 "broadcast_used": row.broadcast_used,
125                 "broadcast_capacity": row.high_broadcast_capacity,
126                 "broadcast_utilization": row.broadcast_used / row.
127                 high_broadcast_capacity,
128                 "security_used": row.security_used,
129                 "security_capacity": row.high_security_capacity,
130                 "security_utilization": row.security_used / row.

```

```

high_security_capacity,
121         "transport_used": row.transport_used,
122         "transport_capacity": row.enhanced_transport_capacity,
123         "transport_utilization": row.transport_used / row.
enhanced_transport_capacity,
124         "budget_used": row.budget_used,
125         "budget_capacity": row.daily_resource_budget_index,
126         "budget_utilization": row.budget_used / row.
daily_resource_budget_index,
127     })
128 pd.DataFrame(utilization_rows).to_csv(out / "q3_daily_resource_utilization
.csv", index=False)
129
130 total_metrics = {
131     "attendance": "dynamic_attendance",
132     "tv_viewers": "dynamic_tv_viewers",
133     "ticket_value_usd": "dynamic_ticket_value",
134     "broadcast_value_usd": "dynamic_broadcast_value",
135     "resource_cost_index": "dynamic_resource_cost",
136     "risk_exposure_index": "dynamic_risk",
137     "net_value": "updated_evaluation_score",
138 }
139 total_rows = []
140 for metric, column in total_metrics.items():
141     static_value = float(static[column].sum())
142     dynamic_value = float(dynamic[column].sum())
143     total_rows.append({
144         "metric": metric,
145         "static_value": static_value,
146         "dynamic_value": dynamic_value,
147         "absolute_change": dynamic_value - static_value,
148         "relative_change": (dynamic_value - static_value) / abs(
static_value) if not np.isclose(static_value, 0) else np.nan,
149     })
150 pd.DataFrame(total_rows).to_csv(out / "q3_static_dynamic_totals.csv",
index=False)
151 max_row = result.loc[result.improvement_rate.idxmax()]
152 min_row = result.loc[result.improvement_rate.idxmin()]
153 return {
154     "matches_with_changed_decision": int(decisions.decision_changed.sum())
,
155     "changed_decision_rate": float(decisions.decision_changed.mean()),
156     "matches_with_positive_contribution_change": int(result.
improvement_rate.gt(0).sum()),
157     "matches_with_negative_contribution_change": int(result.

```



```

        improvement_rate.lt(0).sum()),
158     "mean_match_improvement_rate": float(result.improvement_rate.mean()),
159     "total_static_updated_value": float(result.static_net_value.sum()),
160     "total_dynamic_value": float(result.dynamic_net_value.sum()),
161     "total_improvement_rate": float((result.dynamic_net_value.sum() -
result.static_net_value.sum()) / abs(result.static_net_value.sum())),
162     "maximum_improvement_match": str(max_row.match_id),
163     "maximum_improvement_rate": float(max_row.improvement_rate),
164     "minimum_improvement_match": str(min_row.match_id),
165     "minimum_improvement_rate": float(min_row.improvement_rate),
166 }
167
168
169 def _load_daily_limits() -> pd.DataFrame:
170     tables = load_q1_tables()
171     limits = pd.read_excel(tables.paths.workbook, sheet_name="
dynamic_resource_limits")
172     limits["reference_date"] = limits.reference_date.astype(str)
173     return limits[[
174         "reference_date",
175         "high_broadcast_capacity",
176         "high_security_capacity",
177         "enhanced_transport_capacity",
178         "daily_resource_budget_index",
179     ]]
180
181
182 def main() -> None:
183     out = ROOT / "outputs" / "analysis"
184     out.mkdir(parents=True, exist_ok=True)
185     summary = {
186         "execution_envelope": {
187             "processor": "Intel Core i9-14900HX",
188             "q3_solver": "COPT 8.0.6 evaluation mode, 2000-variable MIP limit"
189         },
190         "random_seed": 20260814,
191         "q2_weight_robustness": q2_weight_robustness(out),
192         "q3_simulation_convergence": q3_simulation_convergence(out),
193         "q3_resource_comparison": q3_resource_comparison(out),
194     }
195     (out / "analysis_summary.json").write_text(json.dumps(summary,
ensure_ascii=False, indent=2), encoding="utf-8")
196     report = [
197         "# Modeling Analysis Campaign",

```

```

198     "",
199     "Parent claims: the restricted-candidate Q2 COPT schedule improves the
    fixed feasibility baseline, and Q3 feedback re-optimization improves the
    static resource plan under the same updated evaluation bounds.",
200     "",
201     "## Q2 Weight Robustness",
202     "",
203     f"Across {summary['q2_weight_robustness']['samples'],} independent
    +/-20% weight perturbations normalized to sum to one, the COPT schedule win
    rate was {summary['q2_weight_robustness']['copt_win_rate']:.2%}. The 5th
    percentile objective advantage was {summary['q2_weight_robustness']['
    difference_p05']:.6f}.".
204     "",
205     "## Q3 Monte Carlo Convergence",
206     "",
207     f"Against 50,000 simulations, the official 20,000-draw run had
    probability MAE {summary['q3_simulation_convergence']['
    probability_mae_20000_vs_50000']:.6f} and maximum absolute difference {
    summary['q3_simulation_convergence']['probability_max_abs_20000_vs_50000
    ']:.6f}.".
208     "",
209     "## Q3 Static vs Dynamic Resources",
210     "",
211     f"Feedback changed at least one resource or price decision for {
    summary['q3_resource_comparison']['matches_with_changed_decision']} of 24
    matches. The globally constrained dynamic objective improved by {summary['
    q3_resource_comparison']['total_improvement_rate']:.3%} over the fixed
    static decisions evaluated in the same updated environment.",
212     "",
213     "Comparability: Q2 changes weights only; Q3 convergence changes
    simulation count only; static/dynamic Q3 uses one candidate set, identical
    capacities and budgets, and common updated-environment normalization bounds
    .",
214     "",
215     "Next route: use these results in the paper and proceed to Q4 external
    schedule comparison.",
216 ]
217 (out / "analysis_report.md").write_text("\n".join(report) + "\n", encoding
    ="utf-8")
218 print(json.dumps(summary, ensure_ascii=False, indent=2))
219
220
221 if __name__ == "__main__":
222     main()

```

scripts/build_paper_figures.py

```
1 from __future__ import annotations
2
3 import json
4 from pathlib import Path
5
6 import matplotlib.pyplot as plt
7 import numpy as np
8 import pandas as pd
9
10
11 ROOT = Path(__file__).resolve().parents[1]
12 OUT = ROOT / "paper" / "figures"
13 COLORS = {"actual": "#4D6A86", "optimized": "#C97A40", "accent": "#4F8A5B", "
           muted": "#8A8A8A"}
14
15
16 def _style() -> None:
17     plt.rcParams.update({
18         "font.family": "DejaVu Sans",
19         "font.size": 9,
20         "axes.spines.top": False,
21         "axes.spines.right": False,
22         "axes.grid": True,
23         "axes.grid.axis": "y",
24         "grid.alpha": 0.22,
25         "figure.dpi": 150,
26         "savefig.dpi": 300,
27         "savefig.bbox": "tight",
28     })
29
30
31 def _save(fig: plt.Figure, name: str) -> None:
32     fig.savefig(OUT / f"{name}.png")
33     fig.savefig(OUT / f"{name}.pdf")
34     plt.close(fig)
35
36
37 def q1_model_comparison() -> None:
38     leaderboard = pd.read_csv(ROOT / "outputs/q1/sklearn_challengers/
39     leaderboard.csv").sort_values("mse")
40     folds = pd.read_csv(ROOT / "outputs/q1/sklearn_challengers/fold_metrics.
41     csv")
42     labels = leaderboard.model.str.replace("_", " ").str.title()
43     colors = [COLORS["accent"] if model == "ridge" else COLORS["actual"] for
```

```

model in leaderboard.model]
42 fig, axes = plt.subplots(1, 2, figsize=(10.2, 4.0), gridspec_kw={"
width_ratios": [1.05, 1.2]})
43 axes[0].barh(labels[::-1], leaderboard.mse[::-1], color=colors[::-1])
44 axes[0].set_xlabel("Mean temporal-validation MSE")
45 axes[0].set_title("Seven-model leaderboard")
46 for model, block in folds.groupby("model"):
47     if model in {"ridge", "extra_trees", "mean"}:
48         axes[1].plot(block.fold, block.mse, marker="o", label=model.
replace("_", " ").title())
49 axes[1].set_ylabel("MSE (million viewers squared)")
50 axes[1].set_title("Representative expanding-window folds")
51 axes[1].legend(frameon=False)
52 fig.tight_layout()
53 _save(fig, "q1_model_comparison")
54
55
56 def q2_indicator_decomposition() -> None:
57     baseline = json.loads((ROOT / "outputs/q2/demo/objective_score.json").
read_text(encoding="utf-8"))
58     copt = json.loads((ROOT / "outputs/q2/copt/solve_metadata.json").read_text
(encoding="utf-8"))["full_p2_score"]
59     keys = ["T", "B", "U", "H", "C", "D", "F", "R"]
60     labels = ["Ticket", "Broadcast", "Uncertainty", "Attractiveness", "Cost",
"Travel", "Fairness", "Risk"]
61     x = np.arange(len(keys))
62     width = 0.38
63     fig, ax = plt.subplots(figsize=(8.2, 3.6))
64     ax.bar(x - width / 2, [baseline[f"norm_{k}"] for k in keys], width, label=
"Feasible baseline", color=COLORS["actual"])
65     ax.bar(x + width / 2, [copt[f"norm_{k}"] for k in keys], width, label="
Restricted COPT", color=COLORS["optimized"])
66     ax.set_xticks(x, labels, rotation=24, ha="right")
67     ax.set_ylabel("Normalized indicator")
68     ax.set_ylim(0, 1.08)
69     ax.legend(frameon=False, ncol=2)
70     ax.set_title("Q2 schedule indicator decomposition")
71     _save(fig, "q2_indicator_decomposition")
72
73
74 def q2_weight_robustness() -> None:
75     data = pd.read_csv(ROOT / "outputs/analysis/q2_weight_robustness_samples.
csv")
76     fig, ax = plt.subplots(figsize=(6.4, 3.5))
77     ax.hist(data.copt_minus_baseline, bins=36, color=COLORS["accent"], alpha

```

```

=0.88)
78     ax.axvline(data.copt_minus_baseline.quantile(0.05), color=COLORS["
optimized"], linestyle="--", label="5th percentile")
79     ax.axvline(0, color="black", linewidth=0.8)
80     ax.set_xlabel("COPT score minus baseline score")
81     ax.set_ylabel("Weight perturbations")
82     ax.set_title("Q2 robustness under 10,000 weight perturbations")
83     ax.legend(frameon=False)
84     _save(fig, "q2_weight_robustness")
85
86
87 def q3_probabilities() -> None:
88     data = pd.read_csv(ROOT / "outputs/q3/copt/result_3_dynamic_strategy.csv")
89     long = pd.concat([
90         data[["team_a", "updated_p_team_a_advance"]].rename(columns={"team_a":
"team", "updated_p_team_a_advance": "probability"}),
91         data[["team_b", "updated_p_team_b_advance"]].rename(columns={"team_b":
"team", "updated_p_team_b_advance": "probability"}),
92     ]).sort_values("probability")
93     fig, ax = plt.subplots(figsize=(8.2, 7.2))
94     colors = np.where(long.probability.between(0.25, 0.75), COLORS["optimized"
], COLORS["actual"])
95     ax.barh(long.team, long.probability, color=colors)
96     ax.axvline(0.5, color="black", linewidth=0.8, linestyle="--")
97     ax.set_xlim(0, 1)
98     ax.set_xlabel("Probability of advancing")
99     ax.set_title("Q3 advancement probabilities after two rounds")
100     _save(fig, "q3_advancement_probabilities")
101
102
103 def q3_importance_attractiveness() -> None:
104     data = pd.read_csv(ROOT / "outputs/q3/copt/simulation_summary.csv")
105     fig, axes = plt.subplots(1, 2, figsize=(8.2, 3.5))
106     axes[0].hist(data.importance, bins=10, color=COLORS["actual"], alpha=0.9)
107     axes[0].set_xlabel("Qualification importance Q")
108     axes[0].set_ylabel("Third-round matches")
109     axes[0].set_title("Qualification importance distribution")
110     axes[1].scatter(
111         data.importance,
112         data.updated_attractiveness,
113         c=data.stakeless_risk,
114         cmap="viridis",
115         edgecolor="white",
116         linewidth=0.5,
117         s=48,

```

```

118     )
119     axes[1].set_xlabel("Qualification importance Q")
120     axes[1].set_ylabel("Updated attractiveness A")
121     axes[1].set_title("Importance and updated attractiveness")
122     fig.tight_layout()
123     _save(fig, "q3_importance_attractiveness")
124
125
126 def q3_static_dynamic() -> None:
127     data = pd.read_csv(ROOT / "outputs/q3/copt/result_3_dynamic_strategy.csv")
128     data["change"] = data.dynamic_net_value - data.static_net_value
129     data = data.sort_values("change")
130     fig, ax = plt.subplots(figsize=(7.8, 5.2))
131     colors = np.where(data.change >= 0, COLORS["accent"], COLORS["optimized"])
132     ax.barh(data.match_id, data.change, color=colors)
133     ax.axvline(0, color="black", linewidth=0.8)
134     ax.set_xlabel("Dynamic minus static contribution")
135     ax.set_title("Q3 match-level reallocation effects")
136     _save(fig, "q3_static_dynamic_contributions")
137
138
139 def q3_convergence() -> None:
140     data = pd.read_csv(ROOT / "outputs/analysis/q3_simulation_convergence.csv"
141     )
142     fig, ax = plt.subplots(figsize=(6.4, 3.5))
143     ax.plot(data.simulations, data.probability_mae_vs_50000, marker="o", color
144     =COLORS["actual"], label="Probability MAE")
145     ax.plot(data.simulations, data.collusion_risk_mae_vs_50000, marker="s",
146     color=COLORS["optimized"], label="Collusion-risk MAE")
147     ax.set_xlabel("Monte Carlo draws")
148     ax.set_ylabel("Absolute error vs 50,000 draws")
149     ax.set_title("Q3 simulation convergence")
150     ax.legend(frameon=False)
151     _save(fig, "q3_simulation_convergence")
152
153
154 def q3_daily_resources() -> None:
155     data = pd.read_csv(ROOT / "outputs/analysis/q3_daily_resource_utilization.
156     csv")
157     metrics = [
158         ("broadcast_utilization", "Priority-3 broadcast"),
159         ("security_utilization", "High security"),
160         ("transport_utilization", "Level-3 transport"),
161         ("budget_utilization", "Resource budget"),
162     ]

```

```

159     fig, axes = plt.subplots(2, 2, figsize=(8.2, 5.2), sharex=True, sharey=
True)
160     for ax, (metric, title) in zip(axes.flat, metrics, strict=True):
161         for plan, color, marker in (("Static", COLORS["actual"], "o"), ("
Dynamic", COLORS["optimized"], "s")):
162             block = data.loc[data.plan.eq(plan)].sort_values("reference_date")
163             ax.plot(
164                 block.reference_date.str.slice(5),
165                 block[metric],
166                 marker=marker,
167                 color=color,
168                 label=plan,
169                 markersize=7 if plan == "Static" else 4.5,
170                 markerfacecolor="white" if plan == "Static" else color,
171                 markeredgewidth=1.4,
172                 linewidth=1.8 if plan == "Static" else 1.4,
173             )
174             ax.axhline(1.0, color="black", linewidth=0.8, linestyle="--")
175             ax.set_title(title)
176             ax.set_ylim(0, 1.08)
177             ax.tick_params(axis="x", rotation=28)
178     axes[0, 0].set_ylabel("Used / capacity")
179     axes[1, 0].set_ylabel("Used / capacity")
180     axes[0, 1].legend(frameon=False, ncol=2, loc="lower left")
181     fig.suptitle("Q3 daily resource utilization under supplied limits", y
=1.01)
182     fig.tight_layout()
183     _save(fig, "q3_daily_resource_utilization")
184
185
186 def q4_tradeoffs() -> None:
187     data = pd.read_csv(ROOT / "outputs/q4/demo/result_4_schedule_comparison.
csv").set_index("indicator_name")
188     names = ["travel_km_per_team", "mean_rest_hours", "minimum_rest_hours", "
matches_per_venue_cv", "venue_match_hhi", "mean_stadium_capacity"]
189     labels = ["Travel / team", "Mean rest", "Minimum rest", "Venue-load CV", "
Venue HHI", "Mean capacity"]
190     actual = data.loc[names, "actual_schedule_value"].to_numpy(dtype=float)
191     optimized = data.loc[names, "optimized_schedule_value"].to_numpy(dtype=
float)
192     directions = data.loc[names, "preferred_direction"].to_numpy()
193     actual_score = np.empty_like(actual)
194     optimized_score = np.empty_like(optimized)
195     for index, direction in enumerate(directions):
196         low, high = min(actual[index], optimized[index]), max(actual[index],

```

```

    optimized[index])
197     if np.isclose(low, high):
198         actual_score[index] = optimized_score[index] = 1.0
199     elif direction == "lower":
200         actual_score[index] = (high - actual[index]) / (high - low)
201         optimized_score[index] = (high - optimized[index]) / (high - low)
202     else:
203         actual_score[index] = (actual[index] - low) / (high - low)
204         optimized_score[index] = (optimized[index] - low) / (high - low)
205 x = np.arange(len(names))
206 width = 0.38
207 fig, ax = plt.subplots(figsize=(7.6, 3.8))
208 ax.bar(x - width / 2, actual_score, width, color=COLORS["actual"], label="
Actual 2026")
209 ax.bar(x + width / 2, optimized_score, width, color=COLORS["optimized"],
label="Q2 optimized")
210 ax.set_xticks(x, labels, rotation=24, ha="right")
211 ax.set_ylabel("Pairwise direction-aligned score")
212 ax.set_ylim(0, 1.08)
213 ax.set_title("Q4 structural trade-offs (higher is preferable)")
214 ax.legend(frameon=False, ncol=2)
215 _save(fig, "q4_structural_tradeoffs")
216
217
218 def main() -> None:
219     OUT.mkdir(parents=True, exist_ok=True)
220     _style()
221     source_figures = ROOT / "outputs/q1/demo/figures"
222     (OUT / "q1_ridge_diagnostics.png").write_bytes((source_figures / "
ridge_diagnostics.png").read_bytes())
223     q1_model_comparison()
224     q2_indicator_decomposition()
225     q2_weight_robustness()
226     q3_probabilities()
227     q3_importance_attractiveness()
228     q3_static_dynamic()
229     q3_convergence()
230     q3_daily_resources()
231     q4_tradeoffs()
232     print(f"Paper figures written to {OUT}")
233
234
235 if __name__ == "__main__":
236     main()

```


scripts/validate_deliverables.py

```
1 from __future__ import annotations
2
3 import hashlib
4 import json
5 import sys
6 from dataclasses import asdict, dataclass
7 from pathlib import Path
8
9 import numpy as np
10 import pandas as pd
11
12
13 ROOT = Path(__file__).resolve().parents[1]
14 sys.path.insert(0, str(ROOT / "src"))
15
16 from c_contest_q1.data import load_q1_tables
17 from c_contest_q1.outputs import validate_demo_outputs
18 from c_contest_q1.paths import discover_template_path
19 from c_contest_q1.q2_demo import validate_q2_schedule
20 from c_contest_q1.q3_demo import validate_q3_result
21 from c_contest_q1.q4_demo import validate_actual_schedule
22
23
24 @dataclass(frozen=True)
25 class Check:
26     name: str
27     ok: bool
28     detail: str
29
30
31 def _official_templates() -> dict[str, Path]:
32     expected = {
33         "actual_schedule_template.csv",
34         "result_1_match_prediction_template.csv",
35         "result_1_test_prediction_template.csv",
36         "result_2_template.csv",
37         "result_3_template.csv",
38         "result_4_template.csv",
39     }
40     return {name: discover_template_path(name) for name in expected}
41
42
43 def _schema_check(name: str, output: Path, template: Path) -> Check:
44     actual_columns = list(pd.read_csv(output, nrows=0).columns)
```

```

45     expected_columns = list(pd.read_csv(template, nrows=0).columns)
46     return Check(
47         name=f"{name}_schema",
48         ok=actual_columns == expected_columns,
49         detail=f"columns={len(actual_columns)}, exact_order={actual_columns ==
50         expected_columns}",
51     )
52
53 def _hash(path: Path) -> str:
54     return hashlib.sha256(path.read_bytes()).hexdigest()
55
56
57 def validate_all() -> tuple[list[Check], dict[str, str]]:
58     tables = load_q1_tables()
59     templates = _official_templates()
60     paths = {
61         "q1_test": ROOT / "outputs/q1/demo/result_1_test_prediction.csv",
62         "q1_match": ROOT / "outputs/q1/demo/result_1_match_prediction.csv",
63         "q2": ROOT / "outputs/q2/copt/result_2_group_schedule.csv",
64         "q3": ROOT / "outputs/q3/copt/result_3_dynamic_strategy.csv",
65         "actual": ROOT / "outputs/q4/actual_schedule.csv",
66         "q4": ROOT / "outputs/q4/demo/result_4_schedule_comparison.csv",
67     }
68     missing = [str(path) for path in paths.values() if not path.exists()]
69     if missing:
70         raise FileNotFoundError(f"Required deliverables not found: {missing}")
71
72     checks: list[Check] = []
73     schema_pairs = {
74         "q1_test": "result_1_test_prediction_template.csv",
75         "q1_match": "result_1_match_prediction_template.csv",
76         "q2": "result_2_template.csv",
77         "q3": "result_3_template.csv",
78         "actual": "actual_schedule_template.csv",
79         "q4": "result_4_template.csv",
80     }
81     checks.extend(
82         _schema_check(name, paths[name], templates[template])
83         for name, template in schema_pairs.items()
84     )
85
86     q1_report = validate_demo_outputs(tables, paths["q1_test"], paths["
87     q1_match"])
88     checks.append(Check("q1_content_and_units", q1_report.ok, "; ".join(

```

```

q1_report.errors) or "140 test rows and 72 match rows validated"))
88
89 q2 = pd.read_csv(paths["q2"])
90 q2_metadata = json.loads((ROOT / "outputs/q2/copt/solve_metadata.json").
read_text(encoding="utf-8"))
91 q2_report = validate_q2_schedule(tables, q2)
92 objective_locations = np.flatnonzero(q2.total_objective_value.notna().
to_numpy()).tolist()
93 q2_objective_ok = objective_locations == [0] and np.isfinite(q2.loc[0, "
total_objective_value"])
94 checks.append(Check("q2_six_hard_constraint_groups", q2_report.ok, "; ".
join(q2_report.errors) or "all independent assertions passed"))
95 checks.append(Check("q2_objective_first_row_only", bool(q2_objective_ok),
f"nonempty_rows={objective_locations}"))
96 q2_reconciled = np.isclose(
97     float(q2_metadata["mip_objective"]),
98     float(q2_metadata["full_p2_score"]["objective"]),
99     atol=1e-8,
100 )
101 q2_evidence = q2_metadata["solve_evidence"]
102 q2_log = (ROOT / "outputs/q2/copt/solver.log").read_text(encoding="utf-8",
errors="replace")
103 q2_solver_ok = (
104     q2_evidence["rows"] == 865
105     and q2_evidence["columns"] == 844
106     and q2_evidence["binary_variables"] == 840
107     and np.isclose(q2_evidence["best_bound"], q2_metadata["mip_objective"
], atol=1e-10)
108     and np.isclose(q2_evidence["relative_gap"], 0.0, atol=1e-12)
109     and "865 rows, 844 columns" in q2_log
110     and "840 binaries" in q2_log
111     and "Best solution : 0.458163650" in q2_log
112     and "Best gap : 0.0000%" in q2_log
113     and "Solution status : integer optimal" in q2_log
114     and "integrality : 0" in q2_log
115 )
116 checks.append(Check(
117     "q2_copt_objective_reconciliation",
118     bool(q2_reconciled and q2_solver_ok),
119     f"copt={q2_metadata['mip_objective']:.12f}, evaluator={q2_metadata['
full_p2_score']['objective']:.12f}, rows/cols/bins=865/844/840, gap={
q2_evidence['relative_gap']:.1f}",
120 ))
121
122 q3 = pd.read_csv(paths["q3"])

```

```

123 q3_report = validate_q3_result(tables, paths["q2"], q3)
124 q3_metadata = json.loads((ROOT / "outputs/q3/copt/solve_metadata.json").
read_text(encoding="utf-8"))
125 q3_log = (ROOT / "outputs/q3/copt/solver.log").read_text(encoding="utf-8",
errors="replace")
126 q3_solver_ok = all(
127     evidence["rows"] == 48
128     and evidence["columns"] == 1623
129     and evidence["binary_variables"] == 1623
130     and np.isclose(evidence["relative_gap"], 0.0, atol=1e-12)
131     for evidence in q3_metadata["solve_evidence"].values()
132 ) and (
133     np.isclose(q3_metadata["solve_evidence"]["static"]["best_bound"],
q3_metadata["static_optimization_objective"], atol=1e-10)
134     and np.isclose(q3_metadata["solve_evidence"]["dynamic"]["best_bound"],
q3_metadata["dynamic_optimization_objective"], atol=1e-10)
135     and np.isclose(q3_metadata["solve_evidence"]["feasible_lower_bound"]["
best_bound"], q3_metadata["feasible_objective_lower_bound"], atol=1e-10)
136     and q3_metadata["feasible_objective_lower_bound"] <= q3_metadata["
static_updated_evaluation"] <= q3_metadata["feasible_objective_upper_bound"
]
137     and "=== Q3 STATIC COPT MODEL ===" in q3_log
138     and "=== Q3 DYNAMIC COPT MODEL ===" in q3_log
139     and "=== Q3 FEASIBLE OBJECTIVE LOWER-BOUND COPT MODEL ===" in q3_log
140     and q3_log.count("48 rows, 1623 columns") == 3
141     and q3_log.count("1623 binaries") >= 3
142     and q3_log.count("Best gap : 0.0000%") == 3
143     and q3_log.count("Solution status : integer optimal") == 3
144     and q3_log.count("integrality : 0") == 3
145 )
146 q3_detail = "; ".join(q3_report.errors) or "24 matches, probability total
32, all capacity checks passed"
147 checks.append(Check("q3_schedule_and_resource_constraints", bool(q3_report
.ok and q3_solver_ok), q3_detail + "; three 48x1623 COPT models at zero gap
, including the feasible Z3 lower bound"))
148
149 actual = pd.read_csv(paths["actual"])
150 actual_errors = list(validate_actual_schedule(actual))
151 teams = pd.concat([actual.team_a, actual.team_b])
152 appearances = pd.concat([
153     actual[["team_a", "round_in_group"]].rename(columns={"team_a": "team"
}),
154     actual[["team_b", "round_in_group"]].rename(columns={"team_b": "team"
}),
155 ])

```

```

156     actual_structure_ok = (
157         len(actual) == 72
158         and actual.group_id.nunique() == 12
159         and teams.nunique() == 48
160         and appearances.groupby("team").size().eq(3).all()
161         and actual.groupby("group_id").size().eq(6).all()
162     )
163     checks.append(Check("q4_actual_source_contract", not actual_errors, "; ".
164         join(actual_errors) or "row-level URL and retrieval date present"))
165     checks.append(Check("q4_actual_tournament_structure", bool(
166         actual_structure_ok), f"matches={len(actual)}, groups={actual.group_id.
167         nunique()}, teams={teams.nunique()}"))
168
169     q4 = pd.read_csv(paths["q4"])
170     q4_numeric = q4[["actual_schedule_value", "optimized_schedule_value", "
171         absolute_difference"]]
172     q4_ok = len(q4) >= 10 and np.isfinite(q4_numeric.to_numpy()).all()
173     checks.append(Check("q4_comparison_content", bool(q4_ok), f"indicators={
174         len(q4)}, finite_core_values={bool(np.isfinite(q4_numeric.to_numpy()).all()
175         )}"))
176
177     hashes = {name: _hash(path) for name, path in paths.items()}
178     return checks, hashes
179
180
181
182
183 def main() -> None:
184     checks, hashes = validate_all()
185     report = {
186         "ok": all(check.ok for check in checks),
187         "checks": [asdict(check) for check in checks],
188         "sha256": hashes,
189     }
190     out = ROOT / "outputs/validation"
191     out.mkdir(parents=True, exist_ok=True)
192     (out / "deliverable_validation.json").write_text(
193         json.dumps(report, ensure_ascii=False, indent=2), encoding="utf-8"
194     )
195     for check in checks:
196         print(f"[{'PASS' if check.ok else 'FAIL'}] {check.name}: {check.detail
197         }")
198     if not report["ok"]:
199         raise SystemExit(1)
200     print(f"Validated {len(checks)} deliverable contracts; report={out / '
201     deliverable_validation.json'}")

```

```

193
194 if __name__ == "__main__":
195     main()

```

scripts/build_submission.py

```

1  """Build the anonymous paper and a whitelist-only support archive."""
2
3  from __future__ import annotations
4
5  import argparse
6  import hashlib
7  import json
8  import re
9  import shutil
10 import tempfile
11 import zipfile
12 from pathlib import Path
13
14
15 ROOT = Path(__file__).resolve().parents[1]
16 SUBMISSION = ROOT / "submission"
17 SIZE_LIMIT = 20 * 1024 * 1024
18
19 SUPPORT_FILES = (
20     "README.md",
21     "pyproject.toml",
22     "requirements-demo.txt",
23     "requirements-copt.txt",
24     "docs/project/COPT_EVIDENCE.md",
25     "docs/project/Q4_DATA_CONTRACT.md",
26     "outputs/q1/demo/result_1_test_prediction.csv",
27     "outputs/q1/demo/result_1_match_prediction.csv",
28     "outputs/q1/demo/leaderboard.csv",
29     "outputs/q1/demo/fold_metrics.csv",
30     "outputs/q1/demo/feature_manifest.json",
31     "outputs/q2/demo/result_2_group_schedule.csv",
32     "outputs/q2/demo/objective_score.json",
33     "outputs/q2/demo/feasibility_report.txt",
34     "outputs/q2/copt/result_2_group_schedule.csv",
35     "outputs/q2/copt/solve_metadata.json",
36     "outputs/q2/copt/solver.log",
37     "outputs/q3/copt/result_3_dynamic_strategy.csv",
38     "outputs/q3/copt/simulation_summary.csv",
39     "outputs/q3/copt/static_selection.csv",

```

```

40     "outputs/q3/copt/dynamic_selection.csv",
41     "outputs/q3/copt/feasible_lower_bound_selection.csv",
42     "outputs/q3/copt/solve_metadata.json",
43     "outputs/q3/copt/solver.log",
44     "outputs/q4/actual_schedule.csv",
45     "outputs/q4/source_hashes.json",
46     "outputs/q4/demo/result_4_schedule_comparison.csv",
47     "outputs/analysis/analysis_summary.json",
48     "outputs/analysis/analysis_report.md",
49     "outputs/analysis/q2_weight_robustness_samples.csv",
50     "outputs/analysis/q3_simulation_convergence.csv",
51     "outputs/analysis/q3_static_dynamic_decisions.csv",
52     "outputs/analysis/q3_daily_resource_utilization.csv",
53     "outputs/analysis/q3_static_dynamic_totals.csv",
54     "outputs/validation/deliverable_validation.json",
55     "research/q4/openfootball_2026_cup.txt",
56     "research/q4/openfootball_2026_stadiums.csv",
57     "research/q4/SOURCE.md",
58 )
59
60 SUPPORT_TREES = ("src", "scripts", "tests")
61 TREE_SUFFIXES = {".py", ".ps1"}
62 PAPER_SOURCES = (
63     "paper/main.tex",
64     "paper/ai_usage.tex",
65     "paper/modelingpaper.sty",
66     "paper/references.bib",
67     "paper/source_appendix.tex",
68 )
69 OFFICIAL_TEMPLATES = (
70     "actual_schedule_template.csv",
71     "result_1_match_prediction_template.csv",
72     "result_1_test_prediction_template.csv",
73     "result_2_template.csv",
74     "result_3_template.csv",
75     "result_4_template.csv",
76 )
77
78
79 def sha256(path: Path) -> str:
80     return hashlib.sha256(path.read_bytes()).hexdigest()
81
82
83 def parse_args() -> argparse.Namespace:
84     parser = argparse.ArgumentParser()

```

```

85     parser.add_argument(
86         "--team-id",
87         required=True,
88         help="Three digits for the official submission, or XXX for a
placeholder build.",
89     )
90     return parser.parse_args()
91
92
93 def validate_team_id(value: str) -> str:
94     value = value.strip().upper()
95     if not re.fullmatch(r"(?:\d{3}|XXX)", value):
96         raise ValueError("team id must be exactly three digits, or XXX for a
placeholder build")
97     return value
98
99
100 def required_sources() -> list[Path]:
101     paths = [ROOT / relative for relative in SUPPORT_FILES]
102     paths.append(ROOT / "paper/build/main.pdf")
103     missing = [str(path.relative_to(ROOT)) for path in paths if not path.
is_file()]
104     if missing:
105         raise FileNotFoundError(f"Required submission artifacts are missing: {
missing}")
106     return paths
107
108
109 def add_support_tree(stage: Path) -> None:
110     for tree_name in SUPPORT_TREES:
111         tree = ROOT / tree_name
112         for source in sorted(tree.rglob("*")):
113             if source.is_file() and source.suffix.lower() in TREE_SUFFIXES:
114                 destination = stage / source.relative_to(ROOT)
115                 destination.parent.mkdir(parents=True, exist_ok=True)
116                 shutil.copy2(source, destination)
117
118
119 def add_paper_sources(stage: Path) -> None:
120     for relative in PAPER_SOURCES:
121         source = ROOT / relative
122         destination = stage / relative
123         destination.parent.mkdir(parents=True, exist_ok=True)
124         shutil.copy2(source, destination)
125

```



```

126
127 def add_official_inputs(stage: Path) -> None:
128     sys_path = str(ROOT / "src")
129     if sys_path not in __import__("sys").path:
130         __import__("sys").path.insert(0, sys_path)
131     from c_contest_q1.paths import discover_source_paths,
discover_template_path
132
133     destination_root = stage / "data" / "official"
134     destination_root.mkdir(parents=True, exist_ok=True)
135     source_paths = discover_source_paths()
136     shutil.copy2(source_paths.workbook, destination_root / "c_contest_inputs.
xlsx")
137     for name in OFFICIAL_TEMPLATES:
138         shutil.copy2(discover_template_path(name), destination_root / name)
139
140
141 def build(team_id: str) -> dict[str, object]:
142     required_sources()
143     SUBMISSION.mkdir(parents=True, exist_ok=True)
144     paper_output = SUBMISSION / f"{team_id}_参赛论文.pdf"
145     zip_output = SUBMISSION / f"{team_id}_支撑材料.zip"
146     shutil.copy2(ROOT / "paper/build/main.pdf", paper_output)
147
148     with tempfile.TemporaryDirectory(prefix="c_contest_submission_") as tmp:
149         stage = Path(tmp) / f"{team_id}_支撑材料"
150         for relative in SUPPORT_FILES:
151             source = ROOT / relative
152             destination = stage / relative
153             destination.parent.mkdir(parents=True, exist_ok=True)
154             shutil.copy2(source, destination)
155             shutil.copy2(ROOT / "paper/build/ai_usage.pdf", stage / "AI工具使用详情
.pdf")
156             add_support_tree(stage)
157             add_paper_sources(stage)
158             add_official_inputs(stage)
159
160         entries = []
161         for source in sorted(path for path in stage.rglob("*") if path.is_file
()):
162             relative = source.relative_to(stage).as_posix()
163             entries.append({"path": relative, "bytes": source.stat().st_size,
"sha256": sha256(source)})
164         (stage / "support_manifest.json").write_text(
165             json.dumps({"team_id": team_id, "files": entries}, ensure_ascii=

```

```

False, indent=2),
166         encoding="utf-8",
167     )
168
169     with zipfile.ZipFile(zip_output, "w", compression=zipfile.ZIP_DEFLATED
, compresslevel=9) as archive:
170         for source in sorted(path for path in stage.rglob("*") if path.
is_file()):
171             archive.write(source, source.relative_to(stage).as_posix())
172
173     for path in (paper_output, zip_output):
174         if path.stat().st_size >= SIZE_LIMIT:
175             raise ValueError(f"{path.name} is {path.stat().st_size} bytes,
exceeding the strict <20 MB limit")
176
177     with zipfile.ZipFile(zip_output) as archive:
178         bad_member = archive.testzip()
179         if bad_member is not None:
180             raise ValueError(f"ZIP CRC validation failed at {bad_member}")
181         names = archive.namelist()
182         if len(names) != len(set(names)):
183             raise ValueError("ZIP contains duplicate member names")
184
185     manifest = {
186         "team_id": team_id,
187         "placeholder_name": team_id == "XXX",
188         "size_limit_bytes": SIZE_LIMIT,
189         "files": {
190             paper_output.name: {"bytes": paper_output.stat().st_size, "sha256"
: sha256(paper_output)},
191             zip_output.name: {"bytes": zip_output.stat().st_size, "sha256":
sha256(zip_output)},
192         },
193         "support_archive_members": len(names),
194     }
195     (SUBMISSION / "submission_manifest.json").write_text(
196         json.dumps(manifest, ensure_ascii=False, indent=2), encoding="utf-8"
197     )
198     return manifest
199
200
201 def main() -> None:
202     team_id = validate_team_id(parse_args().team_id)
203     print(json.dumps(build(team_id), ensure_ascii=False, indent=2))
204

```

```

205
206 if __name__ == "__main__":
207     main()

```

自动测试

tests/conftest.py

```

1 from __future__ import annotations
2
3 import sys
4 from pathlib import Path
5
6
7 PROJECT_ROOT = Path(__file__).resolve().parents[1]
8 SRC_ROOT = PROJECT_ROOT / "src"
9 if str(SRC_ROOT) not in sys.path:
10     sys.path.insert(0, str(SRC_ROOT))

```

tests/test_paths.py

```

1 from __future__ import annotations
2
3 from pathlib import Path
4
5 import pandas as pd
6 import pytest
7
8 from c_contest_q1 import paths
9
10
11 OFFICIAL_TEMPLATES = (
12     "actual_schedule_template.csv",
13     "result_1_match_prediction_template.csv",
14     "result_1_test_prediction_template.csv",
15     "result_2_template.csv",
16     "result_3_template.csv",
17     "result_4_template.csv",
18 )
19
20
21 def _write_workbook(path: Path, sheet_names: set[str] | frozenset[str]) ->
    None:
22     path.parent.mkdir(parents=True, exist_ok=True)
23     with pd.ExcelWriter(path) as writer:
24         for sheet_name in sorted(sheet_names):

```

```

25         pd.DataFrame({"value": [1]}).to_excel(
26             writer, sheet_name=sheet_name, index=False
27         )
28
29
30 def _patch_roots(monkeypatch: pytest.MonkeyPatch, project: Path, contest: Path
31 ) -> None:
32     monkeypatch.setattr(paths, "project_root", lambda: project)
33     monkeypatch.setattr(paths, "contest_root", lambda: contest)
34
35 def test_bundled_inputs_take_priority_over_external_copies(
36     tmp_path: Path, monkeypatch: pytest.MonkeyPatch
37 ) -> None:
38     project = tmp_path / "support"
39     bundled = project / "data" / "official"
40     external = tmp_path / "official_attachments"
41     _patch_roots(monkeypatch, project, tmp_path)
42
43     _write_workbook(bundled / "c_contest_inputs.xlsx", paths.REQUIRED_SHEETS)
44     _write_workbook(external / "external_inputs.xlsx", paths.REQUIRED_SHEETS)
45     for name in OFFICIAL_TEMPLATES:
46         bundled.mkdir(parents=True, exist_ok=True)
47         (bundled / name).write_text("bundled\n", encoding="utf-8")
48         external.mkdir(parents=True, exist_ok=True)
49         (external / name).write_text("external\n", encoding="utf-8")
50
51     sources = paths.discover_source_paths()
52
53     assert sources.workbook == (bundled / "c_contest_inputs.xlsx").resolve()
54     assert sources.test_template == (
55         bundled / "result_1_test_prediction_template.csv"
56     ).resolve()
57     assert sources.match_template == (
58         bundled / "result_1_match_prediction_template.csv"
59     ).resolve()
60     for name in OFFICIAL_TEMPLATES:
61         assert paths.discover_template_path(name) == (bundled / name).resolve()
62
63
64 def test_external_inputs_are_discovered_when_bundle_is_absent(
65     tmp_path: Path, monkeypatch: pytest.MonkeyPatch
66 ) -> None:
67     project = tmp_path / "support"

```

```

68     external = tmp_path / "official_attachments"
69     _patch_roots(monkeypatch, project, tmp_path)
70
71     workbook = external / "inputs.xlsx"
72     _write_workbook(workbook, paths.REQUIRED_SHEETS)
73     external.mkdir(parents=True, exist_ok=True)
74     for name in OFFICIAL_TEMPLATES:
75         (external / name).write_text("official\n", encoding="utf-8")
76
77     sources = paths.discover_source_paths()
78
79     assert sources.workbook == workbook.resolve()
80     for name in OFFICIAL_TEMPLATES:
81         assert paths.discover_template_path(name) == (external / name).resolve
82         ()
83
84 def test_multiple_external_workbooks_fail_instead_of_guessing(
85     tmp_path: Path, monkeypatch: pytest.MonkeyPatch
86 ) -> None:
87     project = tmp_path / "support"
88     _patch_roots(monkeypatch, project, tmp_path)
89     _write_workbook(tmp_path / "attachments_a" / "inputs.xlsx", paths.
REQUIRED_SHEETS)
90     _write_workbook(tmp_path / "attachments_b" / "inputs.xlsx", paths.
REQUIRED_SHEETS)
91
92     with pytest.raises(FileNotFoundError, match="Expected exactly one C-
contest workbook"):
93         paths._find_workbook()

```

tests/test_q1_data.py

```

1  from __future__ import annotations
2
3  from pathlib import Path
4
5  import numpy as np
6  import pandas as pd
7
8  from c_contest_q1.data import load_q1_tables, validate_q1_tables
9  from c_contest_q1.paths import discover_source_paths
10
11
12 def test_discovers_official_workbook_and_templates() -> None:

```

```

13     paths = discover_source_paths()
14
15     assert paths.workbook.is_file()
16     assert not paths.workbook.name.startswith("~$")
17     assert paths.test_template.name == "result_1_test_prediction_template.csv"
18     assert paths.match_template.name == "result_1_match_prediction_template.
19         csv"
20
21 def test_loads_fixed_q1_row_contract() -> None:
22     tables = load_q1_tables()
23
24     assert tables.historical.shape == (700, 25)
25     assert tables.teams.shape == (48, 16)
26     assert tables.group_matches.shape == (72, 7)
27     assert tables.base_predictions.shape == (72, 14)
28     assert tables.historical["match_id"].is_unique
29     assert tables.teams["team_id"].is_unique
30     assert tables.group_matches["match_id"].is_unique
31
32
33 def test_preserves_official_split_and_target_units() -> None:
34     tables = load_q1_tables()
35     history = tables.historical
36     train = history.loc[history["dataset_split"].eq("train")]
37     test = history.loc[history["dataset_split"].eq("test")]
38
39     assert len(train) == 560
40     assert len(test) == 140
41     assert train["tv_viewers"].notna().all()
42     assert test["tv_viewers"].isna().all()
43     assert pd.Timestamp(train["date"].min()) == pd.Timestamp("2018-01-12")
44     assert pd.Timestamp(train["date"].max()) == pd.Timestamp("2024-07-27")
45     assert pd.Timestamp(test["date"].min()) == pd.Timestamp("2024-08-05")
46     assert pd.Timestamp(test["date"].max()) == pd.Timestamp("2025-12-28")
47     assert 90_000_000 < train["tv_viewers"].min() < 100_000_000
48     assert 220_000_000 < train["tv_viewers"].max() < 230_000_000
49     target_millions = train["tv_viewers"].to_numpy() / 1_000_000
50     assert np.isclose(target_millions.mean(), 145.64310036428572)
51
52
53 def test_validates_cross_table_keys_and_template_columns() -> None:
54     tables = load_q1_tables()
55     report = validate_q1_tables(tables)
56

```

```

57     assert report.ok, report.errors
58     assert set(tables.group_matches["match_id"]) == set(tables.
base_predictions["match_id"])
59     assert list(tables.test_template.columns) == [
60         "match_id_test",
61         "predicted_test_tv_viewers",
62     ]
63     assert list(tables.match_template.columns) == [
64         "match_id",
65         "team_a",
66         "team_b",
67         "predicted_tv_viewers",
68     ]
69
70
71 def test_source_files_are_external_or_in_the_exact_bundle_directory() -> None:
72     paths = discover_source_paths()
73     project_root = Path(__file__).resolve().parents[1]
74     bundled_root = (project_root / "data" / "official").resolve()
75
76     for source in (paths.workbook, paths.test_template, paths.match_template):
77         resolved = source.resolve()
78         assert resolved.parent == bundled_root or project_root not in resolved
.parents

```

tests/test_q1_evaluation.py

```

1  from __future__ import annotations
2
3  import numpy as np
4
5  from c_contest_q1.data import load_q1_tables
6  from c_contest_q1.evaluation import (
7      DEFAULT_TEMPORAL_FOLDS,
8      evaluate_model_specs,
9      make_temporal_folds,
10 )
11 from c_contest_q1.features import build_historical_features,
target_in_millions
12 from c_contest_q1.models import demo_model_specs, sklearn_challenger_specs
13
14
15 def test_temporal_folds_preserve_complete_dates_and_order() -> None:
16     frame = build_historical_features(load_q1_tables())
17     folds = make_temporal_folds(frame.metadata)

```

```

18
19     assert len(folds) == len(DEFAULT_TEMPORAL_FOLDS)
20     for fold in folds:
21         train_dates = frame.metadata.iloc[fold.train_indices]["date"]
22         valid_dates = frame.metadata.iloc[fold.valid_indices]["date"]
23         assert train_dates.max() < valid_dates.min()
24         assert set(train_dates).isdisjoint(set(valid_dates))
25         assert len(fold.train_indices) > 100
26         assert len(fold.valid_indices) > 80
27
28
29 def test_demo_evaluation_returns_finite_metrics_and_oof_predictions() -> None:
30     frame = build_historical_features(load_q1_tables())
31     result = evaluate_model_specs(
32         frame.features,
33         frame.metadata,
34         target_in_millions(frame),
35         demo_model_specs(),
36     )
37
38     assert {"mean", "ridge", "hist_gradient_boosting"}.issubset(
39         set(result.leaderboard["model"])
40     )
41     assert np.isfinite(result.leaderboard[["mse", "rmse", "mae"]].to_numpy()).
42     all()
43     assert result.oof_predictions["prediction"].notna().all()
44     assert result.oof_predictions["actual"].notna().all()
45     assert result.oof_predictions["fold"].nunique() == len(
46         DEFAULT_TEMPORAL_FOLDS)
47
48
49 def test_sklearn_challengers_share_the_common_temporal_contract() -> None:
50     frame = build_historical_features(load_q1_tables())
51     result = evaluate_model_specs(
52         frame.features,
53         frame.metadata,
54         target_in_millions(frame),
55         sklearn_challenger_specs(),
56     )
57
58     assert set(result.leaderboard["model"]) == {
59         "elastic_net",
60         "svr_rbf",
61         "extra_trees",
62         "random_forest",

```



```

61     }
62     assert np.isfinite(result.leaderboard[["mse", "rmse", "mae"]].to_numpy()).
    all()

```

tests/test_q1_features.py

```

1  from __future__ import annotations
2
3  import numpy as np
4
5  from c_contest_q1.data import load_q1_tables
6  from c_contest_q1.features import (
7      FORBIDDEN_SOURCE_COLUMNS,
8      build_future_features,
9      build_historical_features,
10     target_in_millions,
11 )
12
13
14 def test_historical_and_future_share_exact_feature_contract() -> None:
15     tables = load_q1_tables()
16     historical = build_historical_features(tables)
17     future = build_future_features(tables)
18
19     assert list(historical.features.columns) == list(future.features.columns)
20     assert len(historical.features) == 700
21     assert len(future.features) == 72
22     assert historical.features.notna().all().all()
23     assert future.features.notna().all().all()
24
25
26 def test_probability_bridge_is_normalized_and_symmetric() -> None:
27     tables = load_q1_tables()
28     historical = build_historical_features(tables)
29     future = build_future_features(tables)
30
31     probability_columns = ["prob_low", "prob_mid", "prob_high"]
32     assert np.allclose(historical.features[probability_columns].sum(axis=1),
33                        1.0)
34     assert np.allclose(future.features[probability_columns].sum(axis=1), 1.0)
35     assert np.allclose(
36         future.features["prob_max"],
37         future.features["prob_high"],
38     )

```

```

39
40 def test_final_features_exclude_all_post_match_source_columns() -> None:
41     tables = load_q1_tables()
42     frame = build_historical_features(tables)
43
44     assert not set(FORBIDDEN_SOURCE_COLUMNS).intersection(frame.features.
45 columns)
46     assert "match_id" not in frame.features.columns
47     assert "team_a" not in frame.features.columns
48     assert "team_b" not in frame.features.columns
49     assert "tv_viewers" not in frame.features.columns
50
51 def test_target_is_explicitly_converted_from_people_to_millions() -> None:
52     tables = load_q1_tables()
53     historical = build_historical_features(tables)
54     y = target_in_millions(historical)
55
56     assert y.shape == (560,)
57     assert 140 < y.mean() < 150
58     assert 90 < y.min() < 100
59     assert 220 < y.max() < 230

```

tests/test_q1_outputs.py

```

1 from __future__ import annotations
2
3 import numpy as np
4 import pandas as pd
5
6 from c_contest_q1.data import load_q1_tables
7 from c_contest_q1.outputs import build_demo_outputs, validate_demo_outputs
8
9
10 def test_demo_outputs_follow_official_templates_and_units(tmp_path) -> None:
11     tables = load_q1_tables()
12     artifacts = build_demo_outputs(tables, tmp_path)
13     report = validate_demo_outputs(tables, artifacts.test_csv, artifacts.
14 match_csv)
15     test_prediction = pd.read_csv(artifacts.test_csv)
16     match_prediction = pd.read_csv(artifacts.match_csv)
17
18     assert report.ok, report.errors
19     assert list(test_prediction.columns) == list(tables.test_template.columns)
20     assert list(match_prediction.columns) == list(tables.match_template.

```

```

        columns)
20     assert len(test_prediction) == 140
21     assert len(match_prediction) == 72
22     assert np.isfinite(test_prediction["predicted_test_tv_viewers"]).all()
23     assert np.isfinite(match_prediction["predicted_tv_viewers"]).all()
24     assert test_prediction["predicted_test_tv_viewers"].between(50, 300).all()
25     assert match_prediction["predicted_tv_viewers"].between(50_000_000, 300
        _000_000).all()

```

tests/test_q2_demo.py

```

1  from __future__ import annotations
2
3  import numpy as np
4
5  from c_contest_q1.data import load_q1_tables
6  from c_contest_q1.q2_demo import build_q2_demo_schedule, evaluate_q2_schedule,
    validate_q2_schedule
7
8
9  def test_q2_schedule_is_feasible_and_has_a_finite_official_score() -> None:
10     root = __import__("pathlib").Path(__file__).resolve().parents[1]
11     tables = load_q1_tables()
12     schedule = build_q2_demo_schedule(tables, root / "outputs" / "q1" / "demo"
        / "result_1_match_prediction.csv")
13     validation = validate_q2_schedule(tables, schedule)
14     scored, report = evaluate_q2_schedule(tables, schedule)
15
16     assert validation.ok, validation.errors
17     assert len(scored) == 72
18     assert scored.travel_cost_index.notna().all()
19     assert np.isfinite(report["objective"])
20     assert scored.total_objective_value.notna().sum() == 1

```

tests/test_q3_demo.py

```

1  from __future__ import annotations
2
3  import numpy as np
4
5  from c_contest_q1.data import load_q1_tables
6  from c_contest_q1.q3_copt import build_q3_candidates
7  from c_contest_q1.q3_demo import build_q3_state
8
9

```

```

10 def test_q3_simulation_and_candidate_contract() -> None:
11     root = __import__("pathlib").Path(__file__).resolve().parents[1]
12     tables = load_q1_tables()
13     artifacts = build_q3_state(
14         tables,
15         root / "outputs" / "q2" / "copt" / "result_2_group_schedule.csv",
16         root / "outputs" / "q1" / "demo" / "result_1_match_prediction.csv",
17         n_simulations=100,
18     )
19     candidates, bounds = build_q3_candidates(tables, artifacts.matches)
20
21     assert len(artifacts.matches) == 24
22     assert np.isclose(artifacts.matches[["updated_p_team_a_advance", "
updated_p_team_b_advance"]].sum().sum(), 32.0)
23     assert artifacts.simulation_summary[["lambda_a", "lambda_b"]].ge(0.15).all
().all()
24     assert artifacts.simulation_summary[["lambda_a", "lambda_b"]].le(4.50).all
().all()
25     q3_scores = artifacts.simulation_summary[["importance", "
updated_attractiveness"]]
26     assert q3_scores.ge(0).all().all()
27     assert q3_scores.le(1).all().all()
28     assert artifacts.matches.collusion_risk.between(0, 1).all()
29     assert candidates.groupby("match_id").size().size == 24
30     assert set(bounds) == {"ticket_value", "broadcast_value", "attractiveness"
, "resource_cost", "risk"}

```

tests/test_q4_demo.py

```

1 from __future__ import annotations
2
3 import pandas as pd
4
5 from c_contest_q1.data import load_q1_tables
6 from c_contest_q1.q4_demo import compare_structural_schedule,
    validate_actual_schedule
7
8
9 def test_q4_actual_schedule_and_comparison_contract() -> None:
10     root = __import__("pathlib").Path(__file__).resolve().parents[1]
11     actual_path = root / "outputs" / "q4" / "actual_schedule.csv"
12     actual = pd.read_csv(actual_path)
13     assert not validate_actual_schedule(actual)
14     assert len(actual) == 72
15     assert pd.concat([actual.team_a, actual.team_b]).value_counts().eq(3).all

```

```

    ()
16     comparison = compare_structural_schedule(
17         load_q1_tables(),
18         actual_path,
19         root / "outputs" / "q2" / "copt" / "result_2_group_schedule.csv",
20     )
21     assert len(comparison) == 13
22     assert comparison.indicator_name.is_unique
23     assert comparison[["actual_schedule_value", "optimized_schedule_value"]].
notna().all().all()

```